

Deep Learning

**A Napkin-style companion for campus placements:
from the perceptron to Transformers and LLMs, with derivations,
hand numerics, traps, and code that checks itself**

Philospher

Built October 6, 2026

Preface

[STUB: preface] Versions: 2.14.1+cu130.

Contents

Preface	iii
I Foundations	9
1 Perceptron and the multilayer perceptron	11
1.1 The artificial neuron	11
1.2 The perceptron	12
1.3 XOR: what one neuron cannot do	14
1.4 The multilayer perceptron	16
1.5 Universal approximation, and why depth still matters	17
1.6 Capacity: what makes a network more expressive	19
1.7 A few problems to think about	20
2 Activations, losses and output layers	21
2.1 Activation functions	21
2.2 Output layers: sigmoid and softmax	23
2.3 Loss functions	25
2.4 A few problems to think about	28
3 Backpropagation	29
3.1 Computation graphs and the chain rule	29
3.2 Backpropagation for an MLP, in matrix form	30
3.3 A complete forward and backward pass by hand	31
3.4 Gradient checking	32
3.5 Reverse-mode autograd from scratch	33
3.6 Vanishing and exploding gradients	35
3.7 What backprop costs	36
3.8 A few problems to think about	36
4 Optimisers and learning-rate schedules	39
4.1 Gradient descent and its one big problem	39
4.2 Momentum and Nesterov	40
4.3 Adaptive methods: AdaGrad, RMSProp, Adam	40
4.4 Paths and curves	43
4.5 Learning-rate schedules	44
4.6 Batch size, clipping and the loss landscape	45
4.7 A few problems to think about	46
5 Initialisation, normalisation and regularisation	47
5.1 Initialisation	47
5.2 Batch normalisation	48
5.3 LayerNorm, GroupNorm, RMSNorm	51
5.4 Dropout	52
5.5 Weight decay and the λ in $E + \lambda \sum w^2$	53
5.6 Other regularisers, and reading the curves	54
5.7 A few problems to think about	55

II	Architectures	57
6	Convolutional networks	59
6.1	Why convolution	59
6.2	The convolution operation	59
6.3	Output shapes	61
6.4	Parameters and FLOPs	61
6.5	Receptive field	62
6.6	Pooling, 1x1 and depthwise convolutions	63
6.7	Architectures worth knowing	64
6.8	Transfer learning and fine-tuning	66
6.9	Detection and segmentation, in one section	67
6.10	A few problems to think about	68
7	Recurrent networks	69
7.1	The vanilla RNN	69
7.2	Backpropagation through time	70
7.3	LSTM and GRU	71
7.4	Sequence-to-sequence, teacher forcing and beam search	73
7.5	RNNs for time series, and where Transformers win	75
7.6	CTC, briefly	75
7.7	A few problems to think about	76
8	Attention and the Transformer	77
8.1	Attention as a soft dictionary lookup	77
8.2	Scaled dot-product attention	77
8.3	Multi-head attention, with shapes	80
8.4	Positional encodings	81
8.5	The Transformer block	82
8.6	Counting parameters and FLOPs	83
8.7	Transformer versus RNN and CNN	85
8.8	Capstone: an encoder–decoder Transformer, twice	86
8.9	A few problems to think about	90
III	Modern Deep Learning	91
9	Language models and LLMs	93
9.1	Tokens and embeddings	93
9.2	Objectives and model families	94
9.3	Scale	95
9.4	Adapting a pretrained model	95
9.5	Decoding	96
9.6	Evaluation and its limits	97
9.7	Making models cheaper: quantisation, distillation, MoE	98
9.8	Retrieval, tools and failure modes	99
9.9	A few problems to think about	99
10	Generative and representation learning	101
10.1	Autoencoders	101
10.2	Variational autoencoders	101

10.3	Generative adversarial networks	102
10.4	Diffusion models	103
10.5	Representation learning without labels	104
10.6	A few problems to think about	105
IV	Interview Mode	107
11	Practice and PyTorch	109
11.1	Tensors, modules and the training loop	109
11.2	The silent-bug catalogue	110
11.3	Debugging a model that does not learn	110
11.4	Scaling up: precision, memory, many GPUs	111
11.5	Three more families, one page each	112
11.6	A few problems to think about	113
12	Implement it	115
12.1	A framework in sixty lines	115
12.2	BatchNorm as a layer	116
12.3	Convolution via im2col	117
12.4	An LSTM layer	118
12.5	A character-level language model	119
12.6	A few problems to think about	120
13	Rapid-fire	121
13.1	Rapid-fire	121
14	Problems	123
14.1	Problems	123
V	Backmatter	125
A	Hints to problems	127
B	Solutions to problems	131
	Bibliography	137

I

Foundations

1 Perceptron and the multilayer perceptron

Every network in this book, from a two-layer classifier to a Transformer with a hundred layers, is built from one object: a weighted sum followed by a fixed non-linear function. This chapter takes that object apart, shows exactly what one of them can and cannot do (the XOR story that OAs keep asking about), and then stacks them into the multilayer perceptron whose parameters we learn to count in our sleep.

§1.1 The artificial neuron

Prototypical example for this section: Inputs $\mathbf{x} = (0.5, -1, 2)$, weights $\mathbf{w} = (0.4, 0.3, -0.2)$, bias $b = 0.1$: the neuron outputs $\phi(\mathbf{w}^\top \mathbf{x} + b)$ for some transfer function ϕ .

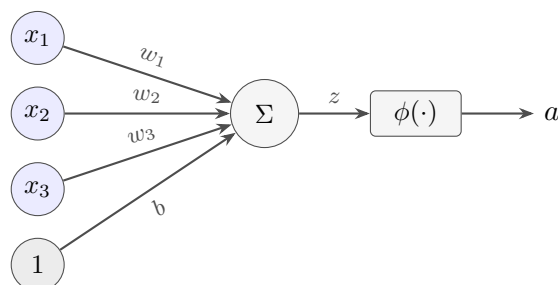
§1.1.i Definition

Definition 1.1.1. An **artificial neuron** with weights $\mathbf{w} \in \mathbb{R}^d$, bias $b \in \mathbb{R}$ and **activation** (or **transfer**) function $\phi : \mathbb{R} \rightarrow \mathbb{R}$ maps an input $\mathbf{x} \in \mathbb{R}^d$ to

$$a = \phi(z), \quad z = \mathbf{w}^\top \mathbf{x} + b = \sum_{i=1}^d w_i x_i + b.$$

The number z is the **pre-activation** (or weighted sum, or logit); a is the **activation**.

The classical choices of ϕ are the **step** (Heaviside) function $\mathbb{K}[z \geq 0]$, the **sigmoid** $\sigma(z) = 1/(1 + e^{-z})$, $\tanh z$ and the **ReLU** $\max(0, z)$. **Chapter 2** compares them properly; for now they are just four ways to squash or clip z .



Example 1.1.2 (One neuron, four transfer functions)

With the prototype numbers, $z = 0.4(0.5) + 0.3(-1) + (-0.2)(2) + 0.1 = -0.40$. Then

ϕ	step	sigmoid	tanh	ReLU
$a = \phi(z)$	0.0000	0.4013	-0.3799	0.0000

All four agree with `torch.sigmoid`, `torch.tanh` and `torch.relu` (checked in `code/dl/c01_perceptron.py`). OA version: inputs $(1, 0, 1, 1)$, weights $(0.6, -0.9, -0.4, 0.5)$, bias -0.3 , sigmoid transfer. Then $z = 0.40$ and the output is $\sigma(z) = 0.5987$. The input that is 0 contributes nothing whatever its weight,

which is exactly what the setter hopes you will forget.

Remark 1.1.3 — Trap. The neuron applies ϕ to the weighted sum; it does not take a weighted sum of $\phi(x_i)$. A second common slip is dropping the bias: then every decision boundary is forced through the origin.

§1.1.ii The biological analogy, and where it breaks

The name comes from a cartoon of a cortical neuron: dendrites collect signals from other cells, synapses scale them (the weights), the cell body integrates them (the sum), and the axon fires if the total crosses a threshold (the step function). The cartoon is useful for exactly one sentence. It breaks almost everywhere an interviewer might push:

- Real neurons communicate with spikes in time; information sits in rates and timing, not in one real number per forward pass.
- Dendrites themselves compute non-linear functions, so one biological neuron is closer to a small network than to one unit.
- Nobody has shown that the brain runs backpropagation: it would need the backward pass to reuse the forward weights exactly (“weight transport”).
- An artificial neuron with a sigmoid is literally logistic regression (see the logistic-regression chapter of the companion book *Classical Machine Learning*); the interesting behaviour comes from composing many of them and training them jointly.

A neuron is a linear function followed by a fixed scalar non-linearity. Everything else is composition.

§1.2 The perceptron

Prototypical example for this section: Six labelled points in the plane; Rosenblatt’s rule adds misclassified points to $\mathbf{w}$ until every point is on the right side.

§1.2.i The algorithm

Rosenblatt’s perceptron [Ro58] is a step-function neuron $\hat{y} = \text{sign}(\mathbf{w}^\top \mathbf{x} + b)$ with labels $y \in \{-1, +1\}$ and a learning rule that only reacts to mistakes.

Algorithm: Perceptron

Initialise $\mathbf{w} = \mathbf{0}$, $b = 0$. Repeat over the data until an epoch makes no mistake: if $y_i(\mathbf{w}^\top \mathbf{x}_i + b) \leq 0$ then $\mathbf{w} \leftarrow \mathbf{w} + \eta y_i \mathbf{x}_i$ and $b \leftarrow b + \eta y_i$.

Why it is the right direction: after the update the new margin of the same point is $y_i(\mathbf{w} + \eta y_i \mathbf{x}_i)^\top \mathbf{x}_i + y_i(b + \eta y_i) = y_i(\mathbf{w}^\top \mathbf{x}_i + b) + \eta(\|\mathbf{x}_i\|^2 + 1)$, strictly larger than before. The rule is stochastic (sub)gradient descent on the **perceptron loss** $\ell(\mathbf{w}, b) = \max(0, -y(\mathbf{w}^\top \mathbf{x} + b))$, whose gradient is $-y\mathbf{x}$ on a mistake and 0 otherwise.

code/dl/c01_perceptron.py

```

1 def perceptron(X, y, epochs=20, lr=1.0):
2     w, b = np.zeros(X.shape[1]), 0.0
3     log = []
4     for ep in range(epochs):
5         mistakes = 0
6         for i in range(len(X)):
7             if y[i] * (w @ X[i] + b) <= 0:      # mistake (or on the boundary)
8                 w, b = w + lr * y[i] * X[i], b + lr * y[i]
9                 mistakes += 1
10                log.append((ep + 1, i + 1, w.copy(), b))
11        if mistakes == 0:
12            return w, b, log, ep + 1
13    return w, b, log, epochs

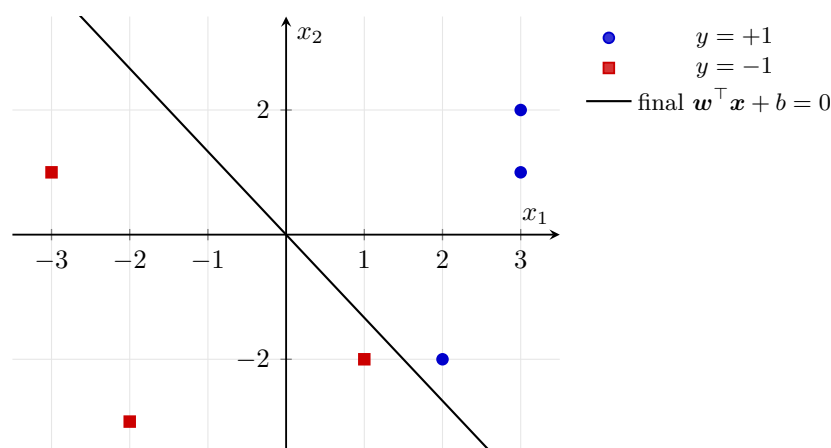
```

Example 1.2.1 (A full trace)

Data: $\mathbf{x}_1 = (3, 1)^+$, $\mathbf{x}_2 = (1, -2)^-$, $\mathbf{x}_3 = (3, 2)^+$, $\mathbf{x}_4 = (-2, -3)^-$, $\mathbf{x}_5 = (2, -2)^+$, $\mathbf{x}_6 = (-3, 1)^-$; $\eta = 1$, points visited in index order. Every update made:

epoch	mistake on	$\mathbf{w}$ after	b after
1	$\mathbf{x}_1$	(3, 1)	1
1	$\mathbf{x}_2$	(2, 3)	0
1	$\mathbf{x}_5$	(4, 1)	1
2	$\mathbf{x}_2$	(3, 3)	0
2	$\mathbf{x}_5$	(5, 1)	1
3	$\mathbf{x}_2$	(4, 3)	0

Epoch 4 makes no mistake, so the algorithm stops after 6 updates at $\mathbf{w} = (4, 3)$, $b = 0$. The result is identical to `sklearn.linear_model.Perceptron(eta0=1, shuffle=False)`, which the script asserts. Notice $\mathbf{x}_2$ and $\mathbf{x}_5$, the two points closest to each other with opposite labels, cause every update after the first.



Remark 1.2.2 — Trap. “Increase the learning rate to make the perceptron converge faster.” With $\mathbf{w}_0 = \mathbf{0}$, every weight vector the algorithm visits is η times the one it visits with $\eta = 1$, so the signs of all predictions, the mistakes and the number of updates are identical. The script reruns the trace with $\eta = 0.1$ and asserts the final

weights are exactly 0.1 times the ones above. The learning rate of the perceptron only matters when $\mathbf{w}_0 \neq \mathbf{0}$.

§1.2.ii The convergence theorem

Theorem 1.2.3 (Perceptron convergence)

Suppose $\|(\mathbf{x}_i, 1)\| \leq R$ for all i and some unit vector $\mathbf{w}^*$ (acting on augmented inputs $(\mathbf{x}, 1)$) satisfies $y_i \mathbf{w}^{*\top}(\mathbf{x}_i, 1) \geq \gamma > 0$ for all i . Then the perceptron makes at most $(R/\gamma)^2$ updates, whatever the order of the data.

Proof. Absorb b into $\mathbf{w}$ by appending 1 to every input, and take $\eta = 1$ (by the remark above this costs nothing). Let $\mathbf{w}_k$ be the weights after k updates, the k -th made on example i . Two inequalities:

- Progress: $\mathbf{w}_k^\top \mathbf{w}^* = \mathbf{w}_{k-1}^\top \mathbf{w}^* + y_i \mathbf{x}_i^\top \mathbf{w}^* \geq \mathbf{w}_{k-1}^\top \mathbf{w}^* + \gamma$, so $\mathbf{w}_k^\top \mathbf{w}^* \geq k\gamma$.
- Bounded growth: $\|\mathbf{w}_k\|^2 = \|\mathbf{w}_{k-1}\|^2 + 2y_i \mathbf{w}_{k-1}^\top \mathbf{x}_i + \|\mathbf{x}_i\|^2 \leq \|\mathbf{w}_{k-1}\|^2 + R^2$ because the update happened only when $y_i \mathbf{w}_{k-1}^\top \mathbf{x}_i \leq 0$. So $\|\mathbf{w}_k\|^2 \leq kR^2$.

By Cauchy–Schwarz $k\gamma \leq \mathbf{w}_k^\top \mathbf{w}^* \leq \|\mathbf{w}_k\| \leq \sqrt{k}R$, hence $k \leq (R/\gamma)^2$. $\square$

Example 1.2.4 (How loose is the bound?)

For the six points above, $R = 3.742$ and the best possible margin (from a hard-margin linear SVM, normalised) is $\gamma = 0.277$, so the bound is $(R/\gamma)^2 \approx 182.0$ updates. The perceptron needed 6. The bound is a worst case over orderings and is driven by the single tightest pair of points.

Question 1.2.5. Why does the rule update when $y_i(\mathbf{w}^\top \mathbf{x}_i + b) \leq 0$ rather than < 0 ? (Start from $\mathbf{w} = \mathbf{0}$.)

Remark 1.2.6 — Trap. The theorem says nothing about non-separable data, where the perceptron never stops, and nothing about *which* separator it returns: unlike an SVM it does not maximise the margin, and the answer depends on the order of the data.

§1.3 XOR: what one neuron cannot do

Prototypical example for this section: $\text{XOR}(x_1, x_2)$ is 1 exactly when one input is 1; no line separates $\{(0, 1), (1, 0)\}$ from $\{(0, 0), (1, 1)\}$.

Proposition 1.3.1

No single step-function neuron computes XOR.

Proof. Suppose $\mathbb{K}[w_1x_1 + w_2x_2 + b \geq 0]$ equals XOR. The four inputs give $b < 0$, $w_1 + w_2 + b < 0$, $w_2 + b \geq 0$ and $w_1 + b \geq 0$. Adding the last two, $w_1 + w_2 + 2b \geq 0$, so $w_1 + w_2 + b \geq -b > 0$, contradicting the second. $\square$

The same argument works for any monotone ϕ thresholded at any level, so “use a sigmoid instead” does not help: a single neuron’s decision boundary is always a hyperplane. This observation, made precise by Minsky and Papert [MiPa69], is the usual one-line answer to “why do we need hidden layers?”.

Example 1.3.2 (The perceptron on XOR)

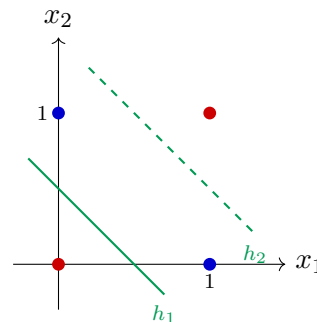
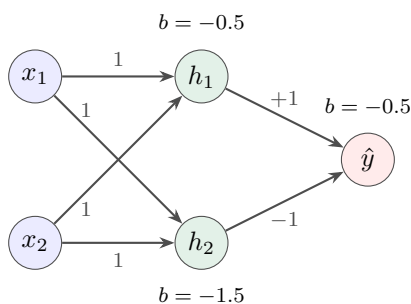
Running the perceptron for 50 epochs on XOR (labels ± 1) gives 200 updates in total and never fewer than 4 mistakes in an epoch: the weights cycle forever. `sklearn`’s perceptron ends at training accuracy 0.50, no better than a coin.

§1.3.i Two layers solve it by hand

XOR = OR and not AND. Each of OR and AND is linearly separable, and so is “ h_1 and not h_2 ” once h_1, h_2 are available as features:

code/dl/c01_perceptron.py

```
1 step = lambda t: (t >= 0).astype(float)
2 def xor_net(X):
3     h1 = step(X @ np.array([1, 1]) - 0.5)    # OR
4     h2 = step(X @ np.array([1, 1]) - 1.5)    # AND
5     return step(h1 - h2 - 0.5), h1, h2      # OR and not AND
```



x_1	x_2	$h_1 = \text{OR}$	$h_2 = \text{AND}$	$\hat{y}$
0	0	0	0	0
0	1	1	0	1
1	0	1	0	1
1	1	1	1	0

The hidden layer *re-represents* the input: in (h_1, h_2) coordinates the four points become $(0,0)$, $(1,0)$, $(1,0)$, $(1,1)$ and the two classes are linearly separable. That is the whole idea of representation learning in one table.

Example 1.3.3 (Can gradient descent find it?)

Train a 2- H -1 network with tanh hidden units and a sigmoid output on the four XOR points with Adam, from 20 random initialisations. With $H = 2$ (the minimum that can represent XOR) 10 of the 20 runs reach 100% accuracy; the rest stall in a poor local configuration. With $H = 8$, 20 of 20 succeed. Over-parameterising makes the optimisation problem easier, a theme that returns with every large model

in this book.

§1.4 The multilayer perceptron

Prototypical example for this section: A 784-256-128-10 ReLU network for MNIST digits has 235,146 parameters.

§1.4.i Forward pass and shapes

A **multilayer perceptron** (MLP, also **fully connected** or **dense** network) with layer widths $n_0, n_1, \dots, n_L$ computes

$$\mathbf{h}^{(0)} = \mathbf{x}, \quad \mathbf{z}^{(l)} = \mathbf{W}^{(l)} \mathbf{h}^{(l-1)} + \mathbf{b}^{(l)}, \quad \mathbf{h}^{(l)} = \phi(\mathbf{z}^{(l)}) \quad (l = 1, \dots, L),$$

with $\mathbf{W}^{(l)} \in \mathbb{R}^{n_l \times n_{l-1}}$ and $\mathbf{b}^{(l)} \in \mathbb{R}^{n_l}$. The last layer usually has no ϕ (it outputs logits; [chapter 2](#)). With a batch $\mathbf{X} \in \mathbb{R}^{B \times n_0}$ stored row-wise, as every framework does, the same layer reads $\mathbf{Z} = \mathbf{XW}^\top + \mathbf{1b}^\top$, shape (B, n_l) ; PyTorch's `nn.Linear(n_in, n_out)` stores `weight` with shape (n_out, n_in) for exactly this reason.

Proposition 1.4.1 (Parameter count)

An MLP with widths $n_0, \dots, n_L$ has $\sum_{l=1}^L (n_{l-1} + 1) n_l$ parameters: each of the n_l units in layer l has n_{l-1} weights and one bias.

Example 1.4.2 (Counting an MNIST MLP)

For $784 \rightarrow 256 \rightarrow 128 \rightarrow 10$: $785 \cdot 256 = 200,960$, $257 \cdot 128 = 32,896$, $129 \cdot 10 = 1,290$; total 235,146, equal to `sum(p.numel() for p in model.parameters())` for the PyTorch model. A 3-4-2 network has 26 parameters, a typical one-line OA item. Activation functions and the input layer have no parameters.

code/dl/c01_perceptron.py

```
1 def mlp_params(sizes):
2     return sum(m * n + n for m, n in zip(sizes[:-1], sizes[1:]))
3 sizes = [784, 256, 128, 10]
4 n_ours = mlp_params(sizes)
```

§1.4.ii Why the non-linearity is not optional

Delete every ϕ and the network collapses: $\mathbf{W}^{(2)}(\mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)}) + \mathbf{b}^{(2)} = (\mathbf{W}^{(2)}\mathbf{W}^{(1)})\mathbf{x} + (\mathbf{W}^{(2)}\mathbf{b}^{(1)} + \mathbf{b}^{(2)})$, a single affine map, however many layers you stack (the script checks this numerically). A deep linear network can represent only what one linear layer can, so it cannot solve XOR either.

Remark 1.4.3 — Trap. “ReLU is linear, so it adds no non-linearity.” ReLU is *piecewise* linear: it is linear on each side of 0 but not linear overall ($\text{ReLU}(-1) + \text{ReLU}(1) \neq \text{ReLU}(0)$). A ReLU network is a piecewise-linear function with possibly a huge number of pieces, which is exactly what the next section exploits.

§1.5 Universal approximation, and why depth still matters

Prototypical example for this section: Thirty-two ReLU units in one hidden layer approximate $\sin(2\pi t) + t/2$ on $[0, 1]$ to within 0.0048.

Theorem 1.5.1 (Universal approximation, informal)

Let ϕ be continuous and not a polynomial. For every continuous $f : K \rightarrow \mathbb{R}$ on a compact $K \subset \mathbb{R}^d$ and every $\varepsilon > 0$ there is a network with one hidden layer of ϕ units and a linear output whose output is within ε of f everywhere on K .

Cybenko proved it for sigmoids [Cy89]; later work extended it to any continuous non-polynomial activation. Two caveats make up the interview answer: the theorem says a suitable width *exists* but not how large it is (it can be exponential in d), and it says nothing about whether gradient descent will find the weights or whether they generalise.

§1.5.i A constructive proof in one dimension

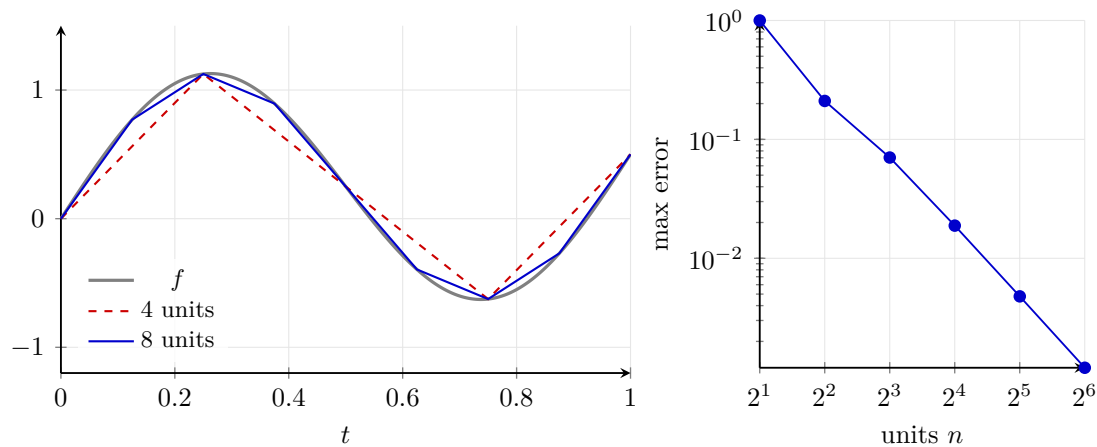
With ReLUs the 1-D case is a construction you can write in an interview. Take knots $0 = t_0 < t_1 < \dots < t_n = 1$ and the piecewise-linear interpolant of f . Its slope on $[t_k, t_{k+1}]$ is s_k . Then

$$\hat{f}(t) = f(0) + \sum_{k=0}^{n-1} a_k \operatorname{ReLU}(t - t_k), \quad a_0 = s_0, \quad a_k = s_k - s_{k-1},$$

because the unit $\operatorname{ReLU}(t - t_k)$ switches on at t_k and changes the slope by a_k there. That is a one-hidden-layer network with n units: hidden weights 1, hidden biases $-t_k$, output weights a_k .

code/dl/c01_perceptron.py

```
1 def relu_net_interp(f, n):
2     """One hidden ReLU layer with n units that interpolates f at n+1 knots."""
3     knots = np.linspace(0, 1, n + 1)
4     vals = f(knots)
5     slopes = np.diff(vals) / np.diff(knots)
6     a = np.r_[slopes[0], np.diff(slopes)] # output weights
7     def net(t):
8         H = np.maximum(0.0, t[:, None] - knots[:-1][None, :]) # hidden layer
9         return vals[0] + H @ a
10    return net
```



Example 1.5.2 (Error rate of the construction)

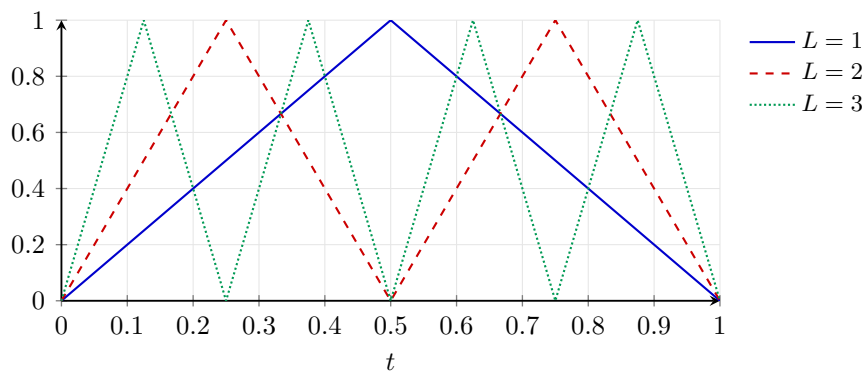
The script verifies that the network equals `numpy.interp` at 401 points and measures the worst error: 0.2105, 0.0704, 0.0188, 0.0048, 0.0012 for $n = 4, 8, 16, 32, 64$. Doubling n divides the error by about 3.98, matching the interpolation bound $h^2 \max |f''|/8$ with $h = 1/n$; for $n = 64$ that bound is 0.0012. In d dimensions the same grid argument needs n^d cells: the curse of dimensionality hiding inside “universal”.

§1.5.ii Depth buys pieces exponentially

A one-hidden-layer ReLU net on $\mathbb{R}$ with m units is piecewise linear with at most $m + 1$ pieces (each unit adds at most one kink). Now compose the tent map $g(t) = 2 \operatorname{ReLU}(t) - 4 \operatorname{ReLU}(t - \frac{1}{2})$, which uses 2 ReLU units and maps $[0, 1]$ onto $[0, 1]$ up and back down. Composing it L times gives a sawtooth.

code/dl/c01_perceptron.py

```
1 def tent_layer(t):          # 2 ReLU units: g(t) = 2 relu(t) - 4 relu(t - 1/2)
2     return 2 * np.maximum(0, t) - 4 * np.maximum(0, t - 0.5)
3 def deep_tent(t, L):
4     for _ in range(L):
5         t = tent_layer(t)
6     return t
```



Counting slope changes numerically, depth $L = 1, \dots, 6$ gives 2, 4, 8, 16, 32, 64 linear pieces, i.e. 2^L , using only $2L$ units. The depth-6 sawtooth uses 12 units; a single hidden layer needs at least 63 to produce 64 pieces. Depth multiplies, width adds.

One hidden layer is enough in principle; depth is what makes it enough in practice.

§1.6 Capacity: what makes a network more expressive

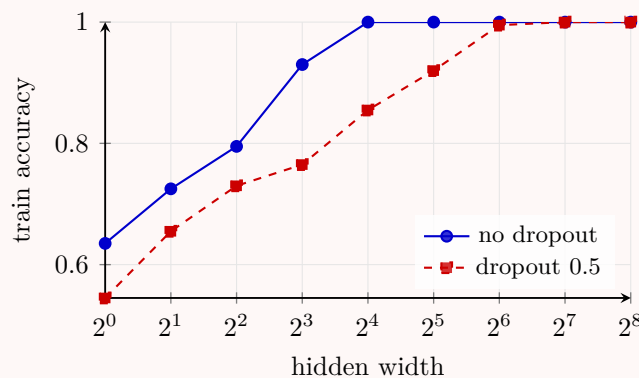
Prototypical example for this section: “Which of these increases a network’s capacity: more layers, dropout, a larger learning rate, more data?” Only the first.

Capacity is the richness of the set of functions the architecture can represent (its hypothesis class). It is a property of the model, not of the data or of the optimiser.

- **Increase capacity:** more layers, more units per layer, more input features, a richer activation, fewer constraints (e.g. removing weight sharing).
- **Reduce effective capacity:** dropout, weight decay, early stopping, data augmentation, weight sharing, bottleneck layers. These restrict which functions training can reach or prefers.
- **Do not change capacity:** the learning rate, the batch size, the number of epochs as such, and the amount of data. They change what training *finds*, not what the model *can* represent.

Example 1.6.1 (Memorising random labels)

Give 200 random points in $\mathbb{R}^{10}$ coin-flip labels, so the only way to fit them is to memorise. Train one-hidden-layer ReLU nets of increasing width to convergence. Training accuracy rises from 63.5% at width 1 to 100.0% at width 16. With dropout $p = 0.5$ the width-16 net reaches only 85.5%: the same architecture, lower effective capacity.



Remark 1.6.2 — Trap. “Dropout increases capacity because it trains an ensemble.” Dropout trains an implicit ensemble of thinned sub-networks that share weights; each sub-network is smaller than the full one, and the averaged predictor is pulled toward simpler functions. It is a regulariser: it lowers effective capacity (chapter 5).

Remark 1.6.3 — Trap. “A bigger learning rate lets the model fit more complex functions.” It changes the path of optimisation, sometimes for the better (noise that escapes sharp minima), but the set of representable functions is fixed by the

architecture.

§1.7 A few problems to think about

Problem 1A. How many trainable parameters does a fully connected network with layer sizes $100 \rightarrow 50 \rightarrow 50 \rightarrow 3$ have (biases included)?

[hint · solution](#)

Problem 1B. Give weights and a bias for a single step neuron computing NAND on $\{0, 1\}^2$. Why does this show that networks of step neurons can compute any Boolean function?

[hint · solution](#)

Problem 1C. A perceptron *without bias* is trained from $\mathbf{w} = \mathbf{0}$ on linearly separable data. Every input is then multiplied by $c > 0$ and training is rerun. How do R , γ , the bound $(R/\gamma)^2$ and the sequence of mistakes change?

[hint · solution](#)

Problem 1D. Show that the one-hidden-layer ReLU network on $\mathbb{R}$

$$f(t) = c + \sum_{k=1}^m a_k \text{ReLU}(w_k t + b_k)$$

is piecewise linear with at most $m + 1$ pieces.

[hint · solution](#)

Problem 1E. Which statements are true? (a) A single sigmoid neuron can represent XOR. (b) Stacking linear layers without activations increases representational power. (c) Adding hidden layers increases capacity. (d) Dropout increases capacity. (e) A one-hidden-layer network can approximate any continuous function on a compact set, given enough units.

[hint · solution](#)

Problem 1F. Prove that the perceptron update never decreases $\mathbf{w}^\top \mathbf{w}^*$ and explain in one sentence why that alone does not prove convergence.

[hint · solution](#)

2 Activations, losses and output layers

Three choices sit at fixed places in every network: the non-linearity between layers, the function that turns the last layer’s numbers into a prediction, and the loss that scores that prediction. OAs test all three with one-minute numerics (“compute the softmax of these five logits”) and with classification questions (“which of these is a loss and which is an activation?”). Interviews test whether you know why the standard pairings are standard.

§2.1 Activation functions

Prototypical example for this section: Sigmoid, tanh, ReLU and GELU evaluated at $z = -2$ give four different answers, and the derivative of the sigmoid is never larger than 0.25.

§2.1.i What the activation is for

Chapter 1 showed that without an activation a deep network collapses to one affine map. The activation is the *only* source of non-linearity in an MLP: weights and biases enter linearly, and the loss is applied after the network, so it cannot change what functions the network represents.

Remark 2.1.1 — Trap. “Which component provides non-linearity: weights, biases, activation function, loss function?” Only the activation function. A non-linear *loss* (cross-entropy is very non-linear) does not help: a linear model trained with cross-entropy is still logistic regression, with a linear boundary.

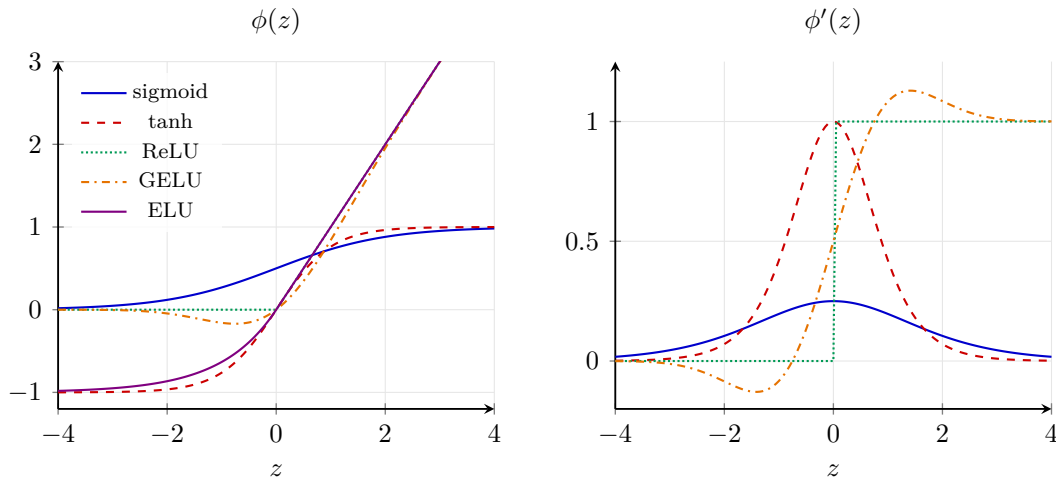
§2.1.ii The zoo, with values

name	$\phi(z)$	$\phi'(z)$
sigmoid	$\sigma(z) = 1/(1 + e^{-z})$	$\sigma(z)(1 - \sigma(z)) \leq \frac{1}{4}$
tanh	$(e^z - e^{-z})/(e^z + e^{-z}) = 2\sigma(2z) - 1$	$1 - \tanh^2 z \leq 1$
ReLU	$\max(0, z)$	$\mathbb{K}[z > 0]$
leaky ReLU	$\max(\alpha z, z), \alpha \approx 0.01$	1 or α
ELU	z if $z > 0$, else $\alpha(e^z - 1)$	1 or αe^z
GELU	$z \Phi(z)$, Φ the standard normal CDF	$\Phi(z) + z \varphi(z)$
SiLU / swish	$z \sigma(z)$	$\sigma(z)(1 + z(1 - \sigma(z)))$
softplus	$\log(1 + e^z)$	$\sigma(z)$

Example 2.1.2 (Values an OA might ask for)

Computed by `code/dl/c02_activations.py`, each function checked against `torch.nn.functional` on a grid of 161 points and each derivative against autograd:

ϕ	$z = -2$	-0.5	0	0.5	2
sigmoid	0.1192	0.3775	0.5000	0.6225	0.8808
tanh	-0.9640	-0.4621	0.0000	0.4621	0.9640
ReLU	0.0000	0.0000	0.0000	0.5000	2.0000
leaky ReLU (0.01)	-0.0200	-0.0050	0.0000	0.5000	2.0000
ELU ($\alpha = 1$)	-0.8647	-0.3935	0.0000	0.5000	2.0000
GELU	-0.0455	-0.1543	0.0000	0.3457	1.9545
SiLU	-0.2384	-0.1888	0.0000	0.3112	1.7616



§2.1.iii Sigmoid and tanh: saturation

Both squash $\mathbb{R}$ into a bounded interval, and both are nearly flat away from 0: $\sigma'(5) = 0.0066$. Backpropagation multiplies one such factor per layer (chapter 3), and since $\sigma' \leq \frac{1}{4}$ ten sigmoid layers shrink a gradient by at least 4^{10} , i.e. to at most 9.54×10^{-7} of its size from that factor alone. This is the **vanishing gradient** problem in its simplest form.

The sigmoid has a second defect: its outputs are all positive. For a neuron fed by sigmoid units, $\partial \mathcal{L} / \partial w_i = \delta \cdot h_i$ with every $h_i > 0$, so all of that neuron's weight gradients share the sign of δ and the weights can only move together up or together down, giving zig-zag updates. tanh is zero-centred and its maximum slope is 1, so it was preferred for hidden layers even before ReLU. Today sigmoid is used at outputs (probabilities) and in gates (LSTM, chapter 7), almost never in hidden layers.

§2.1.iv ReLU and its relatives

ReLU does not saturate for $z > 0$ (derivative exactly 1), is cheap, and produces exact zeros (sparse activations). Its failure mode is the **dead ReLU**: a unit whose pre-activation is negative on every training input gets zero gradient forever (chapter 3 has a simulation). Leaky ReLU and PReLU (learned α) keep a small slope for $z < 0$; ELU makes the negative side smooth and pushes mean activations toward zero.

Remark 2.1.3 — Trap. “ReLU is not differentiable at 0, so it cannot be trained by gradient descent.” The kink is a single point that floating-point inputs essentially never hit exactly; frameworks use the subgradient 0 there, and ReLU is the default

hidden activation of most CNNs.

§2.1.v Smooth modern activations: GELU and SiLU

GELU weights the input by the probability that a standard normal falls below it, $z \Phi(z)$; Transformers (BERT, GPT-style models) use it or SiLU in their feed-forward blocks [HeGi16]. The common approximation $\frac{1}{2}z(1 + \tanh(\sqrt{2/\pi}(z + 0.044715z^3)))$ is within 0.00047 of the exact GELU on $[-6, 6]$. Both are *non-monotone*: SiLU has a minimum -0.278 at $z = -1.278$, GELU a minimum -0.170 at $z = -0.752$ (found numerically). They behave like ReLU for large $|z|$ but pass small negative signals and have a continuous derivative.

Hidden layers: ReLU for CNNs and MLPs, GELU/SiLU for Transformers, tanh and sigmoid only inside recurrent cells and gates. Output layers: chosen by the task, next section.

§2.2 Output layers: sigmoid and softmax

Prototypical example for this section: Logits $\mathbf{z} = (2, 1, 0.1, -1, 0.5)$ for five classes; the softmax gives class 1 probability 0.5585.

§2.2.i Softmax

For K classes the network outputs **logits** $\mathbf{z} \in \mathbb{R}^K$ and

$$\text{softmax}(\mathbf{z})_k = \frac{e^{z_k}}{\sum_{j=1}^K e^{z_j}}.$$

It is positive and sums to 1, preserves the order of the logits, and is invariant to adding a constant to every logit (multiply top and bottom by e^{-c}), but not to scaling them.

Example 2.2.1 (Softmax confidence from five logits)

For $\mathbf{z} = (2, 1, 0.1, -1, 0.5)$: $e^{\mathbf{z}} = (7.3891, 2.7183, 1.1052, 0.3679, 1.6487)$ with sum 13.2291, so

$$\text{softmax}(\mathbf{z}) = (0.5585, 0.2055, 0.0835, 0.0278, 0.1246).$$

The model predicts class 1 with confidence 55.9%. Note that a logit gap of 1 (2 vs 1) is a probability ratio of $e \approx 2.72$, not a difference of 1 in anything.

§2.2.ii Overflow and the log-sum-exp trick

e^{1000} overflows double precision, so the textbook formula applied to $\mathbf{z} = (1000, 1001, 1002)$ returns *NaN* for every entry. The fix uses shift invariance: subtract $m = \max_k z_k$ first. Then every exponent is ≤ 0 , the largest term is exactly 1, and nothing overflows. The same trick gives a stable **log-sum-exp**

$$\text{LSE}(\mathbf{z}) = \log \sum_k e^{z_k} = m + \log \sum_k e^{z_k - m},$$

which is what a cross-entropy implementation actually needs.

code/dl/c02_activations.py

```

1 def softmax(z):
2     z = z - z.max(axis=-1, keepdims=True)          # shift: same result, no overflow
3     e = np.exp(z)
4     return e / e.sum(axis=-1, keepdims=True)
5
6 def logsumexp(z):
7     m = z.max(axis=-1, keepdims=True)
8     return (m + np.log(np.exp(z - m).sum(axis=-1, keepdims=True))).squeeze(-1)
9
10 def cross_entropy(z, y):
11     # z: (B, K) logits, y: (B,) ints
    return np.mean(logsumexp(z) - z[np.arange(len(y)), y])

```

For (1000, 1001, 1002) the stable version returns (0.090, 0.245, 0.665), identical to $\text{softmax}(0, 1, 2)$, and $\text{LSE} = 1002.4076$; both match PyTorch’s `softmax` and SciPy’s `logsumexp`.

§2.2.iii Temperature

Dividing logits by a **temperature** $T > 0$ before the softmax sharpens ($T < 1$) or flattens ($T > 1$) the distribution without changing the arg-max. For the five logits above (last column: entropy in nats; the uniform distribution has $\ln 5 = 1.609$):

T	p_1	p_2	p_3	p_4	p_5	$H(p)$
0.5	0.826	0.112	0.018	0.002	0.041	0.620
1	0.559	0.205	0.084	0.028	0.125	1.217
2	0.372	0.226	0.144	0.083	0.176	1.495
5	0.264	0.216	0.180	0.145	0.195	1.590

As $T \rightarrow 0$ the softmax becomes a one-hot arg-max, as $T \rightarrow \infty$ it becomes uniform. Temperature reappears in knowledge distillation and LLM sampling (chapter 9).

§2.2.iv Sigmoid is two-class softmax

$\text{softmax}(a, b)_1 = e^a / (e^a + e^b) = 1 / (1 + e^{-(a-b)}) = \sigma(a - b)$ (checked numerically). So a binary classifier needs one logit and a sigmoid; two logits and a softmax is equivalent but over-parameterised.

§2.2.v The softmax Jacobian

With $\mathbf{p} = \text{softmax}(\mathbf{z})$, $\partial p_i / \partial z_j = p_i(\delta_{ij} - p_j)$, i.e. $J = \text{diag}(\mathbf{p}) - \mathbf{p}\mathbf{p}^\top$ (verified against `torch.autograd.functional.jacobian`). Rows sum to zero because the outputs always sum to 1. In practice you never form J : combined with cross-entropy it collapses to the gradient $\mathbf{p} - \mathbf{y}$ derived below.

Remark 2.2.2 — Trap. Multi-label is not multi-class. If an image can contain both a cat and a dog, use K independent sigmoids with K binary cross-entropies. A softmax forces the probabilities to compete and sum to 1, so “cat 0.9, dog 0.8” is impossible under it.

§2.3 Loss functions

Prototypical example for this section: True class 1 for the logits above: cross-entropy $-\ln 0.5585 = 0.5824$ nats.

§2.3.i Cross-entropy is maximum likelihood

If the network outputs a distribution $p_\theta(\cdot | \mathbf{x})$ over K classes and the data are independent, the negative log-likelihood of the labels is

$$-\sum_i \log p_\theta(y_i | \mathbf{x}_i) = \sum_i (\text{LSE}(\mathbf{z}_i) - z_{i,y_i}),$$

the **categorical cross-entropy** between the one-hot label and $\mathbf{p}$. Minimising it is minimising the KL divergence from the empirical label distribution to the model (the information-theory section of the companion book *Classical Machine Learning* has the identity $H(q, p) = H(q) + \text{KL}(q||p)$).

Proposition 2.3.1 (Gradient of softmax + cross-entropy)

For one example with logits $\mathbf{z}$, one-hot label $\mathbf{y}$ and $\mathcal{L} = -\sum_k y_k \log \text{softmax}(\mathbf{z})_k$,

$$\frac{\partial \mathcal{L}}{\partial \mathbf{z}} = \text{softmax}(\mathbf{z}) - \mathbf{y}.$$

Proof. $\mathcal{L} = \text{LSE}(\mathbf{z}) - \mathbf{z}^\top \mathbf{y}$ because $\sum_k y_k = 1$. The gradient of LSE is the softmax, and the gradient of $\mathbf{z}^\top \mathbf{y}$ is $\mathbf{y}$. $\square$

For [example 2.2.1](#) with true class 1 the gradient, identical to autograd's, is

$$\frac{\partial \mathcal{L}}{\partial \mathbf{z}} = (-0.4415, 0.2055, 0.0835, 0.0278, 0.1246).$$

The gradient pushes the true logit up by $1 - p_y$ and each wrong logit down by its probability: big mistakes get big corrections, and the gradient never saturates while the prediction is wrong.

Example 2.3.2 (Cross-entropy from logits, many cases)

Each row is checked against `F.cross_entropy`. Classes are numbered from 0 here, as in code.

logits $\mathbf{z}$	y	$\text{softmax}(\mathbf{z})$	pred	p_y	$\text{CE} = -\ln p_y$
(3, 1, 0.2)	0	(0.836, 0.113, 0.051)	0	0.836	0.179
(0, 0, 0, 0)	2	(0.250, 0.250, 0.250, 0.250)	0	0.250	1.386
(2, 2, -1)	1	(0.488, 0.488, 0.024)	0	0.488	0.718
(5, -2, 0, 1)	1	(0.975, 0.001, 0.007, 0.018)	0	0.001	7.026
(1, 2, 3, 4, 5)	0	(0.012, 0.032, 0.086, 0.234, 0.636)	4	0.012	4.452
(-1, 0.5, 0.5, 2.5, 0)	3	(0.022, 0.098, 0.098, 0.723, 0.059)	3	0.723	0.324
(10, 0)	0	(1.000, 0.000)	0	1.000	0.000

Patterns worth memorising: equal logits give $\text{CE} = \ln K$ (row 2); a confident wrong prediction is punished hard (row 4: the true class has probability 0.001); the loss of a correct prediction is small but never exactly 0 (last row); with the same probabilities the loss of predicting the wrong class grows by the logit gap (CE for

class 4 in [example 2.2.1](#) is $3.5824 = 0.5824 + 3$).

Remark 2.3.3 — Trap. Sanity check at initialisation: with small random weights the logits are nearly equal, so the loss of a K -class classifier starts near $\ln K$. For a batch of 64 random 10-class logits of scale 0.01 the script measures 2.3014 against $\ln 10 = 2.3026$. A first loss far from $\ln K$ means a bug (wrong labels, softmax applied twice, a huge initial scale).

§2.3.ii Binary cross-entropy and its stable form

For $y \in \{0, 1\}$ and $p = \sigma(z)$, $\text{BCE} = -[y \log p + (1 - y) \log(1 - p)]$: 0.1054 for $p = 0.9$, $y = 1$; 2.3026 for $p = 0.1$, $y = 1$; $\ln 2 = 0.6931$ at $p = 0.5$; and 4.6052 for a 0.99 prediction of the wrong class. Computing σ first and then log breaks for large $|z|$: at $z = -1000$, $y = 1$ the sigmoid underflows to 0 and the loss becomes ∞ . Working from the logit,

$$\text{BCE}(z, y) = \max(z, 0) - zy + \log(1 + e^{-|z|}),$$

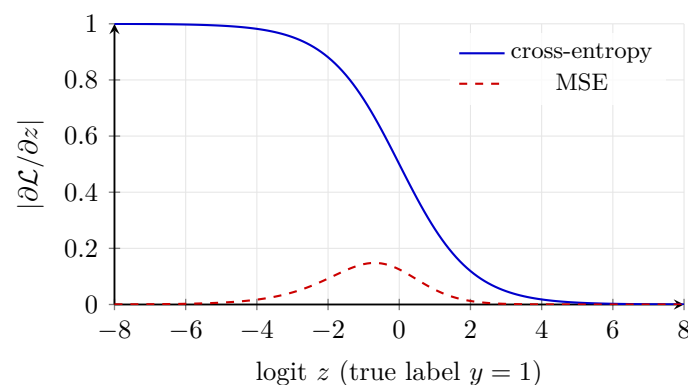
gives the correct 1000.0. This is `BCEWithLogitsLoss`, and the reason you pass logits, not probabilities, to it.

code/dl/c02_activations.py

```
1 def bce_with_logits(z, y):          # = -[y log s(z) + (1-y) log(1-s(z))], stable
2     return np.maximum(z, 0) - z * y + np.log1p(np.exp(-np.abs(z)))
```

§2.3.iii Why not MSE for classification?

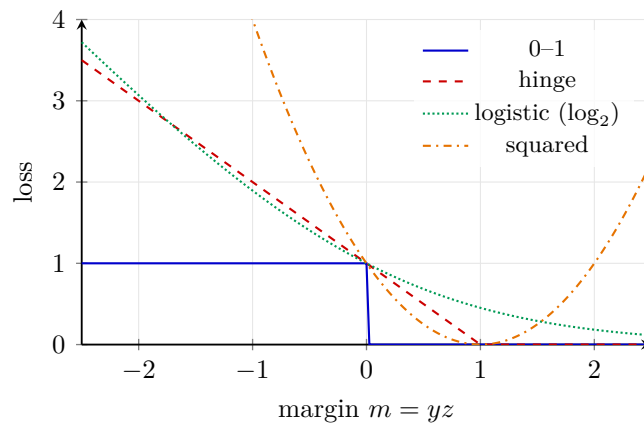
With a sigmoid output and squared error $\frac{1}{2}(\sigma(z) - y)^2$, the gradient is $(\sigma(z) - y) \sigma'(z)$. When the prediction is confidently wrong ($y = 1$, $z = -5$) the factor $\sigma'(z)$ kills it: 0.0066 for MSE against 0.9933 for cross-entropy. The model that is most wrong learns slowest under MSE. Cross-entropy's log exactly cancels the sigmoid's saturation. (MSE on a sigmoid is also non-convex in z ; CE is convex.)



§2.3.iv Margin losses: hinge

With labels $y \in \{-1, +1\}$ and score z , every binary loss is a function of the **margin** $m = yz$. The 0–1 loss $\mathbb{I}[m \leq 0]$ is what we care about but has zero gradient almost everywhere; the **hinge** $\max(0, 1 - m)$ (SVMs) and the logistic loss $\log(1 + e^{-m})$ are convex

upper bounds (the logistic in base 2 so that it passes through $(0, 1)$). The multiclass hinge sums $\max(0, z_j - z_y + 1)$ over wrong classes j ; for [example 2.2.1](#) with true class 2 it is 2.60 (`F.multi_margin_loss` divides by K).



The squared loss punishes *correct* predictions with margin above 1: another reason it is a poor classification loss.

§2.3.v Regression: MSE, MAE, Huber

MSE is the Gaussian negative log-likelihood and is dominated by outliers; MAE (Laplace likelihood) is robust but has a constant-size gradient that does not shrink near the optimum. [Huber](#) interpolates: quadratic within δ , linear outside.

code/dl/c02_activations.py

```
1 def huber(r, delta=1.0):
2     a = np.abs(r)
3     return np.where(a <= delta, 0.5 * r ** 2, delta * (a - 0.5 * delta))
```

With $\delta = 1$, a residual of 10 costs 9.5 under Huber and 50.0 under $\frac{1}{2}r^2$; a residual of 0.2 costs 0.02 under both (matches `F.huber_loss`).

§2.3.vi Focal loss and label smoothing

[Focal loss](#) [Li+17] multiplies cross-entropy by $(1 - p_y)^\gamma$ to down-weight easy, well-classified examples, which dominate the loss when one class (background) vastly outnumbers the rest:

p_y	CE	focal ($\gamma = 2$)	factor $(1 - p_y)^2$
0.9	0.1054	0.0011	0.01
0.5	0.6931	0.1733	0.25
0.1	2.3026	1.8651	0.81

[Label smoothing](#) with parameter ε replaces the one-hot target by $(1 - \varepsilon)\mathbf{y} + \varepsilon/K$; with $\varepsilon = 0.1$, $K = 5$ the target is 0.92 on the true class and 0.02 elsewhere. The model can no longer drive the loss to 0 by pushing one logit to infinity, which curbs over-confidence. For [example 2.2.1](#) the smoothed loss is 0.7304 (`F.cross_entropy(..., label_smoothing=0.1)` agrees).

§2.3.vii Which is a loss, which is an activation?

activations	output maps	losses
ReLU, leaky ReLU, ELU, GELU, SiLU, tanh, sigmoid (hidden)	sigmoid, softmax, identity (linear output)	MSE, MAE, Huber, BCE, CE, hinge, focal, KL divergence, contrastive/triplet

Sigmoid and softmax appear in two columns because they are used both as squashing functions and as output maps; they are never losses.

task	output layer	loss
regression	linear (identity)	MSE / MAE / Huber
binary classification	1 logit, sigmoid	BCE (with logits)
multi-class (one label)	K logits, softmax	categorical CE
multi-label	K logits, K sigmoids	sum of K BCEs
counts	exponential (log-rate)	Poisson NLL

Remark 2.3.4 — Trap. In PyTorch, `nn.CrossEntropyLoss` takes *logits* and applies log-softmax internally. Putting an `nn.Softmax` before it computes softmax twice; training still runs but gradients shrink and the loss floor is wrong. The same holds for `BCEWithLogitsLoss` after a sigmoid (chapter 11 lists this among the silent bugs).

§2.4 A few problems to think about

Problem 2A. A 4-class classifier outputs logits $(1, 1, 1, 1)$. What is the cross-entropy loss? What if the logits are $(101, 101, 101, 101)$?

hint · solution

Problem 2B. Show $\tanh z = 2\sigma(2z) - 1$ and deduce $\tanh'(0) = 4\sigma'(0)$.

hint · solution

Problem 2C. Derive $\partial \text{BCE} / \partial z = \sigma(z) - y$ for $\text{BCE} = -[y \log \sigma(z) + (1 - y) \log(1 - \sigma(z))]$, and verify that the stable form $\max(z, 0) - zy + \log(1 + e^{-|z|})$ equals BCE for both signs of z .

hint · solution

Problem 2D. Which are loss functions? (a) ReLU (b) hinge (c) softmax (d) Huber (e) cross-entropy (f) GELU (g) KL divergence.

hint · solution

Problem 2E. A softmax classifier predicts $p_y = 0.2$ for the true class. By how much must the true logit increase, other logits fixed, to halve the loss? Give a formula, then evaluate for $p_y = 0.2$.

hint · solution

Problem 2F. A team trains an image tagger where each image may have several of 20 tags, with a softmax output and categorical cross-entropy. What goes wrong, and what is the fix?

hint · solution

3 Backpropagation

Backpropagation [RuHiWi86] is the chain rule applied to a computation graph in the order that makes it cheap: from the loss backwards, reusing every intermediate product once. Interviews ask you to derive it for an MLP in matrix form, to run one step by hand on a tiny network, and increasingly to write the autograd engine itself. This chapter does all three, then uses the same algebra to explain why gradients vanish or explode with depth.

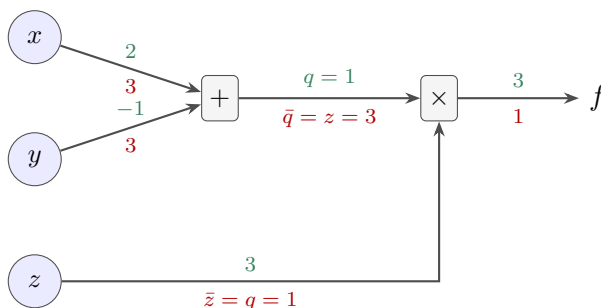
§3.1 Computation graphs and the chain rule

Prototypical example for this section: $f = (x + y)z$ at $x = 2, y = -1, z = 3$: forward gives $q = x + y = 1, f = 3$; backward gives $\partial f / \partial x = \partial f / \partial y = z = 3$ and $\partial f / \partial z = q = 1$.

A **computation graph** is a directed acyclic graph whose nodes are elementary operations and whose edges carry values. To differentiate a scalar output f with respect to every node, define the **adjoint** $\bar{v} = \partial f / \partial v$. For a node v that feeds nodes $u_1, \dots, u_m$,

$$\bar{v} = \sum_{j=1}^m \bar{u}_j \frac{\partial u_j}{\partial v}.$$

Each node needs only its **local gradient** $\partial u / \partial v$; adjoints arrive from above, are multiplied by the local gradient, and are *summed* over all paths (fan-out).



Green numbers are forward values, red are adjoints. The add node *routes* its incoming gradient unchanged to both inputs; the multiply node *swaps*: each input receives the upstream gradient times the other input. A max node routes the whole gradient to the winning input and zero to the rest (that is how max-pooling backpropagates, [chapter 6](#)).

§3.1.i Forward mode versus reverse mode

For $f : \mathbb{R}^n \rightarrow \mathbb{R}^m$ the chain rule can be evaluated input-to-output (**forward mode**: one pass per input direction, computing a Jacobian–vector product Jv) or output-to-input (**reverse mode**: one pass per output, computing a vector–Jacobian product $u^\top J$). A loss has $m = 1$ and n in the millions or billions, so reverse mode gets the entire gradient for the cost of about one extra forward pass, while forward mode would need n passes. Backpropagation is reverse-mode automatic differentiation specialised to neural networks.

Remark 3.1.1 — Trap. Backprop is not numerical differentiation and not symbolic differentiation. Numerical differences cost one pass per parameter and are inexact; symbolic expressions blow up in size. Reverse mode is exact up to floating point and costs a small constant times the forward pass, but it must *store* the forward activations, which is why training needs far more memory than inference.

§3.2 Backpropagation for an MLP, in matrix form

Prototypical example for this section: For a ReLU MLP with softmax output, $\delta^{(L)} = \mathbf{p} - \mathbf{y}$ and each layer sends $\delta^{(l)} = (\mathbf{W}^{(l+1)\top} \delta^{(l+1)}) \odot \text{ReLU}'(\mathbf{z}^{(l)})$ back.

Use the notation of chapter 1: $\mathbf{z}^{(l)} = \mathbf{W}^{(l)} \mathbf{h}^{(l-1)} + \mathbf{b}^{(l)}$, $\mathbf{h}^{(l)} = \phi(\mathbf{z}^{(l)})$, $\mathbf{h}^{(0)} = \mathbf{x}$, loss $\mathcal{L}$ on the output logits $\mathbf{z}^{(L)}$. Define the **error signal** $\delta^{(l)} = \partial \mathcal{L} / \partial \mathbf{z}^{(l)}$.

Theorem 3.2.1 (Backpropagation equations)

For one example,

$$\begin{aligned} \delta^{(L)} &= \frac{\partial \mathcal{L}}{\partial \mathbf{z}^{(L)}} \quad (= \text{softmax}(\mathbf{z}^{(L)}) - \mathbf{y} \text{ for softmax + CE}), \\ \delta^{(l)} &= (\mathbf{W}^{(l+1)\top} \delta^{(l+1)}) \odot \phi'(\mathbf{z}^{(l)}), \quad l = L-1, \dots, 1, \\ \frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(l)}} &= \delta^{(l)} \mathbf{h}^{(l-1)\top}, \quad \frac{\partial \mathcal{L}}{\partial \mathbf{b}^{(l)}} = \delta^{(l)}. \end{aligned}$$

Proof. $z_j^{(l+1)} = \sum_i W_{ji}^{(l+1)} \phi(z_i^{(l)}) + b_j$, so $\partial z_j^{(l+1)} / \partial z_i^{(l)} = W_{ji}^{(l+1)} \phi'(z_i^{(l)})$. The chain rule summed over all j gives $\delta_i^{(l)} = \phi'(z_i^{(l)}) \sum_j W_{ji}^{(l+1)} \delta_j^{(l+1)}$, the second line. Since $z_j^{(l)}$ depends on $W_{ji}^{(l)}$ only through the term $W_{ji}^{(l)} h_i^{(l-1)}$, $\partial \mathcal{L} / \partial W_{ji}^{(l)} = \delta_j^{(l)} h_i^{(l-1)}$, an outer product; the bias has local gradient 1. $\square$

With a batch stored row-wise ($\mathbf{H}^{(l)} \in \mathbb{R}^{B \times n_l}$, $\Delta^{(l)} \in \mathbb{R}^{B \times n_l}$, loss averaged over the batch) the same equations read

$$\Delta^{(l)} = (\Delta^{(l+1)} \mathbf{W}^{(l+1)}) \odot \phi'(\mathbf{Z}^{(l)}), \quad \frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(l)}} = \Delta^{(l)\top} \mathbf{H}^{(l-1)}, \quad \frac{\partial \mathcal{L}}{\partial \mathbf{b}^{(l)}} = \Delta^{(l)\top} \mathbf{1},$$

with $\Delta^{(L)} = (\mathbf{P} - \mathbf{Y})/B$. Check the shapes: $\Delta^{(l)\top} \mathbf{H}^{(l-1)}$ is $(n_l, B)(B, n_{l-1}) = (n_l, n_{l-1})$, the shape of $\mathbf{W}^{(l)}$. Shape-matching is the fastest way to rederive these formulas under pressure: there is usually only one way to multiply the available matrices into the required shape.

code/dl/c03_backprop.py

```
1 def mlp_forward_backward(X, y, Ws, bs):
2     """ReLU hidden layers, softmax + cross-entropy output. X: (B, n0), y: (B,) ints
3     ↪ .
4     Ws[l]: (n_{l+1}, n_l) as in nn.Linear. Returns loss and gradients."""
5     B = X.shape[0]
6     hs, zs = [X], []
7     for l, (W, b) in enumerate(zip(Ws, bs)):
8         z = hs[-1] @ W.T + b # (B, n_{l+1})
9         zs.append(z)
10        hs.append(np.maximum(z, 0) if l < len(Ws) - 1 else z)
11    Z = zs[-1] - zs[-1].max(1, keepdims=True)
```

```

11 P = np.exp(Z) / np.exp(Z).sum(1, keepdims=True)
12 loss = -np.mean(np.log(P[np.arange(B), y]))
13 delta = P.copy(); delta[np.arange(B), y] -= 1; delta /= B      # dL/dz_L
14 gWs, gbs = [None] * len(Ws), [None] * len(Ws)
15 for l in reversed(range(len(Ws))):
16     gWs[l] = delta.T @ hs[l]          # (n_{l+1}, n_l)
17     gbs[l] = delta.sum(0)
18     if l > 0:
19         delta = (delta @ Ws[l]) * (zs[l - 1] > 0) # back through W, then ReLU
20 return loss, gWs, gbs

```

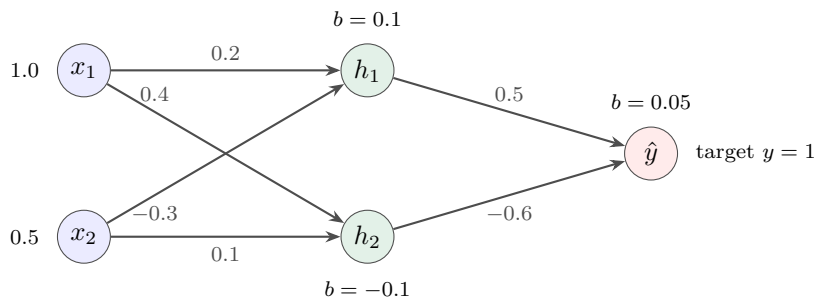
The script runs this on a $5 \rightarrow 8 \rightarrow 6 \rightarrow 3$ network with a batch of 16 and asserts that the loss and all six gradient arrays equal PyTorch's autograd.

layer, forward	local gradient	backward rule
$\mathbf{z} = \mathbf{W}\mathbf{h} + \mathbf{b}$	$\partial \mathbf{z} / \partial \mathbf{h} = \mathbf{W}$	$\bar{\mathbf{h}} = \mathbf{W}^\top \bar{\mathbf{z}}, \bar{\mathbf{W}} = \bar{\mathbf{z}} \mathbf{h}^\top, \bar{\mathbf{b}} = \bar{\mathbf{z}}$
$\mathbf{h} = \phi(\mathbf{z})$ elementwise	$\text{diag } \phi'(\mathbf{z})$	$\bar{\mathbf{z}} = \bar{\mathbf{h}} \odot \phi'(\mathbf{z})$
$\mathbf{h} = \text{ReLU}(\mathbf{z})$	$\mathbb{I}[z > 0]$	$\bar{\mathbf{z}} = \bar{\mathbf{h}} \odot \mathbb{I}[z > 0]$
$s = \sigma(\mathbf{z})$	$s(1 - s)$	$\bar{\mathbf{z}} = \bar{s} s(1 - s)$
$\mathcal{L} = \text{CE}(\text{softmax}(\mathbf{z}), \mathbf{y})$		$\bar{\mathbf{z}} = \mathbf{p} - \mathbf{y}$
$\mathcal{L} = \frac{1}{2} \ \hat{\mathbf{y}} - \mathbf{y}\ ^2$		$\bar{\mathbf{y}} = \hat{\mathbf{y}} - \mathbf{y}$
$\mathbf{C} = \mathbf{A}\mathbf{B}$		$\bar{\mathbf{A}} = \bar{\mathbf{C}}\mathbf{B}^\top, \bar{\mathbf{B}} = \mathbf{A}^\top \bar{\mathbf{C}}$
$u = \max(a, b)$		winner gets $\bar{u}$, loser 0
$u = a + b, u = ab$		route; swap

Backprop is: one forward pass storing $\mathbf{z}^{(l)}$ and $\mathbf{h}^{(l)}$, then for each layer from the top, an outer product for the weight gradient and a transposed matrix product, masked by ϕ' , to pass the error down.

§3.3 A complete forward and backward pass by hand

Prototypical example for this section: A 2-2-1 network with sigmoid hidden units, a sigmoid output and binary cross-entropy, one SGD step with $\eta = 0.5$.



Example 3.3.1 (Forward pass)

$\mathbf{x} = (1, 0.5)$, $\mathbf{W}^{(1)} = \begin{pmatrix} 0.2 & -0.3 \\ 0.4 & 0.1 \end{pmatrix}$, $\mathbf{b}^{(1)} = (0.1, -0.1)$, $\mathbf{w}^{(2)} = (0.5, -0.6)$, $b^{(2)} = 0.05$,

target $y = 1$.

$$\begin{aligned} \mathbf{z}^{(1)} &= \mathbf{W}^{(1)} \mathbf{x} + \mathbf{b}^{(1)} = (0.1500, 0.3500), & \mathbf{h} &= \sigma(\mathbf{z}^{(1)}) = (0.5374, 0.5866), \\ z^{(2)} &= \mathbf{w}^{(2)\top} \mathbf{h} + b^{(2)} = -0.0333, & \hat{y} &= \sigma(z^{(2)}) = 0.4917, \\ \mathcal{L} &= -\ln \hat{y} = 0.7099. \end{aligned}$$

Example 3.3.2 (Backward pass and update)

Output layer: sigmoid + BCE gives $\delta^{(2)} = \hat{y} - y = -0.5083$, so $\partial \mathcal{L} / \partial \mathbf{w}^{(2)} = \delta^{(2)} \mathbf{h} = (-0.2732, -0.2982)$ and $\partial \mathcal{L} / \partial b^{(2)} = -0.5083$.

Hidden layer: $\mathbf{w}^{(2)} \delta^{(2)} = (-0.2542, 0.3050)$, times $\sigma'(\mathbf{z}^{(1)}) = \mathbf{h} \odot (1 - \mathbf{h}) = (0.2486, 0.2425)$ gives $\boldsymbol{\delta}^{(1)} = (-0.0632, 0.0740)$, and

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(1)}} = \boldsymbol{\delta}^{(1)} \mathbf{x}^\top = \begin{pmatrix} -0.0632 & -0.0316 \\ 0.0740 & 0.0370 \end{pmatrix}, \quad \frac{\partial \mathcal{L}}{\partial \mathbf{b}^{(1)}} = (-0.0632, 0.0740).$$

SGD with $\eta = 0.5$: $\mathbf{W}^{(1)} \leftarrow \begin{pmatrix} 0.2316 & -0.2842 \\ 0.3630 & 0.0815 \end{pmatrix}$, $\mathbf{b}^{(1)} \leftarrow (0.1316, -0.1370)$, $\mathbf{w}^{(2)} \leftarrow (0.6366, -0.4509)$, $b^{(2)} \leftarrow 0.3042$. The loss on the same example drops from 0.7099 to 0.5122. Every gradient matches `loss.backward()` in PyTorch.

code/dl/c03_backprop.py

```
1 # forward
2 z1 = W1 @ x0 + b1          # (2,)
3 h1 = sig(z1)               # hidden activations
4 z2 = W2 @ h1 + b2          # scalar logit
5 yhat = sig(z2)
6 L = -(y0 * np.log(yhat) + (1 - y0) * np.log(1 - yhat)) # BCE
7 # backward
8 d2 = yhat - y0             # dL/dz2 (sigmoid + BCE)
9 gW2 = d2 * h1; gb2 = d2
10 d1 = (W2 * d2) * h1 * (1 - h1) # dL/dz1
11 gW1 = np.outer(d1, x0); gb1 = d1
12 # one SGD step
13 W1n, b1n, W2n, b2n = W1 - eta * gW1, b1 - eta * gb1, W2 - eta * gW2, b2 - eta * gb2
```

Two things to notice. The hidden unit with the *negative* outgoing weight gets a positive δ : increasing its activation would lower $\hat{y}$, so its weights are pushed down. And the update to $\mathbf{W}^{(1)}$ is an outer product, rank one: one example moves the first layer only along the direction of its own input $\mathbf{x}$.

Remark 3.3.3 — Trap. With MSE $\frac{1}{2}(\hat{y} - y)^2$ instead of BCE the output error picks up an extra factor: $\delta^{(2)} = (\hat{y} - y) \hat{y}(1 - \hat{y})$. Forgetting which loss the question uses is the most common way to get this OA wrong; the clean $\hat{y} - y$ is specific to sigmoid + BCE (and softmax + CE).

§3.4 Gradient checking

Prototypical example for this section: Compare the analytic $\partial \mathcal{L} / \partial \mathbf{W}^{(1)}$ with $(\mathcal{L}(\theta + h\mathbf{e}_i) - \mathcal{L}(\theta - h\mathbf{e}_i)) / 2h$; a relative error near 10^{-9} means the backward pass is right.

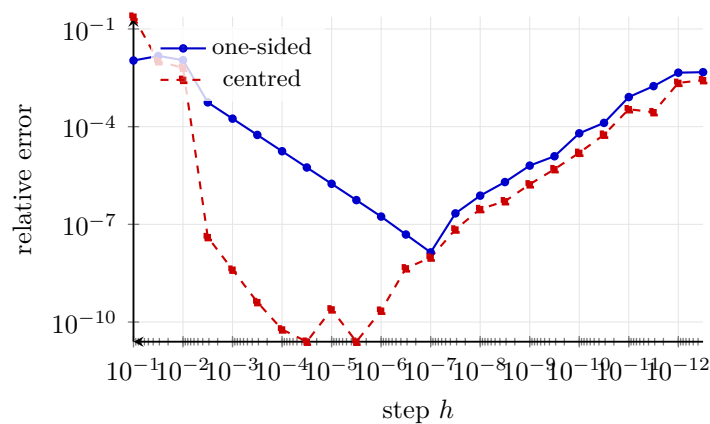
code/dl/c03_backprop.py

```

1 def numerical_grad(f, theta, h=1e-5):
2     g = np.zeros_like(theta)
3     for i in range(theta.size):
4         e = np.zeros_like(theta); e.flat[i] = h
5         g.flat[i] = (f(theta + e) - f(theta - e)) / (2 * h)      # centred
6         ↪ difference
7     return g
8
9 def rel_error(a, b):
10    return np.max(np.abs(a - b) / np.maximum(1e-12, np.abs(a) + np.abs(b)))

```

The centred difference has truncation error $O(h^2)$, the one-sided difference $O(h)$; both suffer round-off error $O(\epsilon_{\text{mach}}/h)$. So the error is U-shaped in h :



In double precision the centred difference is best near $h \approx 3 \times 10^{-6}$ (error 2×10^{-11}), the one-sided near $h \approx 1 \times 10^{-7}$ (error 1×10^{-8}). With $h = 10^{-5}$ the full check on $\mathbf{W}^{(1)}$ gives relative error 4.4×10^{-9} . Plant a bug, dropping the ReLU mask in the backward pass, and the relative error jumps to 1.00.

Remark 3.4.1 — Trap. Gradient checks in `float32` fail spuriously: round-off is then about $10^{-7}/h$, so use `float64` for the check. Also turn off dropout and anything random (the two loss evaluations must see the same mask), and avoid points where ReLUs sit exactly at their kink.

§3.5 Reverse-mode autograd from scratch

Prototypical example for this section: A `Tensor` class that records parents and a backward closure; calling `loss.backward()` visits the graph in reverse topological order.

The whole of an autograd engine is three ideas: each operation returns a new node that remembers its inputs and a function mapping the output's gradient to the inputs' gradients; a topological sort; and accumulation (`+=`) for nodes used more than once. Broadcasting needs one more line: the gradient of a broadcast input is summed over the broadcast dimensions.

code/dl/c03_backprop.py

```

1 class Tensor:
2     def __init__(self, data, parents=(), backward=lambda g: ()):
3         self.data = np.asarray(data, dtype=float)

```

```

4     self.grad = np.zeros_like(self.data)
5     self.parents, self._backward = parents, backward
6
7     @staticmethod
8     def _unbroadcast(g, shape):          # sum gradient over broadcast dims
9         while g.ndim > len(shape):
10             g = g.sum(0)
11         for i, s in enumerate(shape):
12             if s == 1:
13                 g = g.sum(i, keepdims=True)
14         return g
15
16     def __add__(self, o):
17         o = o if isinstance(o, Tensor) else Tensor(o)
18         return Tensor(self.data + o.data, (self, o),
19                       lambda g: (self._unbroadcast(g, self.data.shape),
20                                  self._unbroadcast(g, o.data.shape)))
21
22     def __mul__(self, o):
23         o = o if isinstance(o, Tensor) else Tensor(o)
24         return Tensor(self.data * o.data, (self, o),
25                       lambda g: (self._unbroadcast(g * o.data, self.data.shape),
26                                  self._unbroadcast(g * self.data, o.data.shape)))
27
28     def __matmul__(self, o):
29         return Tensor(self.data @ o.data, (self, o),
30                       lambda g: (g @ o.data.T, self.data.T @ g))
31
32     def relu(self):
33         return Tensor(np.maximum(self.data, 0), (self,), lambda g: (g * (self.data
34 ↪ > 0),))
35
36     def tanh(self):
37         t = np.tanh(self.data)
38         return Tensor(t, (self,), lambda g: (g * (1 - t ** 2),))
39
40     def sum(self):
41         return Tensor(self.data.sum(), (self,), lambda g: (g * np.ones_like(self.
42 ↪ data),))
43
44     def cross_entropy(self, y):          # logits (B, K), integer labels (B,)
45         Z = self.data - self.data.max(1, keepdims=True)
46         P = np.exp(Z) / np.exp(Z).sum(1, keepdims=True)
47         B = len(y)
48         loss = -np.log(P[np.arange(B), y]).mean()
49         def back(g):
50             d = P.copy(); d[np.arange(B), y] -= 1
51             return (g * d / B,)
52         return Tensor(loss, (self,), back)
53
54     def backward(self):
55         order, seen = [], set()
56         def visit(t):                    # topological order by DFS
57             if id(t) not in seen:
58                 seen.add(id(t))
59                 for p in t.parents:
60                     visit(p)
61                 order.append(t)
62         visit(self)
63         self.grad = np.ones_like(self.data)
64         for t in reversed(order):        # outputs before inputs

```

```

63         for p, g in zip(t.parents, t._backward(t.grad)):
64             p.grad = p.grad + g

```

That is 55 non-blank lines. The script builds a ReLU–tanh network with a broadcast bias and a term that uses $\mathbf{h}$ twice ($\mathbf{h} \odot \mathbf{h}$) plus a broadcast scalar, and asserts that the loss and the gradients of all six parameter arrays equal PyTorch’s.

Question 3.5.1. Why does `backward` initialise the output’s gradient to one, and what would `loss.backward()` mean in PyTorch if `loss` were a vector?

§3.6 Vanishing and exploding gradients

Prototypical example for this section: In a 30-layer sigmoid MLP, the gradient reaching the first layer is 5.4×10^{-19} times the one at the last layer.

§3.6.i The product of Jacobians

Unrolling the recursion,

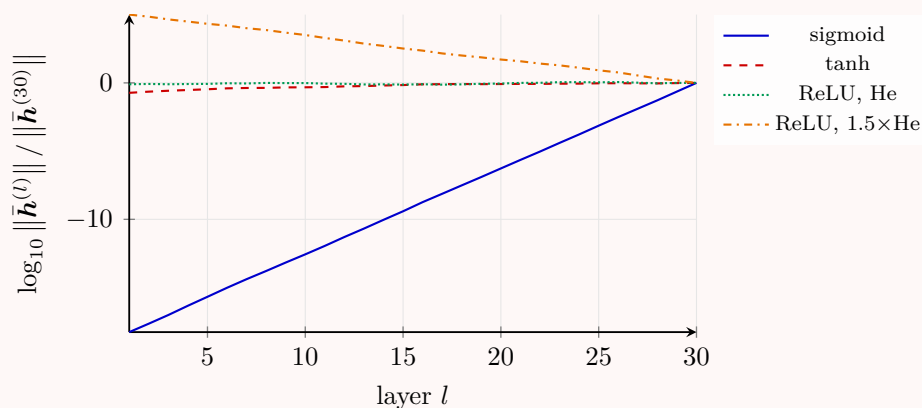
$$\frac{\partial \mathcal{L}}{\partial \mathbf{h}^{(l)}} = \left(\prod_{k=l+1}^L \mathbf{W}^{(k)\top} \text{diag} \phi'(z^{(k)}) \right) \frac{\partial \mathcal{L}}{\partial \mathbf{h}^{(L)}},$$

with the factor for $k = l + 1$ on the left. The norm of a product of $L - l$ random matrices behaves roughly like ρ^{L-l} for some per-layer gain ρ . If $\rho < 1$ the gradient **vanishes** exponentially with depth; if $\rho > 1$ it **explodes**. With sigmoids, $\phi' \leq \frac{1}{4}$ forces ρ well below one unless the weights are large, and large weights saturate the sigmoids, making ϕ' even smaller.

Example 3.6.1 (Gradient norm through depth)

A 30-layer, width-64 network on Gaussian inputs, no biases, gradient of a random linear readout. Ratio of the gradient norm at layer 1 to that at layer 30:

activation	weight init (std)	$\ \bar{\mathbf{h}}^{(1)}\ / \ \bar{\mathbf{h}}^{(30)}\ $
sigmoid	$\sqrt{1/n}$	5.4×10^{-19}
tanh	$\sqrt{1/n}$	1.9×10^{-1}
ReLU	$\sqrt{2/n}$ (He)	7.9×10^{-1}
ReLU	$1.5\sqrt{2/n}$	1.0×10^5



A $1.5\times$ too large initial scale per layer compounds to five orders of magnitude over 30 layers: initialisation is not a detail (chapter 5).

The fixes, each covered later: activations with $\phi' = 1$ on the active side (ReLU family); variance-preserving initialisation (Xavier, He); normalisation layers (BatchNorm, LayerNorm); residual connections, which add an identity term to every Jacobian so the product cannot collapse (ResNet, chapter 6); gating in LSTMs (chapter 7); and gradient clipping for the exploding case (chapter 4).

Remark 3.6.2 — Trap. Vanishing gradients slow down learning in the *early* layers (those closest to the input), not the late ones. The last layers train fine; the network then behaves like a shallow model on top of nearly random features.

§3.6.ii Dying ReLUs

A ReLU unit with $z < 0$ for every input outputs 0 and passes back 0, so its incoming weights never change again: it is **dead**. Two ways to kill units, both simulated in `code/dl/c03_backprop.py`: a large learning-rate step that throws biases far negative, and a bad initialisation. Training a width-200 ReLU layer on a regression task for 300 SGD steps:

learning rate	dead units	final training MSE
0.01	0.0%	0.71
0.1	0.0%	0.02
0.2	68.0%	15.00

Initialising all biases at -3 leaves 99.0% of units dead before training starts. Remedies: a smaller learning rate or warm-up, leaky ReLU/ELU/GELU, He initialisation with zero biases, normalisation before the ReLU.

§3.7 What backprop costs

Prototypical example for this section: A $784 \rightarrow 256 \rightarrow 128 \rightarrow 10$ MLP: 234,752 multiply-adds per example forward, about three times that per training step.

A dense layer with n_{in} inputs and n_{out} outputs costs $n_{\text{in}}n_{\text{out}}$ multiply-accumulates (MACs), i.e. $2n_{\text{in}}n_{\text{out}}$ FLOPs, per example. The backward pass needs two products of the same size, $\bar{\mathbf{h}} = \mathbf{W}^\top \bar{\mathbf{z}}$ and $\bar{\mathbf{W}} = \bar{\mathbf{z}}\mathbf{h}^\top$, so a training step costs about $3\times$ the forward pass: for the MNIST MLP, 469,504 FLOPs forward and about 1,408,512 per example per step. (The first layer's $\bar{\mathbf{h}} = \mathbf{W}^\top \bar{\mathbf{z}}$ can be skipped since nobody needs $\partial\mathcal{L}/\partial\mathbf{x}$.)

Memory is the other bill: every $\mathbf{z}^{(l)}$ or $\mathbf{h}^{(l)}$ needed by a backward rule is kept from the forward pass, so activation memory grows with depth $\times$ batch size $\times$ width. **Gradient checkpointing** stores only some layers' activations and recomputes the rest during the backward pass, trading about one extra forward pass for memory that grows like $\sqrt{L}$ instead of L .

§3.8 A few problems to think about

Problem 3A. For $f = \max(x, y) \cdot z$ at $x = 1, y = 4, z = -2$, give $\partial f / \partial x, \partial f / \partial y, \partial f / \partial z$.

hint · solution



Problem 3B. A linear layer maps a batch $\mathbf{X} \in \mathbb{R}^{32 \times 100}$ to $\mathbf{X}\mathbf{W}^\top + \mathbf{b}$ with $\mathbf{W} \in \mathbb{R}^{50 \times 100}$. What are the shapes of $\partial\mathcal{L}/\partial\mathbf{W}$, $\partial\mathcal{L}/\partial\mathbf{b}$ and $\partial\mathcal{L}/\partial\mathbf{X}$, and which products give them?

[hint](#) · [solution](#)

Problem 3C. Redo [example 3.3.2](#) with the loss $\frac{1}{2}(\hat{y} - y)^2$: give $\delta^{(2)}$ symbolically and say whether the hidden-layer updates get larger or smaller than with BCE.

[hint](#) · [solution](#)

Problem 3D. Show that if all weights of an MLP are initialised to the same constant (and all biases equal), every hidden unit in a layer stays identical throughout training. Do the output units stay identical too?

[hint](#) · [solution](#)

Problem 3E. Explain, with the product-of-Jacobians formula, why a residual block $\mathbf{h}^{(l+1)} = \mathbf{h}^{(l)} + F(\mathbf{h}^{(l)})$ helps gradients reach early layers.

[hint](#) · [solution](#)

Problem 3F. Training loss stays exactly constant from the first step in a ReLU network. Give two causes related to this chapter and a check for each.

[hint](#) · [solution](#)

4 Optimisers and learning-rate schedules

Backpropagation gives the gradient; the optimiser decides what to do with it. The OA version of this chapter is “what is the purpose of Adam?” and one hand-computed update. The interview version is why plain gradient descent zig-zags, what momentum and Adam’s second moment fix, why Adam’s bias correction exists, and why the learning-rate schedule matters as much as the optimiser.

§4.1 Gradient descent and its one big problem

Prototypical example for this section: On $f(x, y) = \frac{1}{2}(x^2 + 10y^2)$ gradient descent must use a step below $2/10$, so progress along x is ten times slower than it could be.

§4.1.i Curvature limits the step

Gradient descent updates $\theta \leftarrow \theta - \eta \nabla \mathcal{L}(\theta)$. Near a minimum the loss is approximately quadratic, $\mathcal{L}(\theta) \approx \mathcal{L}^* + \frac{1}{2}(\theta - \theta^*)^\top \mathbf{H}(\theta - \theta^*)$, and in the eigenbasis of the Hessian $\mathbf{H}$ each coordinate evolves independently: $e_i \leftarrow (1 - \eta \lambda_i) e_i$. Convergence needs $|1 - \eta \lambda_i| < 1$ for all i , i.e. $\eta < 2/\lambda_{\max}$; the slowest direction then shrinks by $1 - \eta \lambda_{\min}$ per step. The best fixed step, $\eta = 2/(\lambda_{\min} + \lambda_{\max})$, gives the contraction factor

$$\frac{\kappa - 1}{\kappa + 1}, \quad \kappa = \frac{\lambda_{\max}}{\lambda_{\min}} \quad (\text{the condition number}).$$

Example 4.1.1 (An ill-conditioned bowl)

For $\mathbf{H} = \text{diag}(1, 10)$: steps above 0.20 diverge (with $\eta = 0.21$, $|y|$ grows to 6.7 in 20 steps from 1). The optimal step $\eta = 0.1818$ contracts the error by 0.8182 per step (the script measures exactly this), so reducing it by 10^6 takes 69 steps. Heavy-ball momentum with its optimal parameters contracts by $(\sqrt{\kappa} - 1)/(\sqrt{\kappa} + 1) = 0.5195$ and needs 22 steps.

The geometric picture: the gradient points across a narrow valley rather than along it, so a step large enough to make progress along the valley overshoots across it. Deep-network losses have Hessians with condition numbers in the thousands or worse, which is why every optimiser in this chapter is, one way or another, a fix for ill-conditioning.

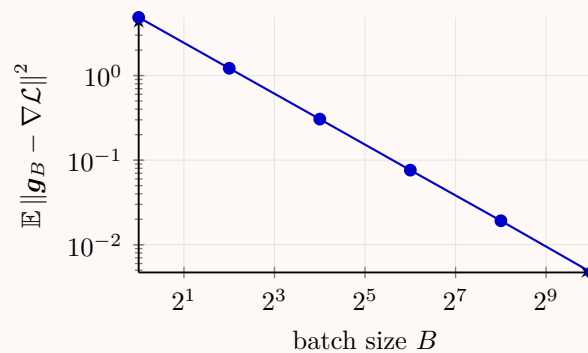
§4.1.ii Stochastic and minibatch gradients

The training loss is an average over N examples. A minibatch gradient over B random examples is an unbiased estimate of the full gradient whose variance falls like $1/B$: averaging B independent terms divides the variance by B .

Example 4.1.2 (Gradient noise against batch size)

For logistic regression on 20,000 synthetic points, the mean squared deviation of a minibatch gradient from the full gradient, as a function of B , has log-log slope

−1.001; going from $B = 1$ to $B = 64$ cuts it by a factor of 63.9.



Vocabulary that OAs test: an **iteration** (step) processes one minibatch; an **epoch** is one pass over the training set. With $N = 50000$ and $B = 128$ an epoch is $\lceil N/B \rceil = 391$ iterations (the last batch is partial unless dropped), so 30 epochs are 11730 steps. “SGD” in deep learning almost always means minibatch SGD.

§4.2 Momentum and Nesterov

Prototypical example for this section: A ball rolling down the valley accumulates speed along the floor while the side-to-side bounces cancel.

Momentum keeps a running sum of past gradients:

$$\mathbf{v} \leftarrow \beta \mathbf{v} + \mathbf{g}, \quad \boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta \mathbf{v} \quad (\text{PyTorch convention}).$$

Components of the gradient that keep their sign add up (on a constant slope $\mathbf{v} \rightarrow \mathbf{g}/(1-\beta)$, so $\beta = 0.9$ is a $10\times$ larger effective step), while components that flip sign every step cancel. That is exactly the valley geometry above. **Nesterov momentum** evaluates the gradient at the look-ahead point $\boldsymbol{\theta} - \eta \beta \mathbf{v}$; in PyTorch’s equivalent form the update uses $\mathbf{g} + \beta \mathbf{v}$ instead of $\mathbf{v}$. It reacts one step earlier when the velocity is about to overshoot.

Example 4.2.1 (Two momentum steps)

$\eta = 0.1$, $\beta = 0.9$, $\theta_0 = 1$, gradients 2 then 1: $v_1 = 2.00$, $\theta_1 = 0.80$; $v_2 = 0.9 \cdot 2 + 1 = 2.80$, $\theta_2 = 0.52$. The second gradient is half the first, yet the second step is larger.

Remark 4.2.2 — Trap. Two conventions exist. Many texts write $\mathbf{v} \leftarrow \beta \mathbf{v} + (1-\beta)\mathbf{g}$ (an exponential moving average); PyTorch’s SGD uses $\mathbf{v} \leftarrow \beta \mathbf{v} + \mathbf{g}$ (its dampening defaults to 0). For the same η the PyTorch step is $1/(1-\beta)$ times larger, so learning rates do not transfer between the two without rescaling.

§4.3 Adaptive methods: AdaGrad, RMSProp, Adam

Prototypical example for this section: Adam’s first step moves every parameter by almost exactly η , whether its gradient is 0.5 or -0.02 .

§4.3.i Per-parameter step sizes

Momentum still uses one step size for all parameters. Adaptive methods divide each coordinate by a running estimate of its gradient scale, which approximately rescales the valley into a round bowl when the Hessian is close to diagonal:

- **AdaGrad**: $G \leftarrow G + \mathbf{g}^2$, $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta \mathbf{g} / (\sqrt{G} + \varepsilon)$. Rare features (sparse gradients) get large steps; but G only grows, so the step size decays to zero, too early for deep networks.
- **RMSProp**: replace the sum by an exponential moving average, $\mathbf{v} \leftarrow \alpha \mathbf{v} + (1 - \alpha) \mathbf{g}^2$, so the scale tracks recent gradients and the step does not die.
- **Adam** [KiBa15] = momentum on $\mathbf{g}$ plus RMSProp on $\mathbf{g}^2$, with bias correction:

Algorithm: Adam, step t (defaults $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\varepsilon = 10^{-8}$)

$$\begin{aligned} \mathbf{m}_t &= \beta_1 \mathbf{m}_{t-1} + (1 - \beta_1) \mathbf{g}_t, & \mathbf{v}_t &= \beta_2 \mathbf{v}_{t-1} + (1 - \beta_2) \mathbf{g}_t^2, \\ \hat{\mathbf{m}}_t &= \mathbf{m}_t / (1 - \beta_1^t), & \hat{\mathbf{v}}_t &= \mathbf{v}_t / (1 - \beta_2^t), \\ \boldsymbol{\theta}_t &= \boldsymbol{\theta}_{t-1} - \eta \hat{\mathbf{m}}_t / (\sqrt{\hat{\mathbf{v}}_t} + \varepsilon). \end{aligned}$$

§4.3.ii Why bias correction

With $\mathbf{m}_0 = \mathbf{0}$, $\mathbf{m}_t = (1 - \beta_1) \sum_{s=1}^t \beta_1^{t-s} \mathbf{g}_s$. If the gradients have constant mean $\mathbf{g}$, then $\mathbb{E} \mathbf{m}_t = (1 - \beta_1^t) \mathbf{g}$: the average is biased toward the zero initialisation by exactly the factor $1 - \beta_1^t$, which dividing removes. The bias matters most for $\mathbf{v}$, because $1 - \beta_2^t$ stays small for hundreds of steps ($1 - 0.999^t \approx t/1000$). Without correction the first step would be $\eta \cdot 0.1 \mathbf{g} / \sqrt{0.001 \mathbf{g}^2} = \eta \cdot 0.1 / \sqrt{0.001} \approx 3.16 \eta$; for $\eta = 10^{-3}$ that is 3.16×10^{-3} , a too-large first step taken at the moment the estimates are least reliable.

Example 4.3.1 (Two Adam steps by hand)

Two parameters at $\boldsymbol{\theta}_0 = (1, 1)$, $\eta = 10^{-3}$, default betas, gradients $\mathbf{g}_1 = (0.5, -0.02)$ then $\mathbf{g}_2 = (0.3, 0.04)$.

Step 1. $\mathbf{m}_1 = 0.1 \mathbf{g}_1 = (0.0500, -0.0020)$, $\mathbf{v}_1 = 0.001 \mathbf{g}_1^2 = (2.50 \times 10^{-4}, 4.00 \times 10^{-7})$. Bias correction divides by 0.1 and 0.001, giving $\hat{\mathbf{m}}_1 = \mathbf{g}_1$, $\hat{\mathbf{v}}_1 = \mathbf{g}_1^2$, so the step is $\eta \mathbf{g}_1 / |\mathbf{g}_1|$: both parameters move by $(1.000 \times 10^{-3}, -1.000 \times 10^{-3})$, one down and one up. The 25-fold difference in gradient size has disappeared.

Step 2. $\mathbf{m}_2 = 0.9 \mathbf{m}_1 + 0.1 \mathbf{g}_2 = (0.0750, 0.0022)$, $\mathbf{v}_2 = 0.999 \mathbf{v}_1 + 0.001 \mathbf{g}_2^2 = (3.398 \times 10^{-4}, 2.000 \times 10^{-6})$. Corrections $1 - 0.9^2 = 0.19$ and $1 - 0.999^2 = 0.001999$ give $\hat{\mathbf{m}}_2 = (0.3947, 0.0116)$ and $\hat{\mathbf{v}}_2 = (1.700 \times 10^{-1}, 1.000 \times 10^{-3})$; the parameters decrease by $(9.575 \times 10^{-4}, 3.661 \times 10^{-4})$, ending at $\boldsymbol{\theta}_2 = (0.998043, 1.000634)$, exactly as `torch.optim.Adam`. The second parameter's gradient changed sign, so its averaged momentum is small relative to its RMS and it moves only about a third of η .

code/dl/c04_optim.py

```
1 class SGD:
2     def __init__(self, lr, momentum=0.0, nesterov=False, weight_decay=0.0):
3         self.lr, self.mu, self.nesterov, self.wd, self.buf = lr, momentum, nesterov
```

```

    ↪ , weight_decay, None
4  def step(self, p, g):
5      g = g + self.wd * p                                # L2 penalty folded into g
6      if self.mu:
7          self.buf = g if self.buf is None else self.mu * self.buf + g
8          g = g + self.mu * self.buf if self.nesterov else self.buf
9      return p - self.lr * g
10
11 class AdaGrad:
12     def __init__(self, lr, eps=1e-10):
13         self.lr, self.eps, self.G = lr, eps, 0.0
14     def step(self, p, g):
15         self.G = self.G + g * g                        # sum of all squared gradients
16         return p - self.lr * g / (np.sqrt(self.G) + self.eps)
17
18 class RMSProp:
19     def __init__(self, lr, alpha=0.99, eps=1e-8):
20         self.lr, self.a, self.eps, self.v = lr, alpha, eps, 0.0
21     def step(self, p, g):
22         self.v = self.a * self.v + (1 - self.a) * g * g  # EMA of squared
    ↪ gradients
23         return p - self.lr * g / (np.sqrt(self.v) + self.eps)
24
25 class Adam:
26     def __init__(self, lr, betas=(0.9, 0.999), eps=1e-8, weight_decay=0.0,
    ↪ decoupled=False):
27         self.lr, (self.b1, self.b2), self.eps = lr, betas, eps
28         self.wd, self.decoupled, self.m, self.v, self.t = weight_decay, decoupled,
    ↪ 0.0, 0.0, 0
29     def step(self, p, g):
30         self.t += 1
31         if self.decoupled:                                # AdamW: decay the weights
    ↪ directly
32             p = p * (1 - self.lr * self.wd)
33         else:                                              # Adam + L2: decay enters the
    ↪ gradient
34             g = g + self.wd * p
35             self.m = self.b1 * self.m + (1 - self.b1) * g
36             self.v = self.b2 * self.v + (1 - self.b2) * g * g
37             m_hat = self.m / (1 - self.b1 ** self.t)      # bias correction
38             v_hat = self.v / (1 - self.b2 ** self.t)
39             return p - self.lr * m_hat / (np.sqrt(v_hat) + self.eps)

```

All nine variants above (SGD, momentum, Nesterov, SGD with weight decay, AdaGrad, RMSProp, Adam, Adam with L2, AdamW) are run for 25 steps on a non-quadratic test function and their trajectories asserted equal to `torch.optim`'s.

Remark 4.3.2 — Trap. “What is the purpose of Adam?” is not “to add momentum” and not “to reduce overfitting”. Adam gives each parameter its own effective learning rate, scaled by the inverse RMS of its recent gradients, and combines that with momentum; the practical effect is fast, robust progress with little learning-rate tuning. It does not regularise.

Remark 4.3.3 — Trap. “Adam always beats SGD.” On many image-classification benchmarks well-tuned SGD with momentum reaches equal or better test accuracy; Adam’s coordinate-wise rescaling can settle in different (often sharper) solutions,

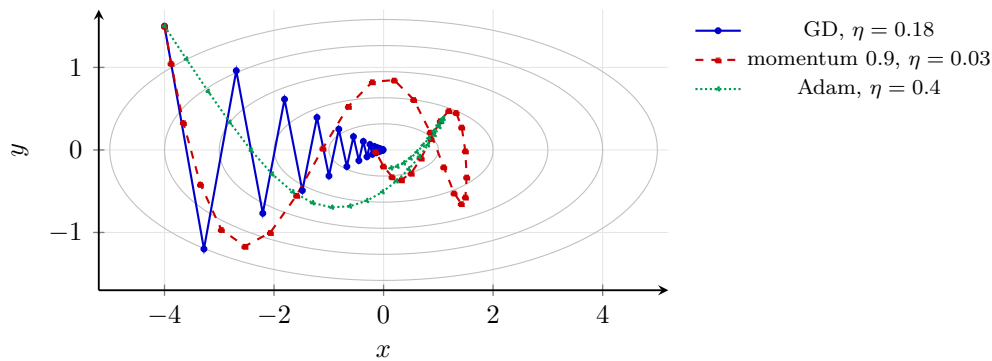
and with L2 regularisation it under-decays parameters with large gradient history. Adam and especially AdamW dominate for Transformers. The honest interview answer is “it depends on the architecture, and the gap is usually closed by tuning the schedule and weight decay”.

§4.3.iii AdamW: weight decay done properly

With plain SGD, adding $\frac{\lambda}{2} \|\theta\|^2$ to the loss and shrinking the weights by $(1 - \eta\lambda)$ each step are the same thing. With Adam they are not: the L2 gradient $\lambda\theta$ is divided by $\sqrt{\hat{v}}$ like everything else, so parameters with large gradients are barely decayed. **AdamW** [LoHu19] decouples the decay, $\theta \leftarrow \theta(1 - \eta\lambda)$, outside the adaptive step (the **decoupled** branch in the code). Chapter 5 measures the difference.

§4.4 Paths and curves

Prototypical example for this section: Thirty steps of GD, momentum and Adam from $(-4, 1.5)$ on the bowl of the first section.

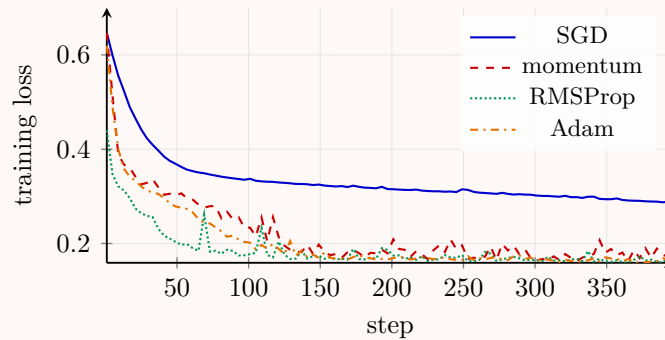


GD with a step close to the stability limit zig-zags across the narrow direction; momentum overshoots along the valley and spirals in; Adam moves at roughly constant speed η per coordinate and, with a fixed learning rate, keeps hovering around the minimum (final loss 0.2504 against 0.0001 for GD). That last behaviour is the argument for decaying the learning rate.

Example 4.4.1 (Training curves on a real network)

A 2-64-64-2 ReLU network on a noisy two-moons dataset, minibatches of 64, the same initialisation for every optimiser; full-data training loss after 100 and 400 steps:

optimiser	step 100	step 400
SGD, $\eta = 0.05$	0.337	0.286
SGD + momentum 0.9	0.223	0.180
RMSPProp, $\eta = 0.003$	0.178	0.161
Adam, $\eta = 0.003$	0.201	0.159



One run on one problem proves nothing general; the point is the shape (momentum and the adaptive methods leave plain SGD behind early) and how to set up a fair comparison: same initialisation, same batches, each optimiser at a sensible learning rate.

§4.5 Learning-rate schedules

Prototypical example for this section: Cosine decay from 0.1 over 100 epochs: 0.0854 at epoch 25, 0.0500 at epoch 50, 0.0146 at epoch 75.

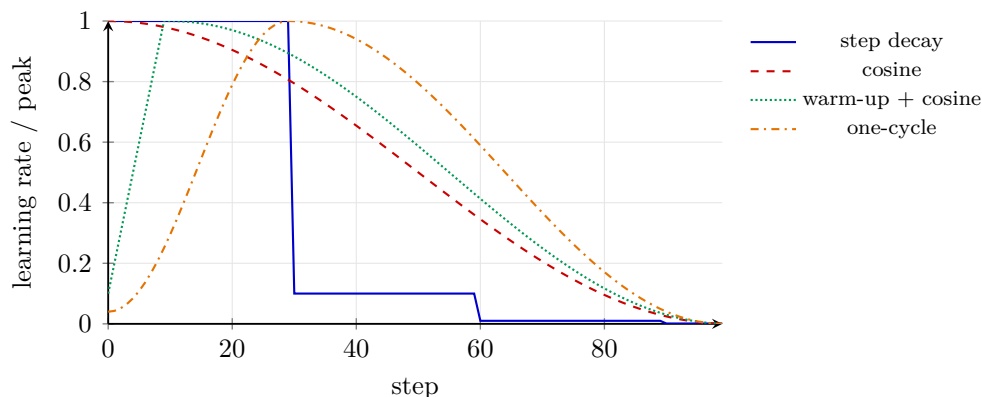
A large learning rate early makes fast progress and adds useful noise; a small one late lets the iterate settle into a minimum (recall Adam hovering above). Schedules interpolate:

code/dl/c04_optim.py

```

1 def step_decay(t, lr0, step=30, gamma=0.1):
2     return lr0 * gamma ** (t // step)
3
4 def cosine(t, lr0, T, lr_min=0.0):
5     return lr_min + 0.5 * (lr0 - lr_min) * (1 + math.cos(math.pi * t / T))
6
7 def warmup_cosine(t, lr0, W, T):
8     if t < W:
9         return lr0 * (t + 1) / W # linear warm-up
10    return cosine(t - W, lr0, T - W)
11
12 def one_cycle(t, lr_max, T, pct=0.3, div=25.0, final_div=1e4):
13     up = pct * T - 1
14     lo, end = lr_max / div, lr_max / div / final_div
15     cos_anneal = lambda a, b, frac: b + (a - b) / 2 * (1 + math.cos(math.pi * frac)
16     ⇐⇒ )
17     if t <= up:
18         return cos_anneal(lo, lr_max, t / up)
19     return cos_anneal(lr_max, end, (t - up) / (T - 1 - up))

```



The script asserts that step decay, cosine and one-cycle reproduce the schedulers `StepLR`, `CosineAnnealingLR` and `OneCycleLR` of PyTorch exactly. Step decay with $\eta_0 = 0.1$, $\gamma = 0.1$ every 30 epochs gives 0.0010 at epoch 65.

Warm-up ramps the learning rate up linearly over the first few hundred or thousand steps. Early on, the weights are random, activations and gradients can be large and Adam’s second-moment estimate rests on a handful of samples, so a full-size step is risky; Transformers in particular often diverge without warm-up. **One-cycle** goes up then down in a single cycle (with momentum moving the opposite way) and is popular for short training budgets.

Remark 4.5.1 — Trap. Schedulers in PyTorch are stepped explicitly. Calling `scheduler.step()` once per batch for a schedule defined in epochs (or the reverse) silently compresses or stretches it by a factor of the number of batches per epoch.

§4.6 Batch size, clipping and the loss landscape

Prototypical example for this section: Doubling the batch halves the gradient variance, so the usual heuristic doubles the learning rate with it.

§4.6.i Batch size

Because the gradient noise variance scales like $1/B$, a larger batch gives a cleaner gradient and allows a proportionally larger step: the **linear scaling heuristic** ($\eta \propto B$, used with warm-up) works up to a problem-dependent critical batch size, beyond which larger batches stop reducing the number of steps needed. Small batches inject more noise, which is widely reported to help generalisation; the explanations (noise steering toward flatter minima) are plausible but not settled, so state them as such. Practical constraints usually decide B : memory, and hardware throughput.

§4.6.ii Gradient clipping

Clipping by norm rescales the whole gradient when its norm exceeds a threshold c : $\mathbf{g} \leftarrow \mathbf{g} \cdot \min(1, c/\|\mathbf{g}\|)$. For $\mathbf{g} = (3, 4)$ and $c = 1$ it gives $(0.6, 0.8)$: same direction, length 1. **Clipping by value** clips each coordinate to $[-c, c]$, giving $(1, 1)$, which changes the direction. Both match PyTorch’s `clip_grad_norm_` and `clip_grad_value_`. Clipping by norm is the standard fix for exploding gradients in RNNs and is routine in Transformer training; it does nothing for vanishing gradients.

§4.6.iii Loss landscapes

In high dimension most critical points of a neural-network loss are **saddle points**: the Hessian has both positive and negative eigenvalues, and for a random critical point it is unlikely that all of thousands of eigenvalues are positive. Gradient descent slows down near saddles (the gradient is small) but stochastic noise and momentum push it off them. Poor local minima are less of a practical problem in large networks than the textbook non-convexity warning suggests; plateaus, ill-conditioning and bad initialisation are the usual culprits when training stalls.

Start with AdamW (Transformers) or SGD with momentum 0.9 (CNNs), a warm-up plus cosine schedule, gradient clipping at norm 1; then tune the peak learning rate first, because it matters more than anything else on this page.

§4.7 A few problems to think about

Problem 4A. A training set has 60,000 examples and the batch size is 256 (last partial batch kept). How many optimiser steps are taken in 10 epochs?

[hint · solution](#)



Problem 4B. Compute Adam's first step for a parameter with gradient $g_1 = -4$, $\eta = 0.01$, default betas, ε negligible. What if the gradient were -0.004 ?

[hint · solution](#)



Problem 4C. Show that for gradients with constant mean $\mathbf{g}$, $\mathbb{E}\mathbf{v}_t = (1 - \beta_2^t)\mathbb{E}\mathbf{g}^2$ for Adam's second moment with $\mathbf{v}_0 = \mathbf{0}$, and compute the correction factor after 10 steps with $\beta_2 = 0.999$.

[hint · solution](#)



Problem 4D. For $f(x) = \frac{\lambda}{2}x^2$ and gradient descent with step η , find all η for which the iterates converge, and the η that converges in one step. What happens at $\eta = 2/\lambda$?

[hint · solution](#)



Problem 4E. A gradient $(6, -8, 0)$ is clipped (i) by norm with threshold 5, (ii) by value with threshold 5. Give both results.

[hint · solution](#)



Problem 4F. With PyTorch-style momentum $\mathbf{v} \leftarrow \beta\mathbf{v} + \mathbf{g}$ and a constant gradient $\mathbf{g}$, find the limiting step and the number of steps until the step reaches 90% of it for $\beta = 0.9$ and $\beta = 0.99$.

[hint · solution](#)



5 Initialisation, normalisation and regularisation

Three families of tricks keep deep networks trainable and keep them from memorising: the scale of the initial weights, layers that renormalise activations, and penalties or noise that restrict what the network can fit. The OA questions here are short but have popular wrong answers (“BatchNorm behaves the same at inference”, “dropout increases capacity”, “what does λ do in $E + \lambda \sum w^2$ ”), and the interview questions are derivations: the Xavier and He variances and the BatchNorm backward pass.

§5.1 Initialisation

Prototypical example for this section: A 50-layer ReLU network: with weight standard deviation $\sqrt{1/n}$ the activations shrink to 3.6×10^{-8} of their input scale; with $\sqrt{2/n}$ they stay at 1.196.

§5.1.i Why not zeros, why not tiny

All-equal weights make every unit in a layer compute the same function and receive the same gradient forever (Problem 3D): **symmetry** must be broken by random initialisation. But the scale of the randomness matters. Each layer multiplies the activation scale by some gain; over L layers the gains multiply, and anything other than a gain of about one either vanishes or explodes, in the forward pass and (**chapter 3**) in the backward pass.

§5.1.ii Deriving Xavier and He

Take $z_j = \sum_{i=1}^n W_{ji} h_i$ with independent zero-mean weights of variance σ_w^2 , independent of the inputs. Then

$$\text{Var}(z_j) = \sum_{i=1}^n \text{Var}(W_{ji} h_i) = n \sigma_w^2 \mathbb{E}[h_i^2].$$

- For tanh near the origin $\tanh z \approx z$, so $\mathbb{E}h^2 \approx \text{Var}(z_{\text{prev}})$ and the variance is preserved when $n\sigma_w^2 = 1$. The backward pass multiplies by $\mathbf{W}^\top$, whose sum runs over n_{out} terms, and preserves gradient variance when $n_{\text{out}}\sigma_w^2 = 1$. **Xavier** (Glorot) initialisation [**GlBe10**] compromises:

$$\sigma_w^2 = \frac{2}{n_{\text{in}} + n_{\text{out}}}, \quad \text{or } W \sim U\left[-\sqrt{\frac{6}{n_{\text{in}} + n_{\text{out}}}}, \sqrt{\frac{6}{n_{\text{in}} + n_{\text{out}}}}\right]$$

(a uniform on $[-a, a]$ has variance $a^2/3$).

- For ReLU, if z is symmetric about zero then $\mathbb{E}[\text{ReLU}(z)^2] = \frac{1}{2} \text{Var}(z)$ (half the mass is zeroed; the script checks this by simulation). To keep $\text{Var } z$ constant we need $n\sigma_w^2 \cdot \frac{1}{2} = 1$: **He** (Kaiming) initialisation [**He+15**],

$$\sigma_w^2 = \frac{2}{n_{\text{in}}}.$$

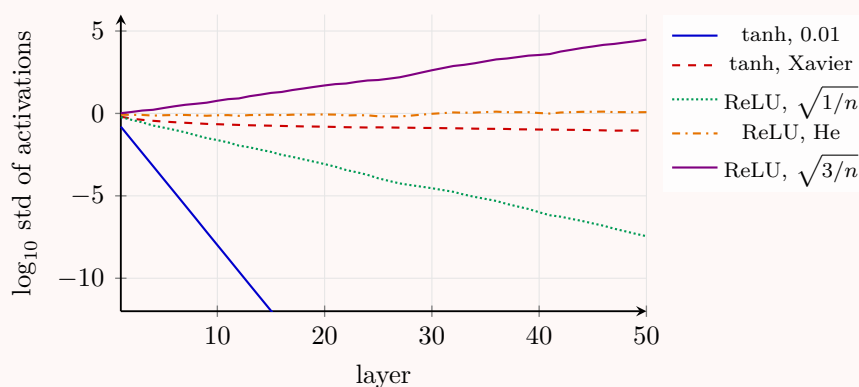
Example 5.1.1 (Numbers for a $256 \rightarrow 128$ layer)

Xavier uniform bound $\sqrt{6/384} = 0.1250$, Xavier normal std 0.0722, He normal std $\sqrt{2/256} = 0.0884$, He uniform bound 0.1531. Each is checked against `torch.nn.init`. PyTorch's *default* for `nn.Linear` is uniform with bound $1/\sqrt{n_{\text{in}}} = 0.0625$ (std 0.0361), which is smaller than He: fine for shallow nets, a reason to initialise deep plain ReLU nets explicitly.

Example 5.1.2 (Activation scale through 50 layers)

Width 256, Gaussian inputs, no biases, no normalisation; standard deviation of the activations at layer 50:

activation	weight std	std at layer 50
tanh	0.01	≈ 0 (already 1.0×10^{-8} at layer 10)
tanh	$\sqrt{1/n}$ (Xavier)	0.089
ReLU	$\sqrt{1/n}$	3.6×10^{-8}
ReLU	$\sqrt{2/n}$ (He)	1.196
ReLU	$\sqrt{3/n}$	3.0×10^4



The $\sqrt{1/n}$ ReLU curve loses a factor $1/\sqrt{2}$ per layer, exactly as the derivation predicts; $\sqrt{3/n}$ gains $\sqrt{3/2}$ per layer. Xavier-tanh decays slowly because tanh is contractive away from zero.

Remark 5.1.3 — Trap. “Use Xavier for ReLU networks.” Xavier assumes a roughly linear activation; ReLU discards half the signal, so it needs the factor 2 of He. The reverse mistake (He for tanh) saturates the units. Biases are usually initialised to zero; that is fine, because the random weights already break symmetry.

§5.2 Batch normalisation

Prototypical example for this section: One feature takes values 1, 2, 3, 6 in a batch: BatchNorm maps them to -1.0690 , -0.5345 , 0.0000 , 1.6036 during training, but at inference the same input 3 need not map to 0.

§5.2.i Forward pass

For a minibatch $\{x_1, \dots, x_B\}$ of one feature (one channel, for convolutions pooled over the spatial positions too), BatchNorm [IoSz15] computes

$$\mu_B = \frac{1}{B} \sum_i x_i, \quad \sigma_B^2 = \frac{1}{B} \sum_i (x_i - \mu_B)^2, \quad \hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \varepsilon}}, \quad y_i = \gamma \hat{x}_i + \beta,$$

with learnable scale γ and shift β per feature. Without γ, β every layer would be forced to zero mean and unit variance; with them the network can undo the normalisation if that is better, so BN does not reduce what the layer can represent.

Example 5.2.1 (BatchNorm by hand)

Batch of four, two features: $(1, 10), (2, 20), (3, 30), (6, 60)$, $\gamma = (1, 2)$, $\beta = (0, 1)$. Feature 1: $\mu = 3.00$, $\sigma^2 = 3.500$ (divide by B), $\sqrt{\sigma^2 + \varepsilon} = 1.8708$, outputs $(-1.0690, -0.5345, 0.0000, 1.6036)$. Feature 2 is feature 1 times ten, so $\hat{x}$ is identical and $y = 2\hat{x} + 1 = (-1.1381, -0.0690, 1.0000, 4.2071)$: BN is invariant to the scale of its input. Both match `nn.BatchNorm1d` in training mode.

§5.2.ii Train versus inference

At inference there may be a single example, and predictions must not depend on what else is in the batch. So during training BN keeps **running estimates** $\mu_{\text{run}} \leftarrow (1 - m)\mu_{\text{run}} + m\mu_B$ and likewise for the variance, and in evaluation mode it uses them instead of batch statistics. In PyTorch $m = 0.1$ (the argument is confusingly called **momentum**), the running mean starts at 0, the running variance at 1, and the running variance is updated with the *unbiased* batch variance (divide by $B - 1$).

Example 5.2.2 (Running statistics after one step)

After the batch above, feature 1 has $\mu_{\text{run}} = 0.9 \cdot 0 + 0.1 \cdot 3 = 0.30$ and $\sigma_{\text{run}}^2 = 0.9 \cdot 1 + 0.1 \cdot 4.667 = 1.3667$ (unbiased variance 4.667, not 3.500). In `eval()` mode the input 3, which was normalised to 0 inside the batch, now maps to $(3 - 0.30)/\sqrt{1.3667} = 2.3096$. The script asserts all three numbers against PyTorch.

Remark 5.2.3 — Trap. “BatchNorm behaves the same at training and inference time.” It does not: training uses the current batch’s statistics (and updates the running averages), inference uses the frozen running averages. Forgetting `model.eval()` before evaluation therefore changes predictions (and keeps modifying the running statistics); it is the most common BN bug. In training mode a batch of one cannot be normalised at all: PyTorch raises a `ValueError` for `BatchNorm1d` on a single example.

§5.2.iii Backward pass

BN’s backward pass is a favourite derivation because μ_B and σ_B depend on every x_i , so each output depends on the whole batch.

Proposition 5.2.4 (BatchNorm gradient)

With $\hat{x}_i$ as above and upstream gradients $g_i = \partial\mathcal{L}/\partial y_i$,

$$\frac{\partial\mathcal{L}}{\partial\gamma} = \sum_i g_i \hat{x}_i, \quad \frac{\partial\mathcal{L}}{\partial\beta} = \sum_i g_i, \quad \frac{\partial\mathcal{L}}{\partial x_i} = \frac{\gamma}{B\sqrt{\sigma_B^2 + \varepsilon}} \left(Bg_i - \sum_j g_j - \hat{x}_i \sum_j g_j \hat{x}_j \right).$$

Proof. Write $s = \sqrt{\sigma_B^2 + \varepsilon}$ and $\bar{g}_i = \gamma g_i = \partial\mathcal{L}/\partial\hat{x}_i$. Since $\hat{x}_i = (x_i - \mu)/s$, $\partial\hat{x}_j/\partial x_i = \frac{1}{s}(\delta_{ij} - \frac{1}{B}) - \frac{x_j - \mu}{s^2} \frac{\partial s}{\partial x_i}$, and $\partial s/\partial x_i = \frac{1}{2s} \cdot \frac{2}{B}(x_i - \mu) = \hat{x}_i/B$ (the term from $\partial\mu/\partial x_i$ sums to zero). Hence $\partial\hat{x}_j/\partial x_i = \frac{1}{s}(\delta_{ij} - \frac{1}{B} - \frac{\hat{x}_j \hat{x}_i}{B})$ and $\frac{\partial\mathcal{L}}{\partial x_i} = \sum_j \bar{g}_j \frac{\partial\hat{x}_j}{\partial x_i} = \frac{1}{Bs} (B\bar{g}_i - \sum_j \bar{g}_j - \hat{x}_i \sum_j \bar{g}_j \hat{x}_j)$. $\square$

code/dl/c05_init_norm.py

```

1 def bn_forward(x, gamma, beta, eps=1e-5):          # x: (B, C), training mode
2     mu = x.mean(0)
3     var = x.var(0)                                # biased (divide by B)
4     xhat = (x - mu) / np.sqrt(var + eps)
5     return gamma * xhat + beta, (xhat, var, eps)
6
7 def bn_backward(dy, gamma, cache):
8     xhat, var, eps = cache
9     B = dy.shape[0]
10    dgamma = (dy * xhat).sum(0)
11    dbeta = dy.sum(0)
12    dxhat = dy * gamma
13    dx = (B * dxhat - dxhat.sum(0) - xhat * (dxhat * xhat).sum(0)) / (B * np.sqrt(
        ↪ var + eps))
14    return dx, dgamma, dbeta

```

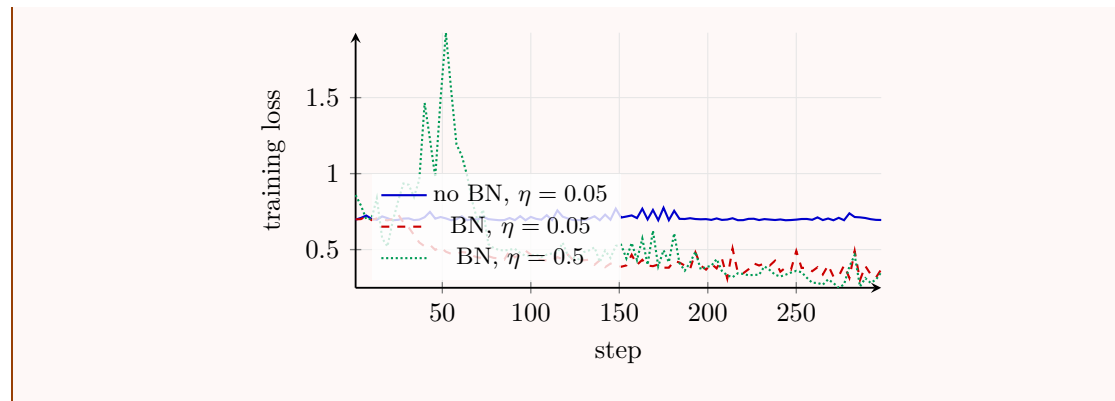
The gradient with respect to x has zero sum and is orthogonal to $\hat{x}$: BN’s output cannot change by shifting or scaling its input, so neither direction can carry gradient. The script asserts $\partial\mathcal{L}/\partial x$, $\partial\mathcal{L}/\partial\gamma$ and $\partial\mathcal{L}/\partial\beta$ against autograd.

§5.2.iv Why BatchNorm helps

The original motivation was “reducing internal covariate shift” (layers chasing a moving input distribution). Later analysis suggests the main benefit is a smoother, better-conditioned loss landscape that tolerates much larger learning rates; the honest interview answer mentions both and says the second has more empirical support. Side effects: BN makes the layer invariant to the scale of its incoming weights, and the batch noise acts as a mild regulariser.

Example 5.2.5 (A 10-layer sigmoid MLP with and without BN)

Same data, initialisation recipe and SGD with momentum; full-data loss after 300 steps: 0.695 without BN at $\eta = 0.05$ (stuck near $\ln 2$), 0.322 with BN at $\eta = 0.05$, 0.375 with BN at a ten times larger $\eta = 0.5$.

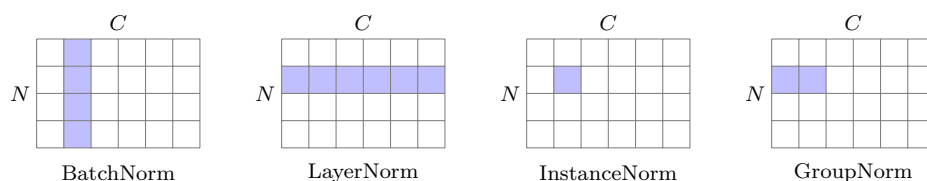


Question 5.2.6. A BatchNorm layer follows a linear layer with a bias. Why is the bias redundant?

§5.3 LayerNorm, GroupNorm, RMSNorm

Prototypical example for this section: LayerNorm normalises one example's features, so the vector $(1, 2, 3, 6)$ becomes $(-1.0690, -0.5345, 0.0000, 1.6036)$ whatever else is in the batch.

The normalisation layers differ only in which axes the mean and variance are taken over. For a convolutional activation of shape (N, C, H, W) :



Each cell stands for all $H \times W$ positions of one channel of one example; the shaded cells are normalised together.

- **BatchNorm**: per channel, over (N, H, W) . Depends on the batch; needs running statistics; weak with small batches.
- **LayerNorm** [BaKiHi16]: per example, over all features (C, H, W) (in a Transformer: over the d_{model} features of each token). No batch dependence, identical at training and inference; the standard in Transformers and RNNs.
- **InstanceNorm**: per example and channel, over (H, W) ; style transfer.
- **GroupNorm**: per example, over groups of channels; $G = 1$ is LayerNorm and $G = C$ is InstanceNorm (both identities asserted in the script). Used for detection and segmentation where batches are tiny.
- **RMSNorm**: LayerNorm without centring and without β , $y = \gamma x / \sqrt{\frac{1}{d} \sum x_i^2 + \epsilon}$; cheaper, and used in many recent LLMs. For $(1, 2, 3, 6)$ the RMS is 3.5355 and the output is $(0.2828, 0.5657, 0.8485, 1.6971)$.

code/dl/c05_init_norm.py

```

1 def layer_norm(x, gamma, beta, eps=1e-5):          # normalise over the last axis
2     mu = x.mean(-1, keepdims=True); var = x.var(-1, keepdims=True)
3     return gamma * (x - mu) / np.sqrt(var + eps) + beta
4
5 def rms_norm(x, gamma, eps=1e-6):                  # no centring, no beta
6     return gamma * x / np.sqrt((x ** 2).mean(-1, keepdims=True) + eps)
7
8 def group_norm(x, G, gamma, beta, eps=1e-5):       # x: (B, C, H, W)
9     B, C, H, W = x.shape
10    xg = x.reshape(B, G, C // G, H, W)
11    mu = xg.mean((2, 3, 4), keepdims=True); var = xg.var((2, 3, 4), keepdims=True)
12    xg = (xg - mu) / np.sqrt(var + eps)
13    return xg.reshape(B, C, H, W) * gamma[None, :, None, None] + beta[None, :, None
    ↪ , None]

```

Parameter count: each of these has $2C$ (or $2d$) learnable parameters, γ and β (RMSNorm: d). BatchNorm2d(64) has 128 learnable parameters plus 128 non-learnable buffers (running mean and variance); the buffers are in the `state_dict` but not in `parameters()`.

Remark 5.3.1 — Trap. BN versus LN in one line: BN normalises each feature across the batch, LN normalises each example across its features. Hence LN works with batch size 1 and variable-length sequences, and behaves identically in training and inference.

§5.4 Dropout

Prototypical example for this section: With $p = 0.5$ and mask $(1, 0, 1, 0)$, activations $(2, 4, 6, 8)$ become $(4, 0, 12, 0)$ during training and stay $(2, 4, 6, 8)$ at test time.

Dropout [Sr+14] zeroes each unit independently with probability p during training. **Inverted dropout**, used by every framework, divides the survivors by $1 - p$ so that the expected activation is unchanged and inference needs no rescaling:

$$\mathbb{E}\left[\frac{m_i h_i}{1 - p}\right] = h_i, \quad m_i \sim \text{Bernoulli}(1 - p).$$

code/dl/c05_regularise.py

```

1 def dropout(h, p, train, rng):
2     if not train or p == 0:
3         return h                                # inference: identity
4     mask = (rng.random(h.shape) >= p)           # keep with probability 1-p
5     return h * mask / (1 - p)                   # inverted scaling

```

The script checks the mean over 200,000 draws, the identity at inference, and that `nn.Dropout(0.5)` scales kept entries by exactly 2. The variance is not preserved: for $h = 2$, $p = \frac{1}{2}$ it is $h^2 p / (1 - p) = 4.00$, which is why dropout placed just before BatchNorm causes a train/test mismatch in the statistics.

Why it regularises: no unit can rely on a specific other unit being present, which discourages co-adapted feature detectors; and training samples one of 2^n thinned sub-networks per step, all sharing weights, so the test-time network approximates the average of an exponential ensemble.

Remark 5.4.1 — Trap. “Dropout is applied at test time too” and “at test time multiply the weights by p ”. With inverted dropout nothing happens at test time; the original (non-inverted) formulation scaled activations by the keep probability $1 - p$ at test time instead. Keeping dropout on at test time on purpose (**MC dropout**) is a different technique, used to estimate uncertainty from the spread of several stochastic predictions.

§5.5 Weight decay and the λ in $E + \lambda \sum w^2$

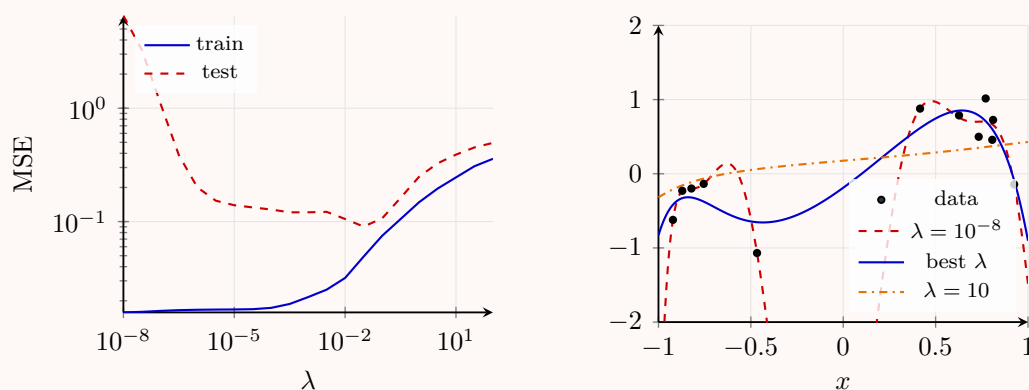
Prototypical example for this section: A degree-9 polynomial fitted to 12 noisy points: test MSE 6.551 with $\lambda = 10^{-8}$, 0.090 at $\lambda = 10^{-1.5}$, 0.390 with $\lambda = 10$.

Adding $\lambda \|\mathbf{w}\|^2$ to the training error E (**L2 regularisation**) penalises large weights. Its gradient $2\lambda\mathbf{w}$ adds a pull toward zero, so one SGD step multiplies the weights by $1 - 2\eta\lambda$ before the data gradient acts (with $\eta = 0.1$, $\lambda = 0.01$: by 0.998), hence the name **weight decay**. Small weights mean a smoother function: a large λ trades higher bias for lower variance. The ridge-regression chapter of *Classical Machine Learning* has the closed form; here we watch it work on a model with far too many parameters.

Example 5.5.1 (Effect of λ)

Degree-9 polynomial features, 12 training points from $\sin \pi x$ plus noise, closed-form fit (asserted equal to `sklearn`’s Ridge for all 21 values of λ):

λ	train MSE	test MSE	$\ \mathbf{w}\ $
10^{-8}	0.016	6.551	456.2
$10^{-1.5}$ (best)	0.049	0.090	3.0
10	0.245	0.390	0.2



Training error rises monotonically with λ ; test error is U-shaped. Too little: the curve threads every point with enormous coefficients (overfitting). Too much: it flattens toward the mean (underfitting).

Remark 5.5.2 — Trap. “Increasing λ increases model complexity” or “decreases training error”: both backwards. Larger λ means smaller weights, a simpler effective model, higher training error, and (up to a point) lower test error. $\lambda = 0$ removes

the regulariser; $\lambda \rightarrow \infty$ drives all penalised weights to zero. Biases are normally not penalised.

§5.5.i L2 is not weight decay under Adam

As [chapter 4](#) showed, under Adam the L2 gradient is divided by $\sqrt{\hat{v}}$. A clean demonstration: two parameters start at 1; parameter A receives noisy loss gradients of size about 1, parameter B receives none. After 2000 steps with $\eta = 10^{-3}$:

	$\lambda = 0.01$		$\lambda = 0.1$	
	A	B	A	B
Adam + L2	0.959	0.021	0.801	0.021
AdamW	0.959	0.980	0.799	0.819

Under Adam + L2 the parameter with no data gradient collapses toward zero *at the same rate for both values of λ* : its only gradient is λw , which Adam normalises to a step of about η per iteration whatever λ is. Under AdamW it decays by exactly $(1 - \eta\lambda)^t$, i.e. 0.980 and 0.819, so λ means what it says. That decoupling is why AdamW is the default for Transformers.

§5.6 Other regularisers, and reading the curves

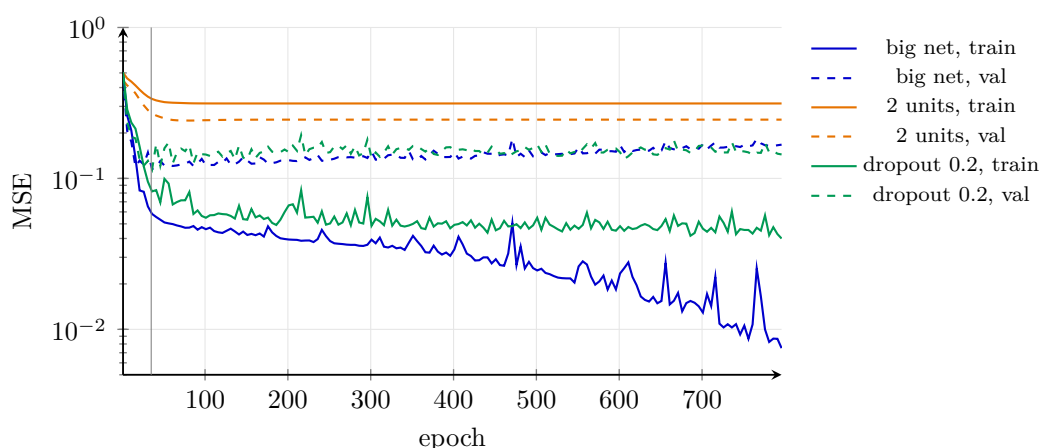
Prototypical example for this section: A 3-layer, width-128 network on 24 noisy points: training MSE falls to 0.008 while validation MSE, best at epoch 35, climbs back to 0.167.

§5.6.i Augmentation, early stopping and friends

- **Data augmentation**: label-preserving transformations (crops, flips, colour jitter, noise; for text, back-translation or token masking) enlarge the effective training set and encode invariances. Often the strongest regulariser for vision.
- **Early stopping**: monitor validation loss, keep the best checkpoint, stop after **patience** epochs without improvement. With patience 100 the run below stops at epoch 135 and restores epoch 35. For a quadratic loss and gradient descent, early stopping behaves much like L2 regularisation: both limit how far the weights travel from their (small) initial values.
- **Label smoothing** ([chapter 2](#)), **mixup** (train on convex combinations $\lambda \mathbf{x}_i + (1 - \lambda) \mathbf{x}_j$ with the same mix of labels), stochastic depth (randomly skipping residual blocks), and ensembling.
- Architectural restrictions: weight sharing (CNNs, RNNs), bottlenecks, smaller models.

§5.6.ii Over- and underfitting from training curves

The **generalisation gap** is validation (or test) loss minus training loss. Three runs on the same 24 points:



- **Overfitting** (big net): training loss keeps falling (final 0.008), validation loss bottoms out at epoch 35 (grey line) and rises; the final gap is 0.160. Remedies: more data or augmentation, regularisation, early stopping, a smaller model.
- **Underfitting** (2 hidden units): both losses plateau high (0.314 train, 0.245 validation) with a small gap. Remedies: a bigger model, longer training, less regularisation, better features.
- **Regularised** (dropout 0.2): training loss stays higher (0.043) and the curves stay closer together; here the validation loss ends at 0.152.

Remark 5.6.1 — Trap. Validation loss *below* training loss is not a bug by itself: dropout and augmentation are active when the training loss is recorded (here we evaluate both in `eval()` mode, so they are comparable), the training loss is often averaged over an epoch in which the model was still improving, and the validation set may simply be easier. It becomes a bug when the validation set overlaps the training set (leakage).

Initialise so that signals neither vanish nor explode (He for ReLU), normalise (BN for CNNs, LN for Transformers), regularise with weight decay (decoupled under Adam), dropout and augmentation, and read the gap between the two curves before reaching for any of them.

§5.7 A few problems to think about

Problem 5A. Give the Xavier-uniform bound and the He-normal standard deviation for a fully connected layer with 512 inputs and 512 outputs.

[hint · solution](#)

Problem 5B. A BatchNorm layer receives the batch (4, 4, 4, 4) for one feature. What is its training-mode output, and what goes wrong?

[hint · solution](#)

Problem 5C. A network uses dropout with $p = 0.2$ in training but someone deploys it with the non-inverted convention (no scaling during training, multiply by $1 - p$ at test time) while the weights were trained with inverted dropout. By what factor are the test-time activations off?

[hint · solution](#)

Problem 5D. Which statements are true? (a) BatchNorm uses batch statistics at inference. (b) LayerNorm does not depend on the batch size. (c) Dropout increases model capacity. (d) Increasing λ in $E + \lambda \sum w^2$ shrinks the weights. (e) He initialisation is designed for ReLU.

[hint](#) · [solution](#)

Problem 5E. Show that a linear layer followed by BatchNorm does not need a bias, and count the parameters of `Linear(256,128,bias=False)` followed by `BatchNorm1d(128)`.

[hint](#) · [solution](#)

Problem 5F. Training loss 0.02, validation loss 0.95 after 50 epochs; at epoch 8 validation loss was 0.41. Diagnose and list three fixes in the order you would try them.

[hint](#) · [solution](#)

II

Architectures

6 Convolutional networks

Convolutional networks are where placement OAs spend most of their deep-learning numerics: output shapes with padding and stride, parameter counts, “which layer reduces spatial size”, “why does a CNN have fewer parameters than a fully connected network”. This chapter gives one formula for each, checks every number against PyTorch, and then walks through the architectures an interviewer expects you to know by their key idea, ResNet above all.

§6.1 Why convolution

Prototypical example for this section: A fully connected layer from a $224 \times 224 \times 3$ image to just 1000 hidden units already has 150.5M parameters; a 3×3 convolution with 16 filters on a $32 \times 32 \times 3$ image has 448.

Images have three properties a fully connected layer ignores:

- **Locality:** nearby pixels are strongly related, distant ones weakly. So each output unit should look at a small neighbourhood (its **receptive field**), not the whole image (**sparse connectivity**).
- **Stationarity:** an edge detector useful in the top-left corner is useful everywhere. So the same weights are applied at every position (**weight sharing**).
- **Translation:** shifting the input should shift the feature map. Convolution is **translation equivariant**: $\text{conv}(\text{shift}(x)) = \text{shift}(\text{conv}(x))$ (up to borders). Pooling then adds a little translation *invariance*.

Example 6.1.1 (Weight sharing in numbers)

Producing a $32 \times 32 \times 16$ feature map from a $32 \times 32 \times 3$ image: a 3×3 convolution needs $(3 \cdot 3 \cdot 3 + 1) \cdot 16 = 448$ parameters. A fully connected layer producing the same 16,384 outputs from the 3,072 inputs needs 50,348,032, about 112,384 times more. The count of a convolution does not depend on the image size at all.

Remark 6.1.2 — Trap. “Why does a CNN have fewer parameters than a fully connected network on the same input?” The answer is *weight sharing plus local connectivity*, not pooling (pooling has no parameters) and not ReLU. And “fewer parameters” does not mean “less computation”: the shared kernel is applied at every position, so convolutions are cheap in parameters but expensive in multiply-adds.

§6.2 The convolution operation

Prototypical example for this section: A 3×3 kernel slid over a 4×4 input without padding gives a 2×2 output; the top-left entry is -3 .

§6.2.i Cross-correlation, not convolution

What every deep-learning library calls convolution is **cross-correlation**:

$$Y_{ij} = \sum_{u=0}^{k-1} \sum_{v=0}^{k-1} K_{uv} X_{i+u, j+v} (+b).$$

Mathematical convolution flips the kernel first ($K_{k-1-u, k-1-v}$). Since the kernel is learned, the flip is irrelevant: the network simply learns the flipped kernel.

1	2	0	1
0	1	3	2
2	1	0	1
1	0	2	3

★

1	0	-1
2	0	-2
1	0	-1

=

-3	-1
0	-4

input X (4×4)
kernel K
output Y

Example 6.2.1 (A 3×3 kernel by hand)

The shaded window times the kernel, non-zero kernel entries only: $(1)(1) + (0)(-1) + (0)(2) + (3)(-2) + (2)(1) + (0)(-1) = -3$. Sliding one step right and down gives the output $\begin{pmatrix} -3 & -1 \\ 0 & -4 \end{pmatrix}$, equal to `F.conv2d` and to SciPy's `correlate2d`. True convolution (SciPy's `convolve2d`) gives $\begin{pmatrix} 3 & 1 \\ 0 & 4 \end{pmatrix}$: this kernel is antisymmetric left-right, so flipping it negates the answer. With padding 1 the output stays 4×4 .

code/dl/c06_cnn.py

```

1 def conv2d_single(X, K, stride=1, pad=0):
2     """What deep-learning 'convolution' computes: cross-correlation (no flip)."""
3     X = np.pad(X, pad)
4     kh, kw = K.shape
5     H = (X.shape[0] - kh) // stride + 1
6     W = (X.shape[1] - kw) // stride + 1
7     Y = np.empty((H, W))
8     for i in range(H):
9         for j in range(W):
10             patch = X[i * stride:i * stride + kh, j * stride:j * stride + kw]
11             Y[i, j] = np.sum(patch * K) # elementwise product, then
12             ↪ sum
13     return Y

```

§6.2.ii Channels

Real layers map C_{in} input channels to C_{out} output channels. Each output channel has its own kernel of shape $C_{\text{in}} \times k \times k$ (it spans *all* input channels) and one bias; the layer's weight tensor in PyTorch has shape $(C_{\text{out}}, C_{\text{in}}, k, k)$:

$$Y_{o,i,j} = b_o + \sum_{c=1}^{C_{\text{in}}} \sum_{u,v} K_{o,c,u,v} X_{c, si+u, sj+v}.$$

§6.3 Output shapes

Prototypical example for this section: $32 \times 32 \times 3$ input, ten 5×5 filters, stride 1, no padding: output $28 \times 28 \times 10$, 760 parameters.

Theorem 6.3.1 (Convolution output size)

For input size n , kernel k , stride s , padding p (each side) and dilation d ,

$$n_{\text{out}} = \left\lfloor \frac{n + 2p - d(k - 1) - 1}{s} \right\rfloor + 1.$$

The same formula, with $p = 0$ by default, holds for pooling windows.

The dilated kernel spans $d(k - 1) + 1$ pixels; the formula counts how many positions of that span fit in the padded input when moving s pixels at a time. “Same” padding for odd k , $s = 1$, $d = 1$ is $p = (k - 1)/2$.

code/dl/c06_cnn.py

```
1 def conv_out(n, k, s=1, p=0, d=1):
2     return (n + 2 * p - d * (k - 1) - 1) // s + 1
```

Example 6.3.2 (Ten output sizes, each asserted against `nn.Conv2d`)

n	k	s	p	d	computation	n_{out}
32	5	1	0	1	$\lfloor (32 - 5)/1 \rfloor + 1$	28
32	3	1	1	1	$\lfloor (32 + 2 - 3)/1 \rfloor + 1$	32
224	7	2	3	1	$\lfloor (224 + 6 - 7)/2 \rfloor + 1$	112
227	11	4	0	1	$\lfloor (227 - 11)/4 \rfloor + 1$	55
28	3	2	1	1	$\lfloor (28 + 2 - 3)/2 \rfloor + 1$	14
64	3	1	2	2	$\lfloor (64 + 4 - 5)/1 \rfloor + 1$	64
7	3	2	0	1	$\lfloor (7 - 3)/2 \rfloor + 1$	3
10	4	3	1	1	$\lfloor (10 + 2 - 4)/3 \rfloor + 1$	3
56	1	1	0	1	$\lfloor (56 - 1)/1 \rfloor + 1$	56
112	3	2	1	1	$\lfloor (112 + 2 - 3)/2 \rfloor + 1$	56

Rows 3 and 4 are the first layers of ResNet- and AlexNet-style networks; row 6 shows that dilation 2 with padding 2 keeps the size; row 8 shows the floor discarding a leftover column.

Remark 6.3.3 — Trap. Forgetting the floor, applying the padding once instead of on both sides, or using k instead of $d(k - 1) + 1$ for a dilated kernel are the three ways to get these wrong. The channel count of the output is the number of filters, never the input channel count.

§6.4 Parameters and FLOPs

Prototypical example for this section: A 3×3 convolution from 16 to 32 channels has $(3 \cdot 3 \cdot 16 + 1) \cdot 32$ parameters, whatever the image size.

Proposition 6.4.1 (Counting a convolution)

A convolution with C_{in} input and C_{out} output channels and a $k \times k$ kernel has $(k^2 C_{\text{in}} + 1) C_{\text{out}}$ parameters, and producing an $H_{\text{out}} \times W_{\text{out}}$ map costs $k^2 C_{\text{in}} C_{\text{out}} H_{\text{out}} W_{\text{out}}$ multiply-accumulates (MACs), i.e. about twice that in FLOPs.

For the OA layer above: 760 parameters and $5 \cdot 5 \cdot 3 \cdot 10 \cdot 28 \cdot 28 = 588,000$ MACs. Papers and tools disagree on whether “FLOPs” means MACs or $2 \times \text{MACs}$; say which you mean.

Example 6.4.2 (A small CNN, layer by layer)

code/dl/c06_cnn.py

```
1 net = nn.Sequential(
2     nn.Conv2d(3, 16, 3, padding=1), nn.ReLU(),           # 32x32x16
3     nn.MaxPool2d(2),                                     # 16x16x16
4     nn.Conv2d(16, 32, 3, padding=1), nn.ReLU(),          # 16x16x32
5     nn.MaxPool2d(2),                                     # 8x8x32
6     nn.Conv2d(32, 64, 3, stride=2, padding=1), nn.ReLU(), # 4x4x64
7     nn.AdaptiveAvgPool2d(1), nn.Flatten(),               # 64
8     nn.Linear(64, 10))
```

For a $32 \times 32 \times 3$ input:

layer	output	parameters	MACs
conv 3×3 , 16	$32 \times 32 \times 16$	448	442,368
max-pool 2×2	$16 \times 16 \times 16$	0	0
conv 3×3 , 32	$16 \times 16 \times 32$	4,640	1,179,648
max-pool 2×2	$8 \times 8 \times 32$	0	0
conv 3×3 , 64, stride 2	$4 \times 4 \times 64$	18,496	294,912
global avg pool	$1 \times 1 \times 64$	0	0
linear $64 \rightarrow 10$	10	650	640
total		24,234	1,917,568

Every shape and parameter count is asserted against PyTorch. Observe the pattern of a CNN: spatial size shrinks, channels grow; the parameters concentrate in the late, wide layers, the computation in the early, large-resolution ones. Global average pooling before the classifier replaces what would otherwise be a $4 \cdot 4 \cdot 64 \rightarrow 10$ linear layer.

§6.5 Receptive field

Prototypical example for this section: Two stacked 3×3 convolutions see a 5×5 patch with $18C^2$ weights; one 5×5 needs $25C^2$.

The **receptive field** of a unit is the set of input pixels that can affect it. With $r_0 = 1$ and **jump** $j_0 = 1$ (distance in input pixels between adjacent units),

$$r_l = r_{l-1} + (k_l - 1) d_l j_{l-1}, \quad j_l = j_{l-1} s_l.$$

code/dl/c06_cnn.py

```
1 def receptive_field(layers):           # layers: list of (kernel, stride,
    ↪ dilation)
```

```

2   r, j = 1, 1                                # receptive field, jump (input pixels
   ↪ per step)
3   for k, s, d in layers:
4       r += (k - 1) * d * j
5       j *= s
6   return r

```

Example 6.5.1 (Receptive fields, formula against a gradient probe)

The script builds each stack, backpropagates from one central output unit and measures the width of the region of non-zero input gradient:

stack	receptive field
$3 \times 3, 3 \times 3$	5
3×3 three times	7
3×3 , max-pool 2 (stride 2), 3×3	8
3×3 stride 2, three times	15
3×3 with dilations 1, 2, 4	15
VGG-style: conv, conv, pool, conv, conv, pool, conv, conv, conv	40

This is the VGG argument: three 3×3 layers see 7×7 with $27C^2$ weights instead of $49C^2$, and insert two extra non-linearities. Strides and pooling grow the receptive field geometrically, dilation grows it without losing resolution (used in segmentation).

Remark 6.5.2 — Trap. The theoretical receptive field overstates what a unit actually uses: the gradient is concentrated near the centre and falls off toward the edges, so the *effective* receptive field is considerably smaller.

§6.6 Pooling, 1x1 and depthwise convolutions

Prototypical example for this section: Max-pooling 2×2 halves height and width, has 0 parameters, and sends the gradient only to the four winners.

§6.6.i Pooling

Max pooling outputs the maximum of each window, **average pooling** the mean. With 2×2 windows and stride 2:

$$X = \begin{pmatrix} 1 & 3 & 2 & 1 \\ 4 & 2 & 0 & 1 \\ 5 & 1 & 2 & 7 \\ 0 & 2 & 3 & 4 \end{pmatrix}, \quad \max = \begin{pmatrix} 4 & 2 \\ 5 & 7 \end{pmatrix}, \quad \text{avg} = \begin{pmatrix} 2.50 & 1.00 \\ 2.00 & 4.00 \end{pmatrix},$$

$$\frac{\partial \sum \max}{\partial X} = \begin{pmatrix} 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{pmatrix}.$$

This last matrix is PyTorch's gradient of the summed max-pool output: the max routes the gradient to the arg-max of each window, as in [chapter 3](#). **Global average pooling**

averages each channel over the whole map, giving one number per channel, and makes the network accept any input size.

Remark 6.6.1 — Trap. “Which layer reduces the spatial size of the feature map?” Pooling with stride > 1 , or a convolution with stride > 1 (or without padding). Not ReLU, not BatchNorm, not dropout, and not a 1×1 convolution with stride 1, which changes only the number of channels. And “pooling has learnable parameters” is false: max and average pooling have none.

§6.6.ii 1x1 convolutions

A 1×1 convolution is a fully connected layer applied independently at each pixel, mixing channels: $C_{\text{in}}C_{\text{out}} + C_{\text{out}}$ parameters. Uses: reducing channels before an expensive 3×3 (**bottleneck**), increasing them after, and adding non-linearity cheaply. A ResNet-style bottleneck $256 \rightarrow 64$ (1×1), $64 \rightarrow 64$ (3×3), $64 \rightarrow 256$ (1×1) has 69,632 weights, against 1,179,648 for two 3×3 layers at width 256.

§6.6.iii Depthwise-separable and grouped convolutions

A **depthwise** convolution applies one $k \times k$ filter per input channel (no channel mixing, `groups=C_in`); a **pointwise** 1×1 convolution then mixes channels. Together they replace a standard convolution (MobileNet):

3×3 , $256 \rightarrow 256$ (no bias)	weights
standard	589,824
depthwise 3×3	2,304
pointwise 1×1	65,536
depthwise-separable total	67,840
grouped, 4 groups	147,456

The ratio is $\frac{k^2 C_{\text{in}} + C_{\text{in}} C_{\text{out}}}{k^2 C_{\text{in}} C_{\text{out}}} = \frac{1}{C_{\text{out}}} + \frac{1}{k^2} = 0.115$, the same for MACs. A grouped convolution with g groups divides the weights by g .

§6.6.iv Transposed convolution

A **transposed convolution** (sometimes misnamed deconvolution) upsamples: each input value scatters a scaled copy of the kernel into the output, overlaps adding up. Its output size is

$$n_{\text{out}} = (n - 1)s - 2p + d(k - 1) + p_{\text{out}} + 1,$$

so $k = 4$, $s = 2$, $p = 1$ doubles 4 to 8, and $k = s = 2$ doubles 16 to 32. Input $\begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$ with kernel $\begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}$ and stride 1 gives $\begin{pmatrix} 1 & 3 & 2 \\ 4 & 9 & 4 \\ 3 & 4 & 0 \end{pmatrix}$, as `F.conv_transpose2d` does. The name is exact: it is multiplication by the transpose of the convolution’s matrix, i.e. the backward pass of a convolution with respect to its input (the script checks $\langle \text{conv}(x), y \rangle = \langle x, \text{conv}^\top(y) \rangle$). When k is not divisible by s , the overlaps are uneven and produce **checkerboard artefacts**; `resize-then-convolve` avoids them.

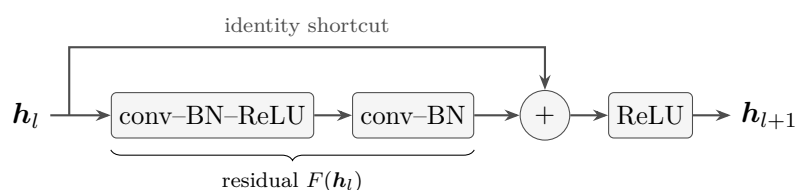
§6.7 Architectures worth knowing

Prototypical example for this section: ResNet’s identity shortcut: $\mathbf{h}_{l+1} = \mathbf{h}_l + F(\mathbf{h}_l)$.

- **LeNet** (1990s): conv–pool–conv–pool–FC for digits; the template.
- **AlexNet** (2012): a much larger LeNet trained on GPUs with ReLU, dropout and data augmentation; started the deep-learning era in vision.
- **VGG**: only 3×3 convolutions in stages separated by 2×2 max pools. Our 16-layer configuration (13 convolutions plus three FC layers, built in `code/dl/c06_cnn.py`) has 138.4 M parameters, of which 89% sit in the FC head (the first FC layer alone, $7 \cdot 7 \cdot 512 \rightarrow 4096$, has 102.8 M) and only 14.7 M in the convolutions. That imbalance is why later networks use global average pooling.
- **Inception** (GoogLeNet): parallel 1×1 , 3×3 , 5×5 and pooling branches concatenated along channels, with 1×1 bottlenecks to keep the cost down; multi-scale features in one block.
- **ResNet** [He+16]: residual blocks, below. Our 18-layer configuration (basic blocks [2, 2, 2, 2], widths 64–512, BN, 1000-way head) counts 11.7 M parameters.
- **DenseNet**: each layer receives the concatenation of all previous feature maps in its block; strong feature reuse, few parameters, more memory.
- **MobileNet**: depthwise-separable convolutions (and in later versions inverted residual bottlenecks) for phones.
- **EfficientNet**: scales depth, width and input resolution together with one compound coefficient instead of growing one of them alone.

§6.7.i ResNet: the degradation problem

Adding layers to a plain network beyond a point makes *training* error worse, not just test error. That is not overfitting (overfitting lowers training error); it is an optimisation failure, since a deeper net could always copy the shallower one and set the extra layers to the identity, yet gradient descent does not find that solution. ResNet makes the identity the default: a block computes $\mathbf{h} + F(\mathbf{h})$, so it only has to learn the residual F , and $F = 0$ is easy to reach.



Gradient flow: $\partial \mathbf{h}_{l+1} / \partial \mathbf{h}_l = \mathbf{I} + \partial F / \partial \mathbf{h}_l$, so the backward product over blocks always contains the identity path (Problem 3E): the gradient from the loss reaches early layers undiminished, even when every $\partial F / \partial \mathbf{h}$ is small. When the shape changes (stride 2, more channels) the shortcut becomes a 1×1 convolution with stride 2. A basic block at width 64 has $2 \cdot 9 \cdot 64^2 + 4 \cdot 64 = 73,984$ parameters (two convolutions without bias, two BNs).

`code/dl/c06_resnet.py`

```
1 class Block(nn.Module):
2     def __init__(self, d, residual):
3         super().__init__()
4         self.lin, self.bn, self.residual = nn.Linear(d, d), nn.BatchNorm1d(d),
        ↪ residual
```

```

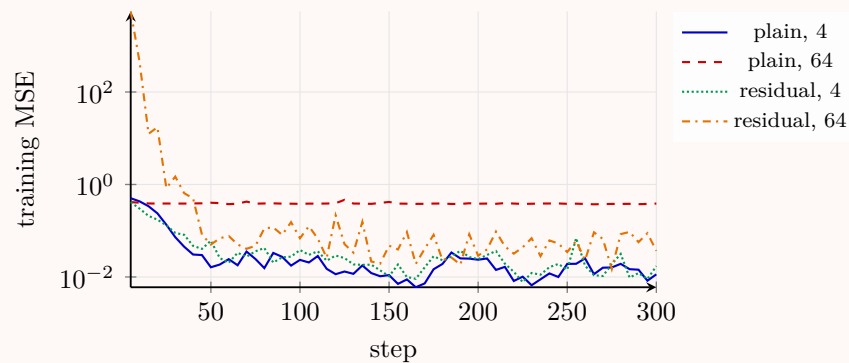
5
6     def forward(self, h):
7         f = torch.relu(self.bn(self.lin(h)))
8         return h + f if self.residual else f      # the only difference: "+ h"

```

Example 6.7.1 (Degradation, reproduced on a small scale)

MLPs of width 32 with BatchNorm, the same data (a 2-D regression), Adam, 300 steps; the only difference between a plain and a residual net is the “+ h”. Full-data training MSE at the end, and the gradient norm reaching the first layer at step 1:

blocks	plain MSE	residual MSE	plain $\ \nabla \mathbf{W}^{(1)}\ $	residual $\ \nabla \mathbf{W}^{(1)}\ $
4	0.0136	0.0159	1.68	0.91
16	0.0153	0.0402	4.70	6.34
64	0.3806	0.0721	114788	21.13



The 64-block plain network trains far worse than the 4-block one even though it contains it as a special case; with BatchNorm its first-layer gradient at initialisation is enormous rather than tiny, another face of the same ill-conditioning. The residual version of the same depth trains. In this short run the deep residual net does not beat the shallow one; making very deep nets *better*, not merely trainable, takes the scale and the tricks of the full recipe.

Remark 6.7.2 — Trap. “ResNet solves overfitting” or “ResNet works because it has more parameters”. The shortcut adds no parameters (identity) and addresses an optimisation problem: degradation of training accuracy with depth, tied to poor gradient flow.

§6.8 Transfer learning and fine-tuning

Prototypical example for this section: Take a network pretrained on a large image dataset, replace its last layer with a 5-class head, and train on 2,000 labelled images of your own.

Features learned on a large dataset transfer: early layers detect edges and textures, middle layers parts, late layers task-specific concepts. Two regimes:

- **Feature extraction:** freeze the pretrained backbone (`requires_grad=False`), train only a new head. Fast, needs little data, cannot adapt the features.

- **Fine-tuning**: initialise from the pretrained weights and continue training some or all layers on the new task, usually with a learning rate 10–100× smaller than from scratch (often smaller still for early layers), so the useful features are adjusted rather than destroyed.

	similar to pretraining data	different
small dataset	feature extraction (train head)	features from earlier layers + head
large dataset	fine-tune most or all layers	fine-tune all (pretraining still helps init)

Remark 6.8.1 — Trap. “Fine-tuning means training only the last layer.” That is feature extraction (linear probing). Fine-tuning updates pretrained weights. Two practical traps: preprocess inputs exactly as in pretraining (same normalisation statistics and resolution), and with frozen BatchNorm layers keep them in `eval()` mode, or their running statistics drift to the new data even though their weights are frozen.

§6.9 Detection and segmentation, in one section

Prototypical example for this section: Two 4×4 boxes offset by (2, 2) overlap in 4 pixels out of 28: IoU 0.1429.

Object detection outputs boxes with class labels and scores. Its evaluation unit is **intersection over union**, $\text{IoU}(A, B) = |A \cap B| / |A \cup B|$; a detection counts as correct if IoU with a ground-truth box exceeds a threshold (often 0.5), and **mAP** averages the area under precision–recall curves over classes (and, in some benchmarks, over IoU thresholds).

code/dl/c06_cnn.py

```

1 def iou(a, b):                                     # boxes as (x1, y1, x2, y2)
2     iw = max(0.0, min(a[2], b[2]) - max(a[0], b[0]))
3     ih = max(0.0, min(a[3], b[3]) - max(a[1], b[1]))
4     inter = iw * ih
5     area = lambda r: (r[2] - r[0]) * (r[3] - r[1])
6     return inter / (area(a) + area(b) - inter)
7
8 def nms(boxes, scores, thr=0.5):
9     order = list(np.argsort(-scores))               # highest score first
10    keep = []
11    while order:
12        i = order.pop(0)
13        keep.append(i)
14        order = [j for j in order if iou(boxes[i], boxes[j]) <= thr]
15    return keep

```

Detectors propose many overlapping boxes per object; **non-maximum suppression** keeps the highest-scoring box and deletes others that overlap it by more than a threshold. With six boxes (two clusters of near-duplicates plus one isolated box, pairwise IoUs within a cluster of 0.822, 0.907 and 0.858) NMS at 0.5 keeps boxes 1, 3, 6, matching an independent vectorised implementation.

- **Two-stage** (R-CNN family): propose regions, then classify and refine each. R-CNN ran a CNN per proposal; Fast R-CNN shares one feature map and pools each region

from it; Faster R-CNN learns the proposals with a region proposal network over **anchors** (reference boxes of several scales and aspect ratios at every position, from which offsets are regressed).

- **One-stage** (YOLO, SSD, RetinaNet): predict class scores and box offsets densely over a grid or anchors in one pass; faster, historically less accurate on small objects; focal loss ([chapter 2](#)) was introduced for the extreme foreground–background imbalance this creates.
- **Semantic segmentation** labels every pixel. **Fully convolutional networks** replace FC layers by convolutions and upsample; **U-Net** [RoFiBr15] is an encoder–decoder whose skip connections concatenate high-resolution encoder features into the decoder to recover fine boundaries. Instance segmentation (e.g. Mask R-CNN) adds a mask head per detected box.

Shapes: $\lfloor (n + 2p - d(k - 1) - 1)/s \rfloor + 1$. **Parameters:** $(k^2 C_{\text{in}} + 1)C_{\text{out}}$. **Weight sharing explains the small counts; stride and pooling shrink space; residual shortcuts make depth trainable.**

§6.10 A few problems to think about

Problem 6A. An input of $64 \times 64 \times 3$ passes through: conv 3×3 , 32 filters, stride 1, padding 1; max-pool 2×2 stride 2; conv 5×5 , 64 filters, stride 2, padding 2. Give the output shape and the number of parameters after each layer.

[hint · solution](#)

Problem 6B. A 7×7 input is convolved with a 3×3 kernel with stride 3 and no padding. What is the output size, and how many input pixels are never used?

[hint · solution](#)

Problem 6C. Compute the receptive field of: conv 3×3 (s1), conv 3×3 (s2), conv 3×3 (s1), max-pool 2×2 (s2), conv 3×3 (s1).

[hint · solution](#)

Problem 6D. How many parameters does a depthwise-separable convolution 3×3 , $128 \rightarrow 256$ have (with biases on both parts), and how does that compare with a standard 3×3 convolution?

[hint · solution](#)

Problem 6E. A plain 56-layer CNN has higher *training* error than a 20-layer one. A colleague suggests dropout. Respond.

[hint · solution](#)

Problem 6F. Box $A = (0, 0, 10, 10)$ and box $B = (5, 0, 15, 10)$ (corners x_1, y_1, x_2, y_2). Compute their IoU. Does NMS with threshold 0.5 suppress the lower-scoring one?

[hint · solution](#)

7 Recurrent networks

An MLP or a CNN maps a fixed-size input to an output. Text, speech, sensor streams and prices arrive as sequences of varying length, where the meaning of an element depends on what came before. Recurrent networks handle this with a loop: a hidden state carried from step to step. OAs ask why feedback connections help, why LSTMs suit time series, and LSTM parameter counts; interviews ask for the LSTM equations, why gradients vanish through time, and why Transformers have largely replaced RNNs.

§7.1 The vanilla RNN

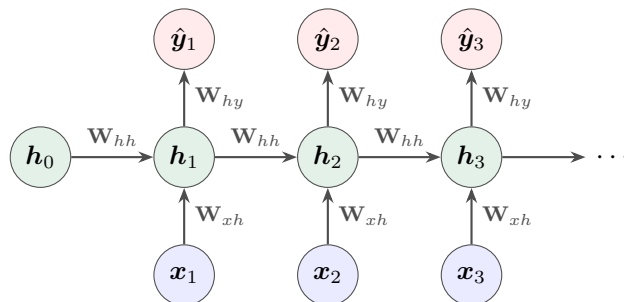
Prototypical example for this section: A one-unit RNN with $w_x = 0.5$, $w_h = 0.8$ reads $(1, 0.5, -1)$ and its state goes $0.4621 \rightarrow 0.5509 \rightarrow -0.0592$.

§7.1.i Recurrence

A **recurrent neural network** keeps a hidden state $\mathbf{h}_t \in \mathbb{R}^H$ and updates it with the same weights at every step:

$$\mathbf{h}_t = \tanh(\mathbf{W}_{xh}\mathbf{x}_t + \mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{b}_h), \quad \hat{\mathbf{y}}_t = \mathbf{W}_{hy}\mathbf{h}_t + \mathbf{b}_y, \quad \mathbf{h}_0 = \mathbf{0}.$$

The **feedback connection** $\mathbf{W}_{hh}\mathbf{h}_{t-1}$ is what makes it recurrent: $\mathbf{h}_t$ is a summary of $\mathbf{x}_1, \dots, \mathbf{x}_t$, so the output at time t can depend on the whole past. Because the weights are shared across time (weight sharing again, now along the time axis), the same network handles any sequence length with a fixed number of parameters.



The unrolled graph is a deep feed-forward network with one layer per time step and tied weights.

Example 7.1.1 (Three steps by hand)

$H = 1$, $w_x = 0.5$, $w_h = 0.8$, no bias, inputs 1, 0.5, -1: $h_1 = \tanh(0.5) = 0.4621$; $h_2 = \tanh(0.25 + 0.8 \cdot 0.4621) = \tanh(0.6197) = 0.5509$; $h_3 = \tanh(-0.5 + 0.8 \cdot 0.5509) = \tanh(-0.0593) = -0.0592$. Identical to `nn.RNN` with these weights.

Remark 7.1.2 — Trap. “Why are feedback connections used in RNNs?” Not to speed up training and not to reduce overfitting: they let the network carry

information from earlier inputs to later steps (memory), so outputs can depend on context and sequences of any length are handled with a fixed set of weights.

§7.1.ii Parameter counts

An RNN layer has $\mathbf{W}_{xh} \in \mathbb{R}^{H \times D}$, $\mathbf{W}_{hh} \in \mathbb{R}^{H \times H}$ and a bias, so $H(D+H)+H$ parameters. PyTorch keeps two bias vectors (`bias_ih` and `bias_hh`), giving $H(D+H)+2H$; say which convention you use. For $D=10$, $H=20$: 640 in PyTorch. The output layer $\mathbf{W}_{hy}$ is counted separately.

§7.2 Backpropagation through time

Prototypical example for this section: In the example above $\partial h_3 / \partial h_1 = w_h^2 (1-h_3^2)(1-h_2^2) = 0.4442$: every step back multiplies by w_h and a tanh derivative.

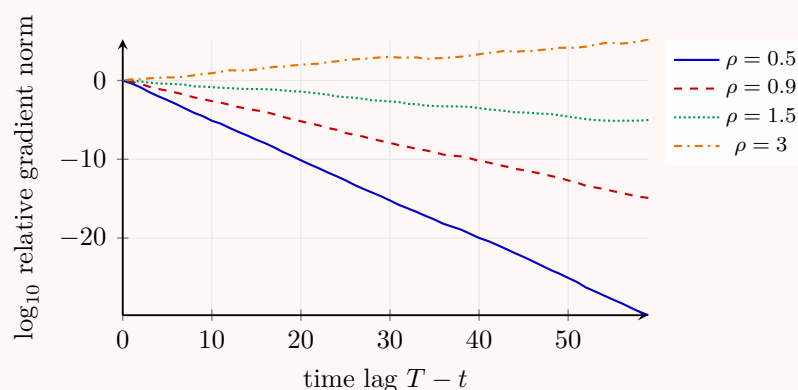
Training unrolls the network over the sequence and backpropagates through the unrolled graph: **backpropagation through time** (BPTT). Since the weights are shared, the gradient for $\mathbf{W}_{hh}$ is the sum of its gradients at every step. The gradient reaching an early state is

$$\frac{\partial \mathcal{L}_T}{\partial \mathbf{h}_k} = \left(\prod_{t=k+1}^T \mathbf{W}_{hh}^\top \text{diag}(1 - h_t^2) \right) \frac{\partial \mathcal{L}_T}{\partial \mathbf{h}_T},$$

the product of Jacobians of [chapter 3](#), now with the *same* matrix $\mathbf{W}_{hh}$ repeated $T-k$ times. If the largest singular value of $\mathbf{W}_{hh}$ times the tanh factor is below one the product shrinks exponentially in the time lag (vanishing); if it is above one it can grow exponentially (exploding).

Example 7.2.1 (Gradient through 60 steps)

$H=64$, $\mathbf{W}_{hh}$ random and rescaled to spectral radius ρ , Gaussian inputs; norm of $\partial(\sum \mathbf{h}_{60}) / \partial \mathbf{h}_t$ at lag 59 relative to lag 0: 1.6×10^{-30} for $\rho=0.5$, 1.2×10^{-15} for $\rho=0.9$, 9.8×10^{-6} for $\rho=1.5$, 1.7×10^5 for $\rho=3$.



Note that $\rho=1.5$ still vanishes: large weights saturate the tanh, whose derivative then shrinks the product. A vanilla RNN sits between two cliffs.

Remedies: **gradient clipping** by norm for explosions ([chapter 4](#); standard for RNNs); **truncated BPTT**, which backpropagates only k steps into the past (cheaper, but

blind to longer dependencies); orthogonal initialisation of $\mathbf{W}_{hh}$; and, above all, gated architectures.

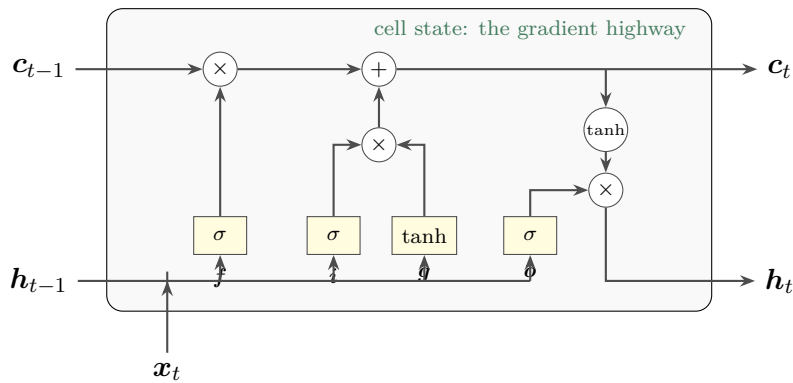
§7.3 LSTM and GRU

Prototypical example for this section: An LSTM's cell state is updated additively, $\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \mathbf{g}_t$, so the gradient along the cell path is multiplied only by the forget gate.

§7.3.i The LSTM cell

The **long short-term memory** [HoSc97] adds a **cell state** $\mathbf{c}_t$ and three gates, each a sigmoid in $(0, 1)$ computed from $\mathbf{x}_t$ and $\mathbf{h}_{t-1}$:

$$\begin{aligned} \mathbf{i}_t &= \sigma(\mathbf{W}_i \mathbf{x}_t + \mathbf{U}_i \mathbf{h}_{t-1} + \mathbf{b}_i) && \text{input gate: how much to write} \\ \mathbf{f}_t &= \sigma(\mathbf{W}_f \mathbf{x}_t + \mathbf{U}_f \mathbf{h}_{t-1} + \mathbf{b}_f) && \text{forget gate: how much to keep} \\ \mathbf{g}_t &= \tanh(\mathbf{W}_g \mathbf{x}_t + \mathbf{U}_g \mathbf{h}_{t-1} + \mathbf{b}_g) && \text{candidate content} \\ \mathbf{o}_t &= \sigma(\mathbf{W}_o \mathbf{x}_t + \mathbf{U}_o \mathbf{h}_{t-1} + \mathbf{b}_o) && \text{output gate: how much to expose} \\ \mathbf{c}_t &= \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \mathbf{g}_t, && \mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t). \end{aligned}$$



Example 7.3.1 (One LSTM step with a scalar state)

$x_t = 1$, $h_{t-1} = 0$, $c_{t-1} = 0.5$; input weights 0.5, 1, 1, 0.5 for i, f, g, o , all other weights and biases 0. Then $i = \sigma(0.5) = 0.6225$, $f = \sigma(1) = 0.7311$, $g = \tanh(1) = 0.7616$, $o = \sigma(0.5) = 0.6225$; $c_t = 0.7311 \cdot 0.5 + 0.6225 \cdot 0.7616 = 0.8396$ and $h_t = 0.6225 \tanh(0.8396) = 0.4268$.

code/dl/c07_rnn.py

```
1 def lstm_cell(x, h, c, W, U, b):
2     """W: (4H, D), U: (4H, H), b: (4H,), gates stacked in PyTorch order i, f, g, o.
3     ↪ """
4     z = W @ x + U @ h + b
5     i, f, g, o = np.split(z, 4)
6     i, f, o = sig(i), sig(f), sig(o)          # input, forget, output gates in (0,
7     ↪ 1)
8     g = np.tanh(g)                            # candidate cell update
9     c = f * c + i * g                        # cell state: forget some, write some
10    h = o * np.tanh(c)                        # hidden state: expose some of the
11    ↪ cell
12    return h, c
```

The script checks this cell on random weights against `nn.LSTMCell` (PyTorch stacks the gates in the order i, f, g, o in one $(4H, D)$ matrix).

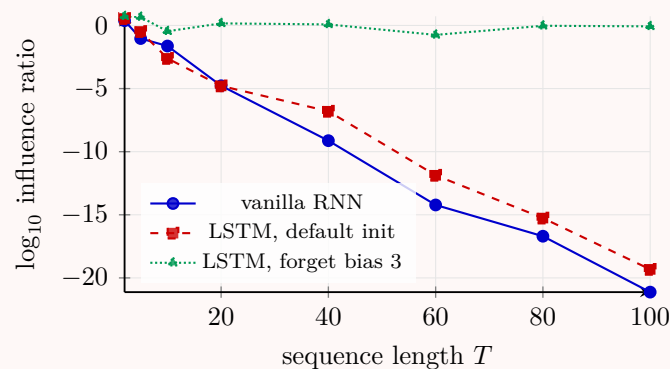
Proposition 7.3.2 (Why the LSTM remembers)

Along the cell path, $\partial \mathbf{c}_t / \partial \mathbf{c}_{t-1} = \text{diag}(\mathbf{f}_t)$ (holding the gates fixed), so the gradient from $\mathbf{c}_T$ to $\mathbf{c}_k$ is the elementwise product $\prod_{t=k+1}^T \mathbf{f}_t$. If the network keeps $\mathbf{f}_t \approx \mathbf{1}$, information and gradients survive many steps, with no repeated matrix product and no tanh squashing in between.

With $f = 0.95$ the signal after 50 steps is $0.95^{50} = 0.077$, while a vanilla RNN path whose per-step factor is at most 0.5 gives at most $0.5^{50} = 8.9 \times 10^{-16}$. The gates are learned, so the network decides what to remember. The gradient through $\mathbf{h}$ still vanishes; the additive cell path is the bypass, the same idea as the ResNet shortcut, applied in time.

Example 7.3.3 (Influence of the first input, at initialisation)

Ratio of the average gradient of the final state with respect to $\mathbf{x}_1$ to that with respect to $\mathbf{x}_T$ ($H = 32$), as the sequence length T grows. At $T = 20$: vanilla RNN 1.7×10^{-5} , LSTM with PyTorch's default initialisation 1.6×10^{-5} , LSTM with forget-gate bias 3 ($f \approx \sigma(3) = 0.953$ at the start) 1.46.



The architecture makes long memory *possible*; at a default initialisation the forget gate sits near $\sigma(0) = 0.5$ and the LSTM forgets as fast as the RNN. Initialising the forget-gate bias to a positive value (commonly 1 or more) is a standard trick for this reason.

§7.3.ii GRU

The **gated recurrent unit** [Ch+14] merges the cell and hidden state and uses two gates:

$$\begin{aligned} \mathbf{r}_t &= \sigma(\mathbf{W}_r \mathbf{x}_t + \mathbf{U}_r \mathbf{h}_{t-1} + \mathbf{b}_r), & \mathbf{z}_t &= \sigma(\mathbf{W}_z \mathbf{x}_t + \mathbf{U}_z \mathbf{h}_{t-1} + \mathbf{b}_z), \\ \mathbf{n}_t &= \tanh(\mathbf{W}_n \mathbf{x}_t + \mathbf{r}_t \odot (\mathbf{U}_n \mathbf{h}_{t-1} + \mathbf{b}_{hn}) + \mathbf{b}_{in}), & \mathbf{h}_t &= (1 - \mathbf{z}_t) \odot \mathbf{n}_t + \mathbf{z}_t \odot \mathbf{h}_{t-1}. \end{aligned}$$

The update gate $\mathbf{z}$ interpolates between keeping the old state and writing the candidate (the same additive path); the reset gate $\mathbf{r}$ decides how much of the past feeds the candidate. Fewer parameters than an LSTM and often similar accuracy. (Conventions differ on whether $\mathbf{z}$ or $1 - \mathbf{z}$ multiplies $\mathbf{h}_{t-1}$; the code follows PyTorch and is checked against `nn.GRUCell`.)

code/dl/c07_rnn.py

```

1 def gru_cell(x, h, W, U, bw, bu):
2     """PyTorch gate order r, z, n; note r multiplies (U_n h + b_un)."""
3     a, bb = W @ x + bw, U @ h + bu
4     ar, az, an = np.split(a, 3); br, bz, bn = np.split(bb, 3)
5     r, z = sig(ar + br), sig(az + bz)      # reset and update gates
6     n = np.tanh(an + r * bn)              # candidate
7     return (1 - z) * n + z * h            # interpolate old and new

```

§7.3.iii Counting parameters

Proposition 7.3.4

With input size D and hidden size H , one layer has (PyTorch convention, two bias vectors per gate block):

$$\text{RNN: } H(D+H) + 2H, \quad \text{GRU: } 3[H(D+H) + 2H], \quad \text{LSTM: } 4[H(D+H) + 2H].$$

With a single bias per gate replace $2H$ by H . A stacked layer $l > 1$ has input size H (or $2H$ if the layer below is bidirectional); a bidirectional layer doubles the count.

Example 7.3.5 (LSTM counts, asserted against `nn.LSTM`)

configuration	PyTorch count	one-bias count
LSTM, $D = 100$, $H = 128$	117,760	117,248
GRU, $D = 100$, $H = 128$	88,320	
RNN, $D = 100$, $H = 128$	29,440	
LSTM, $D = 10$, $H = 20$	2,560	2,480
LSTM, 2 layers	5,920	
LSTM, bidirectional	5,120	
LSTM, 2 layers, bidirectional	15,040	

An LSTM has exactly four times the parameters of a vanilla RNN of the same size, a GRU three times.

Remark 7.3.6 — Trap. In PyTorch, `nn.LSTM` expects input of shape (T, B, D) unless `batch_first=True`. With `batch_first=True`, 2 layers, bidirectional, $H = 20$ and a batch of 4 sequences of length 7, the output has shape $(4, 7, 40)$ (both directions concatenated, last layer only) while $\mathbf{h}_n$ has shape $(4, 4, 20)$ (layers $\times$ directions first, *not* batch first). Feeding (B, T, D) without the flag silently treats the batch as time.

§7.4 Sequence-to-sequence, teacher forcing and beam search

Prototypical example for this section: A decoder whose most likely first token leads to unlikely continuations: greedy decoding finds a sequence of probability 0.0800, beam search with width 2 finds 0.1440.

§7.4.i Architectures

Recurrent layers can be **stacked** (the hidden sequence of one layer is the input of the next) and made **bidirectional** (one RNN reads left to right, another right to left, and their states are concatenated), which helps whenever the whole input is available, as in tagging or encoding a sentence, but not for generating text left to right. Input/output patterns: many-to-one (classification), many-to-many aligned (tagging), and many-to-many unaligned, the **encoder–decoder** or **seq2seq** model: an encoder RNN reads the source into a final state, which initialises a decoder RNN that generates the target one token at a time, feeding back each generated token. Squeezing a whole sentence into one fixed vector is the bottleneck that attention ([chapter 8](#)) removes.

§7.4.ii Teacher forcing

During training the decoder is fed the *true* previous token rather than its own prediction: **teacher forcing**. It makes training stable and parallel across time steps (for the loss), but at inference the model must consume its own outputs, a distribution it never saw during training (**exposure bias**); early mistakes can compound. Scheduled sampling, which gradually feeds the model’s own predictions during training, is one mitigation.

§7.4.iii Beam search

Decoding should find $\arg \max_y \prod_t P(y_t \mid y_{<t}, \mathbf{x})$, but the number of sequences is V^T . **Greedy** decoding takes the best token at each step; **beam search** keeps the k best partial sequences (by total log-probability) at each step.

code/dl/c07_rnn.py

```
1 def beam_search(k, T, start=0):
2     beams = [((), 0.0)] # (tokens, total log-prob)
3     for t in range(T):
4         cand = []
5         for seq, s in beams:
6             prev = seq[-1] if seq else start
7             lp = logp(prev, t)
8             cand += [(seq + (v,), s + lp[v]) for v in range(V)]
9         beams = sorted(cand, key=lambda c: -c[1])[:k] # keep the k best prefixes
10    return beams[0]
```

Example 7.4.1 (Greedy is not optimal)

Tokens a, b, c ; $P(\text{first}) = (0.5, 0.4, 0.1)$; after a the next token is a, b, c with probabilities $(0.4, 0.3, 0.3)$, after b $(0.9, 0.05, 0.05)$, after c $(0.34, 0.33, 0.33)$; three steps. Greedy picks a first and returns aaa with probability 0.0800. Beam search with $k = 2$ keeps b alive and returns baa with probability 0.1440, which brute-force enumeration of all 27 sequences confirms is the optimum. With $k = V^T$ beam search is exhaustive (asserted).

Beam search favours short outputs (each extra token multiplies by a probability below one), so scores are usually length-normalised. For open-ended generation, sampling methods ([chapter 9](#)) produce less repetitive text than beam search.

§7.5 RNNs for time series, and where Transformers win

Prototypical example for this section: Forecasting tomorrow’s load from the last 60 days of hourly readings.

Why LSTMs suit time series: they process observations in order with a state that summarises the past, handle variable-length histories, update the forecast online in $O(1)$ per new observation without reprocessing history, and their gates can keep slow trends while reacting to recent changes. They need comparatively little data and memory. In practice, for many tabular forecasting problems strong baselines (seasonal naive, exponential smoothing, gradient-boosted trees on lag features) are hard to beat, and should be tried first.

Where Transformers ([chapter 8](#)) beat RNNs:

- **Parallelism:** an RNN must compute $\mathbf{h}_1, \mathbf{h}_2, \dots$ one after another; self-attention processes all positions of a training sequence at once, so it uses GPUs far better.
- **Path length:** information between positions i and j passes through $|i - j|$ recurrent steps in an RNN but one attention step in a Transformer, so long-range dependencies are easier to learn.
- **Scaling:** with enough data and compute, Transformers keep improving with size.

Their cost: attention is quadratic in sequence length, and generation needs a growing KV cache rather than a fixed-size state; this is why recurrent-style and state-space models remain active research for very long sequences.

Remark 7.5.1 — Trap. “LSTMs solve the vanishing gradient problem completely.” They mitigate it along the cell path when the forget gate stays open; with $f < 1$ the signal still decays geometrically, and the gradient through $\mathbf{h}$ still vanishes. LSTMs also do not fix *exploding* gradients: clip them.

§7.6 CTC, briefly

Prototypical example for this section: The alignment `hh-e-ll-ll-oo` collapses to `hello`: merge repeats, then delete blanks.

In speech or handwriting recognition the input sequence (audio frames) is much longer than the label sequence and the alignment is unknown. **Connectionist temporal classification** [Gr+06] lets the network emit, per frame, a distribution over labels plus a **blank** symbol. An **alignment** (one symbol per frame) maps to a label sequence by merging repeated symbols and then deleting blanks; the blank is what allows genuine double letters (l-l). The CTC loss is $-\log \sum_{\text{alignments } \pi \rightarrow \mathbf{y}} \prod_t P(\pi_t | \mathbf{x})$, computed by dynamic programming.

code/dl/c07_rnn.py

```
1 def collapse(path, blank=0):
2     outp, prev = [], None
3     for s in path:
4         if s != prev and s != blank:                # merge repeats, then drop blanks
5             outp.append(s)
6             prev = s
7     return tuple(outp)
```

Example 7.6.1 (CTC by brute force)

Four frames, symbols $\{-, a, b\}$, target ab . Of the 81 possible alignments exactly 15 collapse to ab : $-ab$, $-a-b$, $-aab$, $-ab-$, $-abb$, $a-b$, $a-b-$, $a-bb$, $aa-b$, $aaab$, $aab-$, $aabb$, $ab-$, $abb-$, $abbb$. Summing their probabilities under random per-frame softmaxes gives a loss of 0.9381, equal to `F.ctc_loss` (which uses the dynamic programme instead of enumeration).

An RNN is a deep network unrolled in time with tied weights; its gradients are repeated matrix products, hence vanishing and exploding. LSTM and GRU add an additive, gated memory path; Transformers replace recurrence with attention for parallelism and short paths.

§7.7 A few problems to think about

Problem 7A. How many parameters does a single-layer LSTM with input size 50 and hidden size 64 have (PyTorch convention)? A GRU of the same size?

[hint · solution](#)

Problem 7B. A bidirectional single-layer LSTM with hidden size 64 feeds a linear layer that classifies the final output into 5 classes. What is the input size of the linear layer, and how many parameters does it have?

[hint · solution](#)

Problem 7C. For the scalar RNN $h_t = \tanh(wh_{t-1} + ux_t)$, show $|\partial h_T / \partial h_k| \leq |w|^{T-k}$ and conclude what happens for $|w| < 1$. Why does $|w| > 1$ not guarantee an explosion?

[hint · solution](#)

Problem 7D. Why is a bidirectional LSTM unsuitable for next-word prediction, but fine for named-entity tagging?

[hint · solution](#)

Problem 7E. In an LSTM, what values of the forget gate f and input gate i make the cell copy its previous content unchanged, and what values make it overwrite the content completely? Which gate's bias would you initialise to a positive value, and why?

[hint · solution](#)

Problem 7F. With $V = 10,000$ tokens and output length 20, how many sequences does exhaustive decoding consider? How many partial hypotheses does beam search with $k = 5$ score at each step?

[hint · solution](#)

8 Attention and the Transformer

The Transformer [Va+17] is the architecture behind essentially every large language model and much of modern vision and speech. It is also the most common “implement it from scratch” request in ML interviews, from founder rounds at start-ups to research-engineer loops. This chapter builds it piece by piece with the shapes written out at every step, derives why the scores are divided by $\sqrt{d_k}$, counts parameters and FLOPs, and ends with a capstone: a complete encoder–decoder Transformer written in PyTorch and again in NumPy, trained on a toy task, with the two implementations and PyTorch’s own layers asserted to agree.

§8.1 Attention as a soft dictionary lookup

Prototypical example for this section: A decoder translating word t looks back at all encoder states and takes a weighted average, with weights from a softmax over relevance scores.

A Python dictionary returns the value whose key *equals* the query. Attention returns a blend of *all* values, weighted by how well each key matches the query:

$$\text{output} = \sum_i \alpha_i \mathbf{v}_i, \quad \alpha_i = \frac{\exp(\text{score}(\mathbf{q}, \mathbf{k}_i))}{\sum_j \exp(\text{score}(\mathbf{q}, \mathbf{k}_j))}.$$

Because the weights come from a softmax, the lookup is differentiable and can be learned.

Attention entered deep learning to fix the seq2seq bottleneck of chapter 7: instead of squeezing the source sentence into one vector, the decoder at step t computes a fresh **context vector** $\mathbf{c}_t = \sum_i \alpha_{ti} \mathbf{h}_i$ over all encoder states $\mathbf{h}_i$.

- **Additive (Bahdanau) attention** [BaChBe15] scores with a one-hidden-layer network on the previous decoder state and an encoder state, $\text{score}(\mathbf{s}_{t-1}, \mathbf{h}_i) = \mathbf{v}^\top \tanh(\mathbf{W}\mathbf{s}_{t-1} + \mathbf{U}\mathbf{h}_i)$.
- **Multiplicative (Luong) attention:** $\text{score}(\mathbf{s}_t, \mathbf{h}_i) = \mathbf{s}_t^\top \mathbf{W}\mathbf{h}_i$ (or just the dot product $\mathbf{s}_t^\top \mathbf{h}_i$). Cheaper: one matrix multiplication for all pairs.

The Transformer keeps the dot product, adds a scale, and, crucially, uses attention between positions of the *same* sequence (**self-attention**) as its main operation, dropping recurrence entirely.

§8.2 Scaled dot-product attention

Prototypical example for this section: Three tokens with 2-dimensional queries, keys and values; the first output row is (3.000, 4.000).

§8.2.i Definition and a worked example

Stack queries, keys and values as rows: $\mathbf{Q} \in \mathbb{R}^{T_q \times d_k}$, $\mathbf{K} \in \mathbb{R}^{T_k \times d_k}$, $\mathbf{V} \in \mathbb{R}^{T_k \times d_v}$. Then

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right)\mathbf{V},$$

with the softmax taken over each row (over keys). The score matrix has shape (T_q, T_k) ; each row of the weight matrix sums to one; the output has shape (T_q, d_v) .

code/dl/c08_attention.py

```
1 def attention(Q, K, V, mask=None):
2     """Q: (... , Tq, dk), K: (... , Tk, dk), V: (... , Tk, dv); mask True = blocked.
   ↪ """
3     S = Q @ np.swapaxes(K, -1, -2) / np.sqrt(Q.shape[-1])    # (... , Tq, Tk)
   ↪ scores
4     if mask is not None:
5         S = np.where(mask, -np.inf, S)                          # block before the
   ↪ softmax
6     A = softmax(S, axis=-1)                                     # each row sums to 1
7     return A @ V, A
```

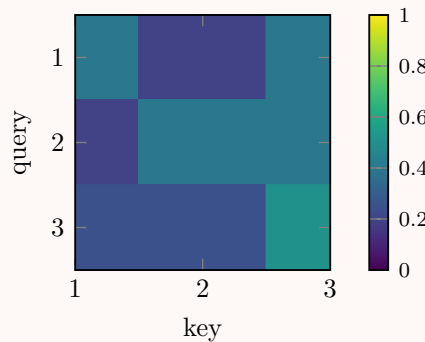
Example 8.2.1 (Attention on three tokens)

$\mathbf{Q} = \mathbf{K} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{pmatrix}$, $\mathbf{V} = \begin{pmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{pmatrix}$, $d_k = 2$.

$$\mathbf{Q}\mathbf{K}^\top = \begin{pmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \\ 1 & 1 & 2 \end{pmatrix}, \quad \frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{2}} = \begin{pmatrix} 0.707 & 0.000 & 0.707 \\ 0.000 & 0.707 & 0.707 \\ 0.707 & 0.707 & 1.414 \end{pmatrix},$$

$$\mathbf{A} = \begin{pmatrix} 0.401 & 0.198 & 0.401 \\ 0.198 & 0.401 & 0.401 \\ 0.248 & 0.248 & 0.503 \end{pmatrix}, \quad \mathbf{A}\mathbf{V} = \begin{pmatrix} 3.000 & 4.000 \\ 3.407 & 4.407 \\ 3.510 & 4.510 \end{pmatrix}.$$

Token 3 matches every key and attends most to itself; token 1 splits its attention equally between keys 1 and 3. The result equals `F.scaled_dot_product_attention`.



§8.2.ii Why divide by $\sqrt{d_k}$

Proposition 8.2.2

If the components of $\mathbf{q}$ and $\mathbf{k}$ are independent with mean 0 and variance 1, then $\mathbf{q}^\top \mathbf{k} = \sum_{i=1}^{d_k} q_i k_i$ has mean 0 and variance d_k .

Proof. Each term has mean $\mathbb{E}q_i \mathbb{E}k_i = 0$ and variance $\mathbb{E}q_i^2 \mathbb{E}k_i^2 = 1$; the d_k terms are independent, so the variances add. $\square$

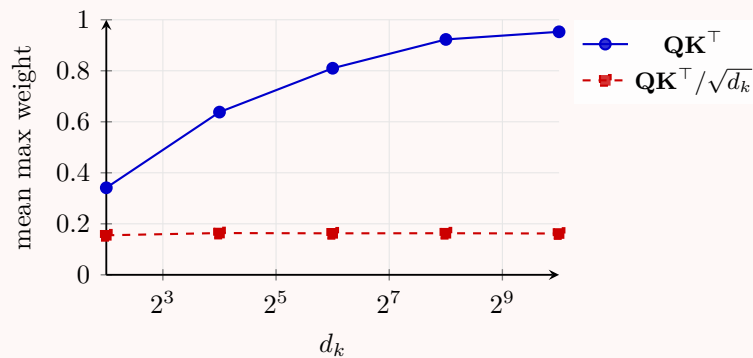
So unscaled scores have standard deviation $\sqrt{d_k}$, and for $d_k = 64$ typical score gaps are around 8: the softmax is then nearly one-hot. A saturated softmax has a Jacobian $\text{diag}(\mathbf{p}) - \mathbf{p}\mathbf{p}^\top$ close to zero, so the gradients through the attention weights vanish and learning stalls. Dividing by $\sqrt{d_k}$ restores unit variance whatever the head size.

Example 8.2.3 (Measuring it)

Random unit-variance $\mathbf{q}, \mathbf{k}$; variance of $\mathbf{q}^\top \mathbf{k}$: 4.2, 16.3, 65.8, 252.0, 1038.8 for $d_k = 4, 16, 64, 256, 1024$. Mean largest attention weight over 32 keys, without and with scaling:

d_k	4	16	64	256	1024
unscaled	0.341	0.638	0.810	0.923	0.953
scaled	0.155	0.164	0.163	0.163	0.162

At $d_k = 1024$ the unscaled weights have entropy 0.117 nats against 3.008 scaled (uniform: $\ln 32 = 3.466$). For one row of 32 scores at $d_k = 512$ the Frobenius norm of the softmax Jacobian is 1.5×10^{-9} unscaled and 0.2405 scaled.



Remark 8.2.4 — Trap. “We divide by $\sqrt{d_k}$ to normalise the attention weights” (the softmax already does that) or “to reduce computation”. The reason is the *variance* of the dot product: it keeps the softmax out of saturation so gradients flow. It is $\sqrt{d_k}$, not d_k , because standard deviation scales with the square root of the variance.

§8.2.iii Masks

A mask sets blocked scores to $-\infty$ *before* the softmax, so blocked positions get weight exactly 0 and each row still sums to one.

- **Causal (look-ahead) mask:** query i may attend only to keys $j \leq i$ (strictly upper triangle blocked). Needed in decoders so that position i cannot see the token it is trained to predict; it is what lets a whole sequence be trained in parallel with teacher forcing.
- **Padding mask:** in a batch of sequences of different lengths, padding positions are blocked as keys so no real token attends to them.

For [example 8.2.1](#), a causal mask and, alternatively, treating key 3 as padding give the weights

$$\mathbf{A}_{\text{causal}} = \begin{pmatrix} 1.000 & 0.000 & 0.000 \\ 0.330 & 0.670 & 0.000 \\ 0.248 & 0.248 & 0.503 \end{pmatrix}, \quad \mathbf{A}_{\text{pad}} = \begin{pmatrix} 0.670 & 0.330 & 0.000 \\ 0.330 & 0.670 & 0.000 \\ 0.500 & 0.500 & 0.000 \end{pmatrix},$$

matching PyTorch with `is_causal=True` and with an `attn_mask` respectively.

Remark 8.2.5 — Trap. Masking *after* the softmax (multiplying the weights by 0/1) leaves rows that no longer sum to one: here the row sums would be 0.401, 0.599, 1.000. And the causal mask is not optional at inference “because future tokens are not there yet”: during training they are there, and the model must be trained under the same constraint it will be used with.

§8.3 Multi-head attention, with shapes

Prototypical example for this section: $d_{\text{model}} = 512$, 8 heads of size 64: four 512×512 projections, 1,050,624 parameters, and attention computed in eight 64-dimensional subspaces in parallel.

One attention pattern per layer is limiting: a token may need to look at its syntactic head, its coreferent and the previous token at the same time. **Multi-head attention** runs h attentions in parallel on learned projections of size $d_k = d_{\text{model}}/h$ and concatenates the results:

$$\begin{aligned} \text{MHA}(\mathbf{X}) &= \text{Concat}(\text{head}_1, \dots, \text{head}_h) \mathbf{W}_O, \\ \text{head}_j &= \text{Attention}(\mathbf{XW}_Q^{(j)}, \mathbf{XW}_K^{(j)}, \mathbf{XW}_V^{(j)}). \end{aligned}$$

In code all heads come from one $d_{\text{model}} \times d_{\text{model}}$ matrix per role, reshaped:

step	shape
input $\mathbf{X}$	(B, T, d_{model})
$\mathbf{XW}_Q^\top, \mathbf{XW}_K^\top, \mathbf{XW}_V^\top$	(B, T, d_{model}) each
split heads: reshape, transpose	(B, h, T, d_k)
scores $\mathbf{QK}^\top / \sqrt{d_k}$ (+ mask)	(B, h, T, T)
softmax over the last axis, times $\mathbf{V}$	(B, h, T, d_k)
merge heads: transpose, reshape	(B, T, d_{model})
output projection $\mathbf{W}_O$	(B, T, d_{model})

code/dl/c08_attention.py

```
1 def mha(X, Wq, Wk, Wv, Wo, bq, bk, bv, bo, h, mask=None):
2     """X: (B, T, d); W*: (d, d) applied as X @ W.T + b (nn.Linear layout)."""
3     B, T, d = X.shape
4     dk = d // h
5     split = lambda Y: Y.reshape(B, T, h, dk).transpose(0, 2, 1, 3) # (B, h, T, dk)
6     Qh, Kh, Vh = (split(X @ W.T + b) for W, b in ((Wq, bq), (Wk, bk), (Wv, bv)))
7     Oh, A = attention(Qh, Kh, Vh, mask) # (B, h, T, dk)
8     concat = Oh.transpose(0, 2, 1, 3).reshape(B, T, d) # (B, T, d)
9     return concat @ Wo.T + bo, A
```

The script copies the weights of a PyTorch `nn.MultiheadAttention` into this function and asserts that outputs and per-head attention weights agree, with and without a causal mask. Parameter count: $4d_{\text{model}}^2 + 4d_{\text{model}}$ (four projections with biases), 288 for $d_{\text{model}} = 8$; 1,050,624, 2,362,368, 4,198,400 for $d_{\text{model}} = 512, 768, 1024$ (all asserted). The count does not depend on h : more heads means smaller heads, not more parameters.

Remark 8.3.1 — Trap. Self-attention without positional information is *permutation-equivariant*: shuffle the input tokens and the outputs are shuffled the same way (asserted in the script). “The dog bit the man” and “the man bit the dog” would produce the same set of vectors. That is why positional encodings are not optional.

§8.4 Positional encodings

Prototypical example for this section: Sinusoidal encodings for $d_{\text{model}} = 8$: position 1 is (0.8415, 0.5403, 0.0998, 0.9950, 0.0100, 1.0000, 0.0010, 1.0000).

- **Sinusoidal** (original Transformer): added to the token embeddings,

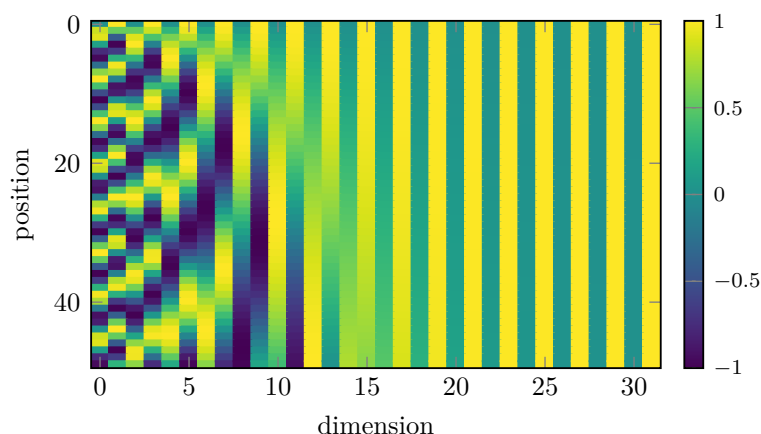
$$PE_{p,2i} = \sin(p/10000^{2i/d_{\text{model}}}), \quad PE_{p,2i+1} = \cos(p/10000^{2i/d_{\text{model}}}).$$

Each pair of dimensions is a clock at its own frequency, from fast (one cycle per 2π positions) to very slow. For every offset k there is a fixed rotation R_k with $PE_{p+k} = R_k PE_p$ for all p (asserted), so relative offsets are linearly accessible; no parameters, and defined for any length.

- **Learned absolute** embeddings (BERT, GPT-2): a trainable $T_{\text{max}} \times d_{\text{model}}$ table. Flexible, but undefined beyond T_{max} .
- **RoPE** (rotary): instead of adding anything, rotate each consecutive pair of query and key components by an angle proportional to the position. Then $\langle R_m \mathbf{q}, R_n \mathbf{k} \rangle$ depends only on $m - n$: relative position enters the score directly. Used by many recent LLMs.
- **ALiBi** [PrSmLe22]: no position embeddings at all; add a penalty $-m_h |i - j|$ to the attention scores, with a fixed slope m_h per head; for 8 heads the slopes are

$$0.5, 0.25, 0.125, 0.0625, \dots, 0.00390625.$$

Designed for extrapolation to longer sequences than seen in training.



code/dl/c08_attention.py

```

1 def rope(x, pos, base=10000.0):
2     """Rotate consecutive pairs (x0,x1), (x2,x3), ... by angle pos * theta_j."""
3     d = x.shape[-1]
4     theta = base ** (-np.arange(0, d, 2) / d)
5     c, s = np.cos(pos * theta), np.sin(pos * theta)
6     x1, x2 = x[..., 0::2], x[..., 1::2]
7     out = np.empty_like(x)
8     out[..., 0::2], out[..., 1::2] = x1 * c - x2 * s, x1 * s + x2 * c
9     return out

```

The script checks that $\langle \text{rope}(\mathbf{q}, m), \text{rope}(\mathbf{k}, n) \rangle$ is the same (4.3978) for $(m, n) = (3, 1)$, $(10, 8)$, $(52, 50)$, and that the rotation preserves norms.

§8.5 The Transformer block

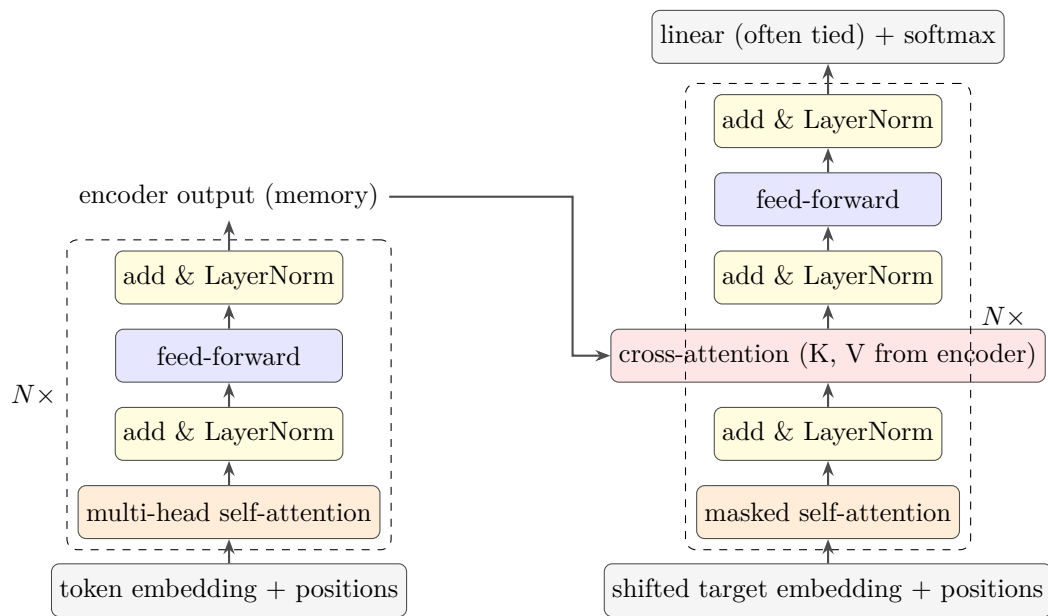
Prototypical example for this section: One encoder layer: $\mathbf{X} \leftarrow \mathbf{X} + \text{MHA}(\text{LN}(\mathbf{X}))$, then $\mathbf{X} \leftarrow \mathbf{X} + \text{FFN}(\text{LN}(\mathbf{X}))$.

§8.5.i Sublayers

- **Self-attention** mixes information *across positions*.
- **Position-wise feed-forward network:** the same two-layer MLP applied to every position independently, $\text{FFN}(\mathbf{x}) = \mathbf{W}_2 \phi(\mathbf{W}_1 \mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2$ with hidden size d_{ff} (typically $4d_{\text{model}}$) and $\phi = \text{ReLU}$ or GELU . It mixes information *across features*; most of the parameters live here.
- **Residual connections** around each sublayer (gradient highway, as in ResNet) and **LayerNorm** (chapter 5).

The original Transformer is **post-LN**: $\mathbf{X} \leftarrow \text{LN}(\mathbf{X} + \text{Sublayer}(\mathbf{X}))$. Most modern models are **pre-LN**: $\mathbf{X} \leftarrow \mathbf{X} + \text{Sublayer}(\text{LN}(\mathbf{X}))$, with a final LayerNorm at the end of the stack. Pre-LN keeps an un-normalised identity path from input to output, trains more stably at depth, and is far less sensitive to warm-up; post-LN can reach slightly better results when it trains.

§8.5.ii Encoder and decoder



(Residual arrows around each sublayer are omitted; the diagram shows the post-LN ordering of the original paper.) The decoder has three sublayers: *masked* self-attention over the target prefix, **cross-attention** whose queries come from the decoder and whose keys and values come from the encoder output, and the FFN. Cross-attention shapes: queries $(B, T_t, d_{\text{model}})$, keys and values $(B, T_s, d_{\text{model}})$, scores (B, h, T_t, T_s) , output $(B, T_t, d_{\text{model}})$. The output layer often shares its weight matrix with the input embedding (**weight tying**).

Three families use these blocks: **encoder-only** (BERT; bidirectional attention, for understanding tasks), **decoder-only** (GPT-style; causal self-attention only, for generation) and **encoder-decoder** (original Transformer, T5; for sequence-to-sequence). Chapter 9 picks this up.

§8.6 Counting parameters and FLOPs

Prototypical example for this section: One encoder layer with $d_{\text{model}} = 512$, $d_{\text{ff}} = 2048$: 1,050,624 in attention plus 2,099,712 in the FFN, 3,152,384 with the two LayerNorms.

code/dl/c08_attention.py

```
1 def enc_layer_params(d, dff):
2     attn = 4 * d * d + 4 * d           # W_Q, W_K, W_V, W_O and their biases
3     ffn = d * dff + dff + dff * d + d  # two linear layers
4     ln = 2 * (2 * d)                  # two LayerNorms (gamma, beta)
5     return attn + ffn + ln
6
7 def dec_layer_params(d, dff):
8     return enc_layer_params(d, dff) + (4 * d * d + 4 * d) + 2 * d  # + cross-attn
9                                     ↪ + 3rd LN
```

- Encoder layer: $(4d_{\text{model}}^2 + 4d_{\text{model}}) + (2d_{\text{model}}d_{\text{ff}} + d_{\text{ff}} + d_{\text{model}}) + 4d_{\text{model}}$. With $d_{\text{ff}} = 4d_{\text{model}}$ this is about $12d_{\text{model}}^2$: a third in attention, two thirds in the FFN.
- Decoder layer: an extra attention block and LayerNorm: 4,204,032 for $d_{\text{model}} = 512$.

- Embeddings: Vd_{model} (plus $T_{\text{max}}d_{\text{model}}$ for learned positions).

For $d_{\text{model}} = 512$ and 768 the script asserts both layer formulas against PyTorch’s encoder and decoder layer classes.

Example 8.6.1 (Whole models, counted)

Three configurations, counted by the script:

- `nn.Transformer(512, 8, 6, 6, 2048)` (6 encoder and 6 decoder layers, the configuration of the original “base” model): 44,140,544 parameters in the stack, asserted. With a shared 37,000-token embedding (an illustrative vocabulary size), 18.9 M more, about 63.1 M in total.
- A BERT-base-shaped encoder ($L = 12$, $d_{\text{model}} = 768$, $d_{\text{ff}} = 3072$, $V = 30,522$, 512 learned positions, 2 segment embeddings, a pooler): 7,087,872 per layer, 85.1 M in the 12 layers, 23.8 M in embeddings, 109.5 M in total.
- A GPT-2-small-shaped decoder-only model ($L = 12$, $d_{\text{model}} = 768$, $V = 50,257$, 1024 positions, tied output): 124.4 M, of which 32% is embeddings.

These are counts of the configurations as defined here, computed by the script, not quotations of anyone’s published figure; small differences from published numbers come from details such as biases and output heads. Rule of thumb for large models: $\approx 12Ld_{\text{model}}^2$ (7.1 M per layer at $d_{\text{model}} = 768$) plus embeddings.

§8.6.i Complexity

Per layer and sequence of length n : the projections and FFN cost $O(nd_{\text{model}}^2)$; the score matrix $\mathbf{QK}^\top$ and the product with $\mathbf{V}$ cost $O(n^2d_{\text{model}})$ and need $O(n^2)$ memory per head. For $d_{\text{model}} = 768$ (FLOPs, one layer, one sequence):

n	attention $4n^2d_{\text{model}}$	projections $8nd_{\text{model}}^2$	attention probabilities, 12 heads, fp16
512	805.3 M	2.4 B	6.3 MB
4,096	51.5 B	19.3 B	402.7 MB
32,768	3.3 T	154.6 B	25.8 BB

Attention overtakes the projections once $n > 2d_{\text{model}}$. The quadratic term is the motivation for sparse, linear and windowed attention variants, and for FlashAttention’s memory savings.

§8.6.ii KV cache, MQA and GQA

A decoder generates one token at a time. Without care, step t recomputes keys and values for all t previous tokens in every layer. Since those never change (causal masking), they are stored: the **KV cache** holds $\mathbf{K}$ and $\mathbf{V}$ for every layer, head and past position, and each new token computes only its own $\mathbf{q}, \mathbf{k}, \mathbf{v}$ and attends to the cache. Memory:

$$2 \times L \times n_{\text{kv heads}} \times d_{\text{head}} \times T \times \text{bytes per value per sequence}.$$

For an illustrative configuration with 32 layers, 32 heads of size 128, fp16 and 4096 tokens, that is 0.5 MiB per token and 2 GiB per sequence. **Multi-query attention** (one shared K/V head) cuts it to 0.062 GiB; **grouped-query attention** with 8 K/V heads

shared by groups of query heads to 0.5 GiB. The cache is why long-context serving is memory-bound.

Remark 8.6.2 — Trap. “The KV cache stores the attention weights” or “stores the queries”. It stores keys and values of past tokens; queries are needed only for the current token, and attention weights are recomputed each step because a new query changes them. The cache helps decoding (generation), not training, where all positions are processed in parallel anyway.

§8.6.iii FlashAttention, in concept

FlashAttention [Da+22] computes *exact* attention without ever materialising the $n \times n$ matrix in slow GPU memory: it streams blocks of keys and values through fast on-chip memory and maintains, per query, a running maximum, a running softmax denominator and a running output, rescaling them when a new block raises the maximum (an [online softmax](#)); the backward pass recomputes blocks instead of storing them. FLOPs are unchanged; memory traffic, and so wall-clock time and memory, drop sharply.

code/dl/c08_attention.py

```
1 def blockwise_attention(Q, K, V, block=4):
2     """Process keys/values in blocks, keeping a running max m, running
3     normaliser l and running output O for each query (the FlashAttention idea)."""
4     Tq, dk = Q.shape
5     m = np.full(Tq, -np.inf); l = np.zeros(Tq); O = np.zeros((Tq, V.shape[1]))
6     for s in range(0, K.shape[0], block):
7         Sb = Q @ K[s:s + block].T / np.sqrt(dk)           # scores for this
8         # block only
9         m_new = np.maximum(m, Sb.max(1))
10        scale = np.exp(m - m_new)                           # rescale old partial
11        # sums
12        P = np.exp(Sb - m_new[:, None])
13        l = l * scale + P.sum(1)
14        O = O * scale[:, None] + P @ V[s:s + block]
15        m = m_new
16    return O / l[:, None]
```

The blockwise version (keys processed four at a time) equals standard attention to machine precision.

§8.7 Transformer versus RNN and CNN

Prototypical example for this section: Relating the first and the five-hundredth token: 499 recurrent steps in an RNN, about $\log_k 500$ dilated convolution layers, one attention step.

layer type	cost per layer	sequential steps	longest path between positions
self-attention	$O(n^2 d)$	$O(1)$	$O(1)$
recurrent	$O(nd^2)$	$O(n)$	$O(n)$
convolution, kernel k	$O(knd^2)$	$O(1)$	$O(\log_k n)$ (dilated) or $O(n/k)$

For seq2seq, the Transformer wins on (1) **parallel training**: no sequential dependence across time during training, so GPUs are saturated; (2) **short paths**: any two positions interact directly, so long-range dependencies are learned easily and gradients do not pass

through hundreds of steps; (3) **content-based, dynamic weights**: attention weights depend on the input, unlike a convolution's fixed kernel; (4) **scaling**: performance keeps improving with data and parameters. It loses on cost at very long n and needs positional encodings and lots of data; RNNs remain attractive for streaming with constant memory per step.

§8.8 Capstone: an encoder–decoder Transformer, twice

Prototypical example for this section: Train on “reverse this digit sequence” (lengths 3–8, padded), reach 100.0% exact-match accuracy, and reproduce every logit in NumPy.

§8.8.i The PyTorch model

Pre-LN encoder and decoder layers built from scratch (no `nn.Transformer`), $d_{\text{model}} = 64$, $h = 4$, $d_{\text{ff}} = 128$, two encoder and two decoder layers, sinusoidal positions, embeddings scaled by $\sqrt{d_{\text{model}}}$ and shared between source, target and the output layer: 168,512 parameters. Vocabulary: PAD, BOS, EOS and ten digits.

```
code/dl/c08_capstone.py

1 class MHA(nn.Module):
2     def __init__(self, d, h):
3         super().__init__()
4         self.h, self.dk = h, d // h
5         self.q, self.k, self.v, self.o = (nn.Linear(d, d) for _ in range(4))
6
7     def forward(self, x, mem, mask):          # mask: True = blocked, (B/1, 1,
    ↪ Tq, Tk)
8         B, Tq, d = x.shape
9         Tk = mem.shape[1]
10        split = lambda y, T: y.view(B, T, self.h, self.dk).transpose(1, 2) # (B,
    ↪ h, T, dk)
11        Q, K, V = split(self.q(x), Tq), split(self.k(mem), Tk), split(self.v(mem),
    ↪ Tk)
12        S = Q @ K.transpose(-1, -2) / math.sqrt(self.dk)                    # (B, h,
    ↪ Tq, Tk)
13        A = torch.softmax(S.masked_fill(mask, float("-inf")), dim=-1)
14        self.last_attn = A.detach()
15        return self.o((A @ V).transpose(1, 2).reshape(B, Tq, d))
16
17 class FFN(nn.Module):
18     def __init__(self, d, dff):
19         super().__init__()
20         self.l1, self.l2 = nn.Linear(d, dff), nn.Linear(dff, d)
21     def forward(self, x):
22         return self.l2(torch.relu(self.l1(x)))
23
24 class EncoderLayer(nn.Module):              # pre-LN: x + f(LN(x))
25     def __init__(self, d, h, dff):
26         super().__init__()
27         self.ln1, self.att, self.ln2, self.ffn = nn.LayerNorm(d), MHA(d, h), nn.
    ↪ LayerNorm(d), FFN(d, dff)
28     def forward(self, x, src_mask):
29         y = self.ln1(x)
30         x = x + self.att(y, y, src_mask)
31         return x + self.ffn(self.ln2(x))
32
```

```

33 class DecoderLayer(nn.Module):
34     def __init__(self, d, h, dff):
35         super().__init__()
36         self.ln1, self.self_att = nn.LayerNorm(d), MHA(d, h)
37         self.ln2, self.cross_att = nn.LayerNorm(d), MHA(d, h)
38         self.ln3, self.ffn = nn.LayerNorm(d), FFN(d, dff)
39     def forward(self, y, mem, tgt_mask, src_mask):
40         z = self.ln1(y)
41         y = y + self.self_att(z, z, tgt_mask) # masked self-attention
42         y = y + self.cross_att(self.ln2(y), mem, src_mask) # queries from decoder,
        ↪ keys/values from encoder
43         return y + self.ffn(self.ln3(y))
44
45     def sinusoidal(T, d):
46         pos = torch.arange(T)[: , None].float(); i = torch.arange(d // 2)[None].float()
47         ang = pos / 10000 ** (2 * i / d)
48         pe = torch.zeros(T, d); pe[:, 0::2], pe[:, 1::2] = torch.sin(ang), torch.cos(
        ↪ ang)
49         return pe
50
51 class Transformer(nn.Module):
52     def __init__(self, V, d=64, h=4, dff=128, n_enc=2, n_dec=2, Tmax=16):
53         super().__init__()
54         self.d, self.emb = d, nn.Embedding(V, d) # shared by encoder,
        ↪ decoder, output
55         nn.init.normal_(self.emb.weight, std=d ** -0.5) # so that emb * sqrt(d)
        ↪ has unit scale
56         self.register_buffer("pe", sinusoidal(Tmax, d))
57         self.enc = nn.ModuleList(EncoderLayer(d, h, dff) for _ in range(n_enc))
58         self.dec = nn.ModuleList(DecoderLayer(d, h, dff) for _ in range(n_dec))
59         self.ln_enc, self.ln_dec = nn.LayerNorm(d), nn.LayerNorm(d)
60
61     def embed(self, tok):
62         return self.emb(tok) * math.sqrt(self.d) + self.pe[:tok.shape[1]]
63
64     def encode(self, src):
65         src_mask = (src == PAD)[: , None, None, :] # (B, 1, 1, Ts): hide
        ↪ padding keys
66         x = self.embed(src)
67         for layer in self.enc:
68             x = layer(x, src_mask)
69         return self.ln_enc(x), src_mask
70
71     def decode(self, tgt_in, mem, src_mask):
72         T = tgt_in.shape[1]
73         causal = torch.triu(torch.ones(T, T, dtype=torch.bool), 1)[None, None]
74         tgt_mask = causal | (tgt_in == PAD)[: , None, None, :]
75         y = self.embed(tgt_in)
76         for layer in self.dec:
77             y = layer(y, mem, tgt_mask, src_mask)
78         return self.ln_dec(y) @ self.emb.weight.T # tied output projection
        ↪ -> logits
79
80     def forward(self, src, tgt_in):
81         mem, src_mask = self.encode(src)
82         return self.decode(tgt_in, mem, src_mask)

```

Before training, the script copies weights into PyTorch's own modules and asserts agreement: our MHA against `nn.MultiheadAttention` (cross-attention with key padding), our encoder layer against `nn.TransformerEncoderLayer(norm_first=True)` and our

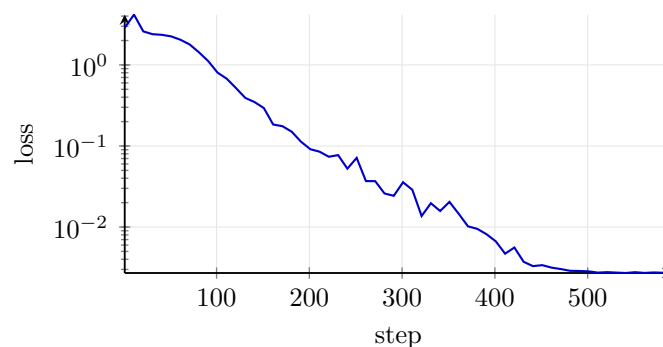
decoder layer against `nn.TransformerDecoderLayer(norm_first=True)` (causal and padding masks).

§8.8.ii Training

code/dl/c08_capstone.py

```
1 for step in range(STEPS):
2     src, tgt = make_batch(64, g)
3     logits = model(src, tgt[:, :-1])           # teacher forcing: input
        ↪ is shifted target
4     loss = F.cross_entropy(logits.reshape(-1, VOCAB), tgt[:, 1:].reshape(-1),
        ↪ ignore_index=PAD)
5     opt.zero_grad(); loss.backward()
6     nn.utils.clip_grad_norm_(model.parameters(), 1.0)
7     opt.step(); sched.step()
8     losses.append(loss.item())
```

AdamW with linear warm-up and cosine decay, gradient clipping at norm 1, batches of 64 random sequences, 600 steps (seconds on a CPU). The decoder input is the target shifted right (starting with BOS) and the loss ignores PAD positions. The loss falls from 5.711 to 0.0027 (mean of the last 50 steps).

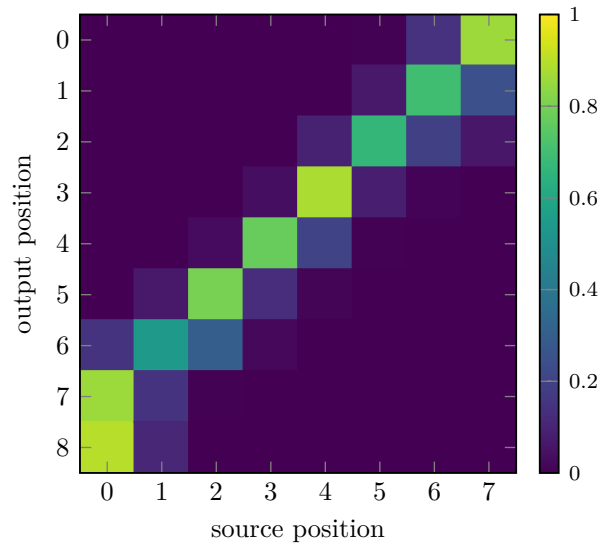


§8.8.iii Decoding and results

code/dl/c08_capstone.py

```
1 @torch.no_grad()
2 def greedy_torch(model, src):
3     mem, src_mask = model.encode(src)
4     ys = torch.full((src.shape[0], 1), BOS)
5     for _ in range(LMAX + 1):
6         nxt = model.decode(ys, mem, src_mask)[:, -1].argmax(-1, keepdim=True)
7         ys = torch.cat([ys, nxt], 1)
8     return ys
```

On 1000 fresh sequences greedy decoding gives 100.0% exact-match accuracy (100.00% of tokens). Example: input 2 1 3 7 1 1 6, output 6 1 1 7 3 1 2. The cross-attention of the last decoder layer (averaged over heads) for the input 1 9 9 0 9 7 2 7 shows what the model learned: output position t attends to input position $7 - t$ (counting from 0), the anti-diagonal (average weight there 0.756); the last row is the EOS step.



§8.8.iv The NumPy port

The same forward pass in NumPy, reading the trained `state_dict`:

code/dl/c08_capstone.py

```

1 def np_layernorm(x, w, b, eps=1e-5):
2     mu = x.mean(-1, keepdims=True); var = x.var(-1, keepdims=True)
3     return (x - mu) / np.sqrt(var + eps) * w + b
4
5 def np_softmax(s):
6     s = s - s.max(-1, keepdims=True); e = np.exp(s)
7     return e / e.sum(-1, keepdims=True)
8
9 def np_mha(P, pre, x, mem, mask, h):
10    B, Tq, d = x.shape; Tk = mem.shape[1]; dk = d // h
11    lin = lambda z, n: z @ P[f"{pre}.{n}.weight"].T + P[f"{pre}.{n}.bias"]
12    split = lambda z, T: z.reshape(B, T, h, dk).transpose(0, 2, 1, 3)
13    Q, K, V = split(lin(x, "q"), Tq), split(lin(mem, "k"), Tk), split(lin(mem, "v")
14    ↪ , Tk)
15    S = np.where(mask, -np.inf, Q @ K.transpose(0, 1, 3, 2) / np.sqrt(dk))
16    O = (np_softmax(S) @ V).transpose(0, 2, 1, 3).reshape(B, Tq, d)
17    return lin(O, "o")
18
19 def np_ffn(P, pre, x):
20    hdn = np.maximum(0, x @ P[f"{pre}.l1.weight"].T + P[f"{pre}.l1.bias"])
21    return hdn @ P[f"{pre}.l2.weight"].T + P[f"{pre}.l2.bias"]
22
23 def np_forward(P, src, tgt_in, h=4, n_enc=2, n_dec=2):
24    d = P["emb.weight"].shape[1]
25    ln = lambda z, n: np_layernorm(z, P[f"{n}.weight"], P[f"{n}.bias"])
26    embed = lambda t: P["emb.weight"][t] * np.sqrt(d) + P["pe"][:,t.shape[1]]
27    src_mask = (src == PAD)[: , None, None, :]
28    x = embed(src)
29    for l in range(n_enc):
30        y = ln(x, f"enc.{l}.ln1"); x = x + np_mha(P, f"enc.{l}.att", y, y, src_mask
31        ↪ , h)
32        x = x + np_ffn(P, f"enc.{l}.ffn", ln(x, f"enc.{l}.ln2"))
33    mem = ln(x, "ln_enc")
34    T = tgt_in.shape[1]

```

```

33     tgt_mask = np.triu(np.ones((T, T), bool), 1)[None, None] | (tgt_in == PAD)[:],
    ↪ None, None, :]
34     y = embed(tgt_in)
35     for l in range(n_dec):
36         z = ln(y, f"dec.{l}.ln1"); y = y + np_mha(P, f"dec.{l}.self_att", z, z,
    ↪ tgt_mask, h)
37         y = y + np_mha(P, f"dec.{l}.cross_att", ln(y, f"dec.{l}.ln2"), mem,
    ↪ src_mask, h)
38         y = y + np_ffn(P, f"dec.{l}.ffn", ln(y, f"dec.{l}.ln3"))
39     return ln(y, "ln_dec") @ P["emb.weight"].T

```

In float64 the NumPy logits equal PyTorch's on 50 test sequences (to 10^{-9}), and greedy decoding in NumPy produces exactly the same 200 output sequences as PyTorch (100.0% correct). Writing the NumPy version is the best interview preparation in this book: every reshape, transpose and mask has to be right for the assertion to pass.

Attention is a softmax-weighted average with learned queries, keys and values, scaled by $\sqrt{d_k}$ to keep the softmax soft. A Transformer layer is attention (mix positions) plus an MLP (mix features), each wrapped in a residual connection and LayerNorm; positions must be injected; cost is quadratic in length.

§8.9 A few problems to think about

Problem 8A. Two queries $\mathbf{q}_1 = (1, 0)$, $\mathbf{q}_2 = (0, 2)$, keys $\mathbf{k}_1 = (1, 1)$, $\mathbf{k}_2 = (0, 1)$, values $v_1 = 10$, $v_2 = 20$, $d_k = 2$. Compute both attention outputs.

[hint · solution](#)

Problem 8B. Give the parameter count of one multi-head self-attention block with $d_{\text{model}} = 256$ and $h = 4$, and of the FFN with $d_{\text{ff}} = 1024$, biases included.

[hint · solution](#)

Problem 8C. Scores for one query over four keys are $(8, 4, 0, -4)$ with $d_k = 64$ but someone forgot the scaling. Compare the softmax weights with and without dividing by $\sqrt{d_k}$.

[hint · solution](#)

Problem 8D. What are the shapes of $\mathbf{Q}$, $\mathbf{K}$, $\mathbf{V}$, the score tensor and the output in the cross-attention of a decoder with batch $B = 2$, target length 5, source length 9, $d_{\text{model}} = 32$, $h = 4$?

[hint · solution](#)

Problem 8E. Why does a decoder-only language model need a causal mask during training but an encoder like BERT does not?

[hint · solution](#)

Problem 8F. Estimate the KV-cache size for a model with 24 layers, 16 heads of size 64, fp16, a batch of 8 sequences of 2048 tokens. How does grouped-query attention with 4 K/V heads change it?

[hint · solution](#)

III

Modern Deep Learning

9 Language models and LLMs

A language model assigns probabilities to sequences of tokens; a large language model is a Transformer trained to do that on a very large corpus and then adapted to follow instructions. Placement OAs mostly ask what fine-tuning means; interviews at AI companies ask how tokenisation works, how decoding knobs change outputs, what LoRA saves, how to serve a model in less memory, and how retrieval reduces hallucination. This chapter covers each at the depth of one good interview answer, with the arithmetic done by code.

§9.1 Tokens and embeddings

Prototypical example for this section: Byte-pair encoding learns that $e+p$ is the most frequent pair in a small corpus and merges it first.

Models see integer **tokens**, not characters or words. Word-level vocabularies cannot handle unseen words; character-level sequences are long. **Subword** tokenisers sit in between: frequent words become one token, rare words split into pieces. **Byte-pair encoding** (BPE) [SeHaBi16] starts from characters (with an end-of-word marker) and repeatedly merges the most frequent adjacent pair:

```
code/dl/c09_llm.py

1 def bpe_train(word_freq, n_merges):
2     words = {tuple(w) + ("</w>"), f for w, f in word_freq.items()} # start from
    ↪ characters
3     merges = []
4     for _ in range(n_merges):
5         pairs = Counter()
6         for sym, f in words.items():
7             for a, b in zip(sym, sym[1:]):
8                 pairs[a, b] += f # count adjacent pairs,
    ↪ weighted by frequency
9         if not pairs:
10             break
11         best = max(pairs, key=lambda p: (pairs[p], p)) # most frequent pair (ties:
    ↪ lexicographic)
12         merges.append((best, pairs[best]))
13         new = {}
14         for sym, f in words.items(): # replace the pair
    ↪ everywhere
15             out_, i = [], 0
16             while i < len(sym):
17                 if i < len(sym) - 1 and (sym[i], sym[i + 1]) == best:
18                     out_.append(sym[i] + sym[i + 1]); i += 2
19                 else:
20                     out_.append(sym[i]); i += 1
21             new[tuple(out_)] = f
22         words = new
23     return merges, words
24
25 def bpe_encode(word, merges):
26     sym = list(word) + ["</w>"]
27     for (a, b), _ in merges: # apply merges in learned
```

```

    ↪ order
28     i = 0
29     while i < len(sym) - 1:
30         if sym[i] == a and sym[i + 1] == b:
31             sym[i:i + 2] = [a + b]
32         else:
33             i += 1
34     return sym

```

Example 9.1.1 (BPE on a toy corpus)

Word counts: deep 6, deeper 3, deepest 2, keep 4, keeper 2, sleep 3. The first eight merges (pair count in brackets, _ marks end of word) are ep (20), eep (20), eep_ (13), eepe (7), deep_ (6), r_ (5), eeper_ (5), keep_ (4). The corpus shrinks from 119 symbols to 39. Unseen words are still encodable: sleeper → s l eeper_, keepest → k eepe s t _. The script checks that decoding inverts encoding and that encoding each training word reproduces its training segmentation.

Production tokenisers work on bytes (so any string is encodable), learn tens of thousands to a few hundred thousand merges, and (WordPiece, unigram LM) differ in how they choose merges. Consequences worth saying in an interview: costs and context limits are counted in tokens; numbers and rare scripts fragment into many tokens; spelling-level tasks are hard because the model never sees letters.

An **embedding** layer is a lookup table $\mathbf{E} \in \mathbb{R}^{V \times d}$: token i becomes row i . It is equivalent to multiplying a one-hot vector by $\mathbf{E}$, and it is trained like any other weight. Similar tokens end up with similar vectors because they appear in similar contexts (chapter 10 derives word2vec).

§9.2 Objectives and model families

Prototypical example for this section: GPT predicts the next token; BERT fills in masked tokens; T5 maps an input text to an output text.

- **Causal (autoregressive) LM**: $p(x_1, \dots, x_T) = \prod_t p(x_t \mid x_{<t})$, trained with cross-entropy on every position at once under a causal mask. Decoder-only (GPT family). Natural for generation.
- **Masked LM** (BERT [De+19]): choose 15% of positions; of those, replace 80% by a [MASK] token, 10% by a random token, keep 10%; predict the originals using context on both sides. Simulated on 100,000 tokens: 15.2% selected, 12.2% masked, 1.5% random, 1.5% kept. The random and kept cases stop the model from learning that only [MASK] positions matter. Encoder-only; strong representations for classification and retrieval; not a generator.
- **Sequence-to-sequence** (T5, BART): an encoder reads the input, a decoder generates the output; pretrained by reconstructing corrupted spans.

Definition 9.2.1. The **perplexity** of a model on a sequence of T tokens is

$$\text{PPL} = \exp\left(-\frac{1}{T} \sum_{t=1}^T \log p(x_t \mid x_{<t})\right),$$

the exponential of the per-token cross-entropy: the effective number of equally likely choices per token.

For per-token probabilities 0.5, 0.25, 0.1, 0.4, 0.8 the mean NLL is 1.1043 nats and $\text{PPL} = 3.017$ (the inverse geometric mean of the probabilities, as checked). A uniform model over V tokens has $\text{PPL} = V$ (checked for $V = 50000$). Perplexities are only comparable with the same tokeniser and test text.

§9.3 Scale

Prototypical example for this section: Training a 1B-parameter model on 20B tokens costs about $6ND = 1.2 \times 10^{20}$ FLOPs.

Training one token through a dense Transformer costs about $2N$ FLOPs forward and $4N$ backward, so total training compute is $C \approx 6ND$ for N parameters and D tokens. **Scaling laws** are empirical power laws: in the regimes studied, test loss falls smoothly as a power of N , D and C when the other two are not bottlenecks [Ka+20]. A later analysis concluded that compute-optimal training scales N and D in roughly equal proportion, which in their fits came to roughly twenty tokens per parameter [HoBoMe+22]; many later models train on far more tokens than that because inference cost favours smaller models. At a sustained 40% of 100 TFLOP/s, the 1B/20B example above is about 35 GPU-days. Treat all such exponents and constants as properties of particular fits, not laws of nature.

§9.4 Adapting a pretrained model

Prototypical example for this section: LoRA with rank 8 on one 4096×4096 weight trains 65,536 parameters instead of 16,777,216 (0.39%).

§9.4.i Fine-tuning, from cheapest to most expensive

- **Prompting** / in-context learning: no weights change; instructions and examples go in the input.
- **Feature extraction** (linear probing): freeze the model, train a small head on its embeddings.
- **Prompt / prefix tuning**: learn a few virtual-token embeddings (or per-layer key/value prefixes) prepended to the input; the model stays frozen.
- **Adapters**: small bottleneck MLPs inserted in each layer and trained, model frozen.
- **LoRA** [Hu+22]: below.
- **Full fine-tuning**: update every weight on the task data, with a small learning rate. Best quality per example when data are plentiful, but one full copy of the model per task, and optimiser state (chapter 11) for every parameter.

Instruction tuning is supervised fine-tuning on (instruction, good response) pairs, turning a next-token predictor into an assistant. **RLHF** [OuWuJi+22] then collects human preferences between pairs of responses, trains a reward model on them, and optimises the policy against that reward with reinforcement learning (typically PPO) plus a KL penalty that keeps it close to the supervised model. **DPO** [Ra+23] reaches the same

kind of objective without a separate reward model or RL loop: a classification-style loss directly on preference pairs that raises the log-probability ratio (relative to a frozen reference model) of preferred over rejected responses.

§9.4.ii LoRA

Freeze the pretrained $\mathbf{W}_0 \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ and learn a low-rank update:

$$\mathbf{W} = \mathbf{W}_0 + \frac{\alpha}{r} \mathbf{B} \mathbf{A}, \quad \mathbf{B} \in \mathbb{R}^{d_{\text{out}} \times r}, \quad \mathbf{A} \in \mathbb{R}^{r \times d_{\text{in}}}, \quad r \ll d.$$

$\mathbf{B}$ starts at zero so training starts exactly from the pretrained model; after training $\mathbf{B} \mathbf{A}$ can be merged into $\mathbf{W}_0$, so inference costs nothing extra.

```
code/dl/c09_llm.py

1 class LoRALinear(nn.Module):
2     def __init__(self, base: nn.Linear, r=8, alpha=16):
3         super().__init__()
4         self.base = base
5         for p in self.base.parameters():
6             p.requires_grad_(False)                # frozen pretrained
7             ↪ weight
8             self.A = nn.Parameter(torch.randn(r, base.in_features) * 0.01)
9             self.B = nn.Parameter(torch.zeros(base.out_features, r))    # zero: starts
10            ↪ as a no-op
11            self.scale = alpha / r
12
13    def forward(self, x):
14        return self.base(x) + (x @ self.A.T @ self.B.T) * self.scale
```

The script checks that the layer equals the base layer at initialisation, that merging gives the same output, and that the trainable count is $r(d_{\text{in}} + d_{\text{out}})$.

Example 9.4.1 (What LoRA saves)

For a 4096×4096 matrix: rank 4, 8, 16, 64 train 32,768, 65,536, 131,072, 524,288 parameters, i.e. 0.20%, 0.39%, 0.78%, 3.12% of the full matrix. For an illustrative 32-layer model with $d_{\text{model}} = 4096$ ($\approx 12Ld_{\text{model}}^2 = 6.4$ B weights in its blocks), rank-8 LoRA on $\mathbf{W}_Q$ and $\mathbf{W}_V$ only trains 4.2 M (0.065%). The saving is in gradients and optimiser state (and in storing one small adapter per task); the frozen weights still have to fit in memory for the forward pass.

Remark 9.4.2 — Trap. “LoRA reduces inference latency” or “LoRA makes the model smaller”. It reduces the number of *trainable* parameters and the training memory; after merging, the model has exactly the original size and speed. Combined with a quantised frozen base (QLoRA-style) it also cuts the memory of the frozen weights.

§9.5 Decoding

Prototypical example for this section: Logits (3, 2.5, 1, 0.5, 0, −1): greedy always says token 1; top- p with $p = 0.9$ samples among 3 tokens.

Given next-token logits, **greedy** decoding takes the arg-max, **beam search** keeps k hypotheses (chapter 7), and sampling methods reshape the distribution before drawing:

code/dl/c09_llm.py

```

1 def temperature(z, T):
2     return softmax(z / T)
3
4 def top_k(z, k):
5     keep = np.argsort(-z)[:k]
6     zz = np.full_like(z, -np.inf); zz[keep] = z[keep]
7     return softmax(zz)
8
9 def top_p(z, p):
10     probs = softmax(z)
11     order = np.argsort(-probs)
12     cum = np.cumsum(probs[order])
13     n = int(np.searchsorted(cum, p) + 1)
14     zz = np.full_like(z, -np.inf); zz[order[:n]] = z[order[:n]]
15     return softmax(zz)

```

Example 9.5.1 (Four decoding distributions)

For $z = (3, 2.5, 1, 0.5, 0, -1)$:

rule	distribution over the six tokens
softmax ($T = 1$)	(0.529, 0.321, 0.072, 0.043, 0.026, 0.010)
$T = 0.5$	(0.716, 0.264, 0.013, 0.005, 0.002, 0.000)
$T = 2$	(0.358, 0.279, 0.132, 0.103, 0.080, 0.048)
top- k , $k = 2$	(0.622, 0.378, 0.000, 0.000, 0.000, 0.000)
top- p , $p = 0.9$	(0.574, 0.348, 0.078, 0.000, 0.000, 0.000)

Cumulative sorted probabilities are (0.529, 0.849, 0.921, 0.964, 0.990, 1.000), so the nucleus for $p = 0.9$ is the first three tokens, renormalised (checked against a brute-force implementation).

Temperature $\rightarrow 0$ recovers greedy; top- k uses a fixed number of candidates even when the distribution is flat or peaked, which top- p adapts to. Greedy and beam search tend to produce repetitive, generic text in open-ended generation; sampling with moderate temperature and top- p is the usual default there, while greedy is preferred for tasks with one right answer (code, extraction).

§9.6 Evaluation and its limits

Prototypical example for this section: “the cat is on the mat” against the reference “the cat sat on the mat”: BLEU-2 0.707.

BLEU (translation) multiplies a brevity penalty by the geometric mean of clipped n -gram precisions of the output against references; **ROUGE** (summarisation) emphasises recall, e.g. ROUGE-L uses the longest common subsequence. In the example, unigram precision 0.833, bigram precision 0.600, brevity penalty 1.0, BLEU-2 0.707; the LCS has length 5, so ROUGE-L recall and precision are both 0.833. The limits: a correct paraphrase (“a cat was sitting on the mat”) gets bigram precision only 0.333; one wrong word can flip meaning at a small score cost; and none of these measures factuality. Perplexity measures fit to text, not usefulness. LLMs are therefore evaluated on task benchmarks, by human preference, and increasingly by other models as judges, each with

its own biases (contamination of benchmarks by training data being the main one).

§9.7 Making models cheaper: quantisation, distillation, MoE

Prototypical example for this section: A 7B-parameter model needs 14.0 GB for its weights in fp16 and 3.5 GB in 4-bit.

§9.7.i Quantisation

Weights alone: 28.0 GB in fp32, 14.0 in fp16/bf16, 7.0 in int8, 3.5 in 4-bit, for 7×10^9 parameters. Symmetric per-tensor int8:

```
code/dl/c09_llm.py
1 def quantize_int8(w):
2     scale = np.abs(w).max() / 127                # symmetric, per tensor
3     q = np.clip(np.round(w / scale), -127, 127).astype(np.int8)
4     return q, scale
```

For a 64×64 Gaussian matrix (std 0.05) with one outlier of 0.4 the scale is 3.15×10^{-3} and the worst error 1.57×10^{-3} (codes identical to `torch.quantize_per_tensor`); without the outlier the worst error is 6.59×10^{-4} , and the mean error with it is $2.4 \times$ larger. One outlier stretches the grid for everyone, which is why practical schemes use per-channel or per-group scales and treat outliers separately. **Post-training quantisation** needs no retraining; **quantisation-aware training** simulates rounding during training.

§9.7.ii Distillation

A small **student** is trained to match a large **teacher**'s softened output distribution: $\mathcal{L} = T^2 \text{KL}(\text{softmax}(z_t/T) \parallel \text{softmax}(z_s/T))$, usually mixed with the ordinary label loss. Temperature reveals the teacher's "dark knowledge": for teacher logits (4, 1.5, 1, -1) the $T = 1$ distribution is (0.878, 0.072, 0.044, 0.006) but at $T = 2$ it is (0.628, 0.180, 0.140, 0.052), exposing which wrong classes are nearly right. With student logits (3, 2, 0, -0.5) the loss is 0.2255 (equal to `F.kl_div`); the T^2 keeps gradient magnitudes comparable across temperatures.

§9.7.iii Mixture of experts

An MoE layer replaces the FFN by E expert FFNs and a router that sends each token to the top k . For an illustrative layer with $d_{\text{model}} = 4096$, a gated FFN of width 14,336 (176.2 M parameters) and $E = 8$, $k = 2$: 1.4 B parameters stored, 352.3 M (25%) active per token. Capacity grows with total parameters while compute per token follows active parameters; the costs are memory for all experts, routing load balance (auxiliary losses), and communication when experts live on different devices.

Remark 9.7.1 — Trap. "An MoE model with X total parameters runs as fast as a dense model with X parameters" (it runs closer to the speed of its *active* size, if memory bandwidth allows) and "quantisation reduces FLOPs" (it mainly reduces memory and memory traffic; speed-ups need kernels for the low-precision format).

§9.8 Retrieval, tools and failure modes

Prototypical example for this section: A question about the KV cache retrieves the document about the KV cache, which is pasted into the prompt before the model answers.

Retrieval-augmented generation (RAG): embed documents into vectors offline; at query time embed the question, retrieve the top- k most similar chunks (cosine similarity, approximate nearest-neighbour index), and put them in the prompt with an instruction to answer from them and cite them. In a four-document toy with TF-IDF vectors the query “what does the KV cache store during decoding?” retrieves document 2 (cosine 0.214, matching scikit-learn); real systems use learned dense embeddings, chunking, re-ranking and hybrid keyword search. RAG gives the model fresh and private knowledge without retraining and makes answers checkable. **Tool use** and **agents** extend the idea: the model emits structured calls (search, code execution, APIs), the system runs them and feeds results back, looping until done.

Hallucination: fluent, confident, wrong output, a consequence of training to produce likely text rather than true text. Mitigations: grounding with retrieval and citations, asking the model to abstain when the context lacks the answer, lower temperature for factual tasks, verification steps and tools, evaluation sets that measure it. Safety basics an interviewer expects: prompt injection (instructions hidden in retrieved or user content), data leakage of private information, harmful-content filtering, bias, and the principle that a model’s output is untrusted input to the rest of the system.

Pretrain to predict tokens, adapt cheaply (LoRA, instruction tuning, preference optimisation), decode with the right knob, serve with fewer bits, and ground answers in retrieved evidence.

§9.9 A few problems to think about

Problem 9A. A language model assigns probabilities 0.2, 0.5 and 0.1 to the three tokens of a test sentence. What is its perplexity on the sentence?

hint · solution

Problem 9B. How many trainable parameters does LoRA with $r = 16$ add to a 1024×4096 projection, and what fraction of the matrix is that?

hint · solution

Problem 9C. Logits $(2, 1, 0)$. Give the sampling distribution for temperature 0.5, for top- k with $k = 1$, and for top- p with $p = 0.5$. Which of them are deterministic?

hint · solution

Problem 9D. A team fine-tunes a 3B-parameter model by full fine-tuning in fp16 with Adam, and runs out of memory. Give two changes from this chapter and what each saves.

hint · solution

Problem 9E. Explain why BERT cannot simply be used for left-to-right text generation, and why GPT is a weaker sentence encoder for retrieval than a bidirectional encoder of the same size.

hint · solution

10 Generative and representation learning

Discriminative models learn $p(y \mid \mathbf{x})$. Generative models learn $p(\mathbf{x})$ well enough to sample from it; representation learning learns features without labels. Interviews ask for the VAE objective and the reparameterisation trick, the GAN minimax game and its failure modes, the idea of diffusion, and how contrastive learning, word2vec and Vision Transformers produce representations. Each gets one section, with the key identity verified numerically.

§10.1 Autoencoders

Prototypical example for this section: A linear autoencoder with a 3-unit bottleneck, trained by gradient descent, reaches exactly PCA's reconstruction error (0.00705 vs 0.00705).

An **autoencoder** maps $\mathbf{x}$ to a low-dimensional code $\mathbf{z} = f(\mathbf{x})$ and back, $\hat{\mathbf{x}} = g(\mathbf{z})$, trained to minimise $\|\mathbf{x} - \hat{\mathbf{x}}\|^2$. The bottleneck forces it to keep the directions that matter. With linear f, g and squared error the optimum spans the top principal subspace: the script trains one on 10-dimensional data and checks that its error equals PCA's and that the decoder's column space coincides with the top three principal components (see the PCA chapter of *Classical Machine Learning*). Non-linear autoencoders generalise PCA to curved manifolds. Variants: **denoising** (reconstruct $\mathbf{x}$ from a corrupted input, learning robust features), **sparse** (penalise code activity). A plain autoencoder is not a good generator: nothing makes the code space dense, so decoding a random $\mathbf{z}$ usually gives garbage. The VAE fixes this.

§10.2 Variational autoencoders

Prototypical example for this section: For an encoder output $\mu = 1$, $\sigma = 0.5$, the KL term of the VAE loss is 0.8181 nats.

§10.2.i The ELBO

A VAE [KiWe14] posits latent $\mathbf{z} \sim p(\mathbf{z}) = \mathcal{N}(\mathbf{0}, \mathbf{I})$ and a decoder $p_\theta(\mathbf{x} \mid \mathbf{z})$. The likelihood $p_\theta(\mathbf{x}) = \int p_\theta(\mathbf{x} \mid \mathbf{z})p(\mathbf{z}) \, d\mathbf{z}$ is intractable, so an encoder $q_\phi(\mathbf{z} \mid \mathbf{x}) = \mathcal{N}(\boldsymbol{\mu}_\phi(\mathbf{x}), \text{diag } \boldsymbol{\sigma}_\phi^2(\mathbf{x}))$ approximates the posterior.

Theorem 10.2.1 (Evidence lower bound)

For any q ,

$$\log p_\theta(\mathbf{x}) \geq \mathbb{E}_{q_\phi(\mathbf{z} \mid \mathbf{x})} [\log p_\theta(\mathbf{x} \mid \mathbf{z})] - \text{KL}(q_\phi(\mathbf{z} \mid \mathbf{x}) \parallel p(\mathbf{z})),$$

with equality iff $q_\phi(\mathbf{z} \mid \mathbf{x}) = p_\theta(\mathbf{z} \mid \mathbf{x})$.

Proof. $\log p_\theta(\mathbf{x}) = \mathbb{E}_q[\log \frac{p_\theta(\mathbf{x}, \mathbf{z})}{q(\mathbf{z}|\mathbf{x})}] + \text{KL}(q(\mathbf{z}|\mathbf{x}) \| p_\theta(\mathbf{z}|\mathbf{x}))$, which follows from $p_\theta(\mathbf{x}, \mathbf{z}) = p_\theta(\mathbf{z}|\mathbf{x})p_\theta(\mathbf{x})$. The KL is non-negative, and the first term splits into $\mathbb{E}_q \log p_\theta(\mathbf{x}|\mathbf{z}) - \text{KL}(q \| p(\mathbf{z}))$. $\square$

The VAE maximises the ELBO: a **reconstruction** term (with a Gaussian decoder, a squared error; with Bernoulli pixels, a BCE) plus a **regulariser** pulling each $q(\mathbf{z}|\mathbf{x})$ toward the prior, which makes the latent space smooth and samplable. For diagonal Gaussians the KL has a closed form per dimension:

$$\text{KL}(\mathcal{N}(\mu, \sigma^2) \| \mathcal{N}(0, 1)) = \frac{1}{2}(\mu^2 + \sigma^2 - 1 - \log \sigma^2).$$

code/dl/c10_generative.py

```
1 def kl_gauss_std(mu, logvar):          # KL( N(mu, sigma^2) || N(0, 1) ), per
    ↪ dimension
2     return 0.5 * (mu ** 2 + np.exp(logvar) - 1 - logvar)
```

With $\mu = 1$, $\sigma = 0.5$ it is 0.8181, and 0.0 at $\mu = 0$, $\sigma = 1$ (both checked against `torch.distributions`). Networks output $\log \sigma^2$ rather than σ so that the variance is positive without constraints.

§10.2.ii The reparameterisation trick

The reconstruction term is an expectation over $\mathbf{z} \sim q_\phi$, and we need its gradient with respect to ϕ ; sampling is not differentiable. Write $\mathbf{z} = \boldsymbol{\mu} + \boldsymbol{\sigma} \odot \boldsymbol{\varepsilon}$ with $\boldsymbol{\varepsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$: the randomness moves into an input, $\mathbf{z}$ becomes a deterministic, differentiable function of $(\boldsymbol{\mu}, \boldsymbol{\sigma})$, and ordinary backpropagation gives low-variance gradient estimates. Check: for $f(z) = z^2$, $\mathbb{E}f = \mu^2 + \sigma^2$ has gradients $(2\mu, 2\sigma) = (2, 1)$; a 200,000-sample reparameterised estimate gives (1.999, 0.998).

Remark 10.2.2 — Trap. “The VAE’s KL term is what makes reconstructions sharp.” It does the opposite: it trades reconstruction quality for a well-behaved latent space, and Gaussian decoders average over plausible outputs, so VAE samples tend to be blurry. If the KL term wins completely the decoder ignores $\mathbf{z}$ (**posterior collapse**); weighting it (β -VAE) trades the two.

§10.3 Generative adversarial networks

Prototypical example for this section: Data $\mathcal{N}(0, 1)$, generator $\mathcal{N}(1.5, 1)$: the best discriminator outputs 0.755 at $x = 0$ and 0.245 at $x = 1.5$.

A GAN [Go+14] trains a generator $G(\mathbf{z})$ against a discriminator $D(\mathbf{x}) \in (0, 1)$:

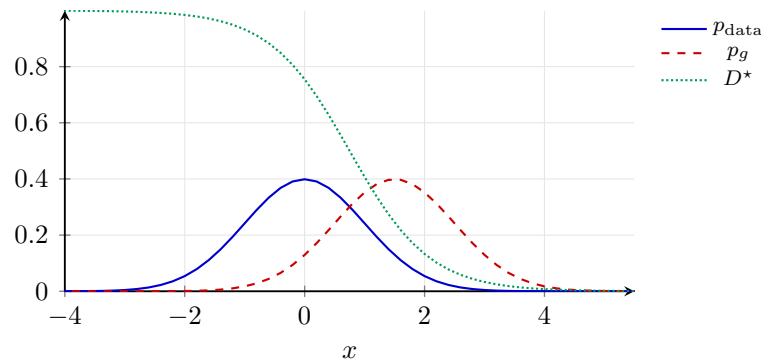
$$\min_G \max_D V(D, G) = \mathbb{E}_{\mathbf{x} \sim p_{\text{data}}} \log D(\mathbf{x}) + \mathbb{E}_{\mathbf{z} \sim p(\mathbf{z})} \log(1 - D(G(\mathbf{z}))).$$

Proposition 10.3.1

For fixed G with sample density p_g , the optimal discriminator is $D^*(\mathbf{x}) = p_{\text{data}}(\mathbf{x}) / (p_{\text{data}}(\mathbf{x}) + p_g(\mathbf{x}))$, and then $V(D^*, G) = -\log 4 + 2 \text{JSD}(p_{\text{data}} \| p_g)$, minimised exactly when $p_g = p_{\text{data}}$.

Proof. Pointwise, $a \log D + b \log(1 - D)$ is maximised at $D = a/(a + b)$. Substituting and adding and subtracting $\log 4$ gives $\text{KL}(p_{\text{data}} \| m) + \text{KL}(p_g \| m) - \log 4$ with $m = \frac{1}{2}(p_{\text{data}} + p_g)$, i.e. $2 \text{JSD} - \log 4$. $\square$

For the prototype the integral gives $V = -0.9440$ and $\text{JSD} = 0.2212$, so $V = -1.3863 + 2 \text{JSD}$ as claimed (JSD from SciPy).



In practice G is trained to maximise $\log D(G(z))$ (the **non-saturating** loss): early on D rejects fakes easily and $\log(1 - D(G(z)))$ has almost no gradient. Failure modes: **mode collapse** (the generator covers a few modes that fool D), oscillation instead of convergence, vanishing generator gradients when D is too good. **WGAN** [ArChBo17] replaces the JS-based objective with an estimate of the Wasserstein distance, using a Lipschitz-constrained critic (weight clipping, later a gradient penalty); its loss is smoother when the two distributions barely overlap and correlates with sample quality.

§10.4 Diffusion models

Prototypical example for this section: With a linear noise schedule over $T = 1000$ steps, after 300 steps only $\sqrt{\bar{\alpha}} = \sqrt{0.3964}$ of the original signal remains.

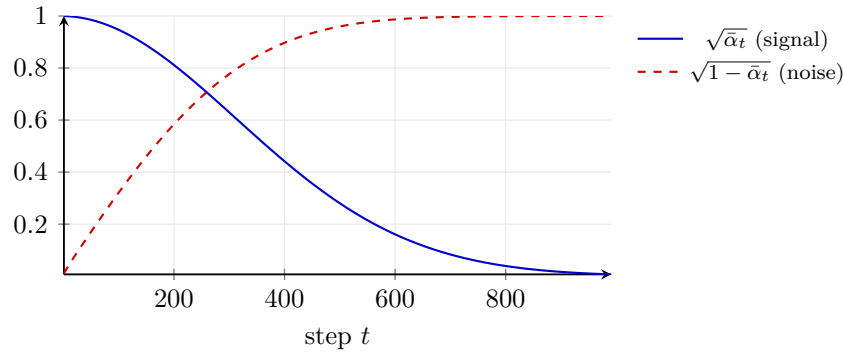
A **diffusion model** (DDPM [HoJaAb20]) defines a fixed *forward* process that adds Gaussian noise in T small steps, $q(\mathbf{x}_t | \mathbf{x}_{t-1}) = \mathcal{N}(\sqrt{1 - \beta_t} \mathbf{x}_{t-1}, \beta_t \mathbf{I})$, and learns to reverse it. Composing Gaussians gives a closed form for any t :

$$q(\mathbf{x}_t | \mathbf{x}_0) = \mathcal{N}(\sqrt{\bar{\alpha}_t} \mathbf{x}_0, (1 - \bar{\alpha}_t) \mathbf{I}), \quad \bar{\alpha}_t = \prod_{s \leq t} (1 - \beta_s).$$

code/dl/c10_generative.py

```
1 def q_sample(x0, t, eps): # closed form of t noising steps at once
2     return np.sqrt(abar[t]) * x0 + np.sqrt(1 - abar[t]) * eps
```

The script runs 300 individual noising steps on 100,000 samples and confirms the mean $\sqrt{\bar{\alpha}_t} x_0$ and variance $1 - \bar{\alpha}_t$ of the closed form. With β from 10^{-4} to 0.02, $\bar{\alpha}$ drops to one half at step 259 and to 0.000040 at T : $\mathbf{x}_T$ is essentially pure noise.



Training (derivation sketch): the ELBO of the reverse chain decomposes into per-step KL terms between Gaussians; with the reverse mean parameterised through a noise predictor ε_θ , each term reduces (up to weights that are usually dropped) to

$$\mathcal{L} = \mathbb{E}_{t, x_0, \varepsilon} \|\varepsilon - \varepsilon_\theta(\sqrt{\alpha_t}x_0 + \sqrt{1 - \alpha_t}\varepsilon, t)\|^2 :$$

pick a random step, noise a clean example in one shot, and regress the noise. Sampling starts from $x_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ and applies the learned denoising step T times (faster samplers skip steps). Compared with GANs: stable training and good mode coverage, at the price of many network evaluations per sample. Text-to-image systems condition ε_θ on a text embedding and often run diffusion in an autoencoder’s latent space.

Normalising flows, in one paragraph: compose invertible maps with tractable Jacobian determinants so that $\log p(\mathbf{x}) = \log p(\mathbf{z}) + \log |\det \partial \mathbf{z} / \partial \mathbf{x}|$ is exact; training is maximum likelihood. Exact density and exact inversion, but architectures are constrained to be invertible.

§10.5 Representation learning without labels

Prototypical example for this section: Four images and their four captions: the CLIP loss is a 4-way cross-entropy in each direction with the matching pair on the diagonal.

§10.5.i Contrastive learning: SimCLR and CLIP

Contrastive learning pulls representations of positive pairs together and pushes others apart. SimCLR [Ch+20] makes positives by two random augmentations of one image; CLIP [RaKiHa+21] pairs an image with its caption. With L_2 -normalised embeddings and temperature τ , the **InfoNCE** loss for a batch is a cross-entropy over the similarity matrix:

code/dl/c10_generative.py

```
1 def clip_loss(img, txt, tau):
2     logits = img @ txt.T / tau                # (B, B): pair i with every
        ↪ caption j
3     labels = torch.arange(len(img))           # the matching caption is on the
        ↪ diagonal
4     return 0.5 * (F.cross_entropy(logits, labels) + F.cross_entropy(logits.T,
        ↪ labels))
```

For a batch of 4 with diagonal cosine similarities (0.77, 0.92, 0.78, 0.71) and $\tau = 0.1$ the loss is 0.0083 (random embeddings would give about $\ln 4 = 1.3863$); the hand computation

matches. Larger batches mean more negatives and a harder, more informative task. CLIP's joint space enables zero-shot classification: embed "a photo of a {class}" for each class and pick the nearest to the image.

§10.5.ii word2vec

Skip-gram [Mi+13] learns word vectors by predicting context words from a centre word. The full softmax over the vocabulary is expensive, so **negative sampling** replaces it by $1 + k$ binary classifications: the true context word against k words drawn from a noise distribution (unigram frequency to the power $3/4$):

$$\mathcal{L} = -\log \sigma(\mathbf{u}_o^\top \mathbf{v}_c) - \sum_{j=1}^k \log \sigma(-\mathbf{u}_{n_j}^\top \mathbf{v}_c).$$

code/dl/c10_generative.py

```
1 def sgns_loss_grad(Win, Wout, c, o, negs):
2     v = Win[c]                                # centre word vector
3     u_pos, u_neg = Wout[o], Wout[negs]        # context and negative "
4     ↪ output" vectors
5     s_pos, s_neg = sig(u_pos @ v), sig(-u_neg @ v)
6     loss = -np.log(s_pos) - np.log(s_neg).sum()
7     g_v = -(1 - s_pos) * u_pos + ((1 - s_neg)[:, None] * u_neg).sum(0)
8     g_pos = -(1 - s_pos) * v
9     g_neg = (1 - s_neg)[:, None] * v[None, :]
10    return loss, g_v, g_pos, g_neg
```

The loss (2.7431 in the script's example) and all three hand-derived gradients match autograd. Words in similar contexts get similar vectors; vector arithmetic captures some relations. Unlike Transformer embeddings, these are *static*: one vector per word regardless of context.

§10.5.iii Vision Transformers

A ViT [DoBeKo+21] cuts an image into $P \times P$ patches, flattens and linearly projects each (equivalently, a convolution with kernel and stride P), adds position embeddings and a learnable [CLS] token, and runs a standard Transformer encoder. For $224 \times 224 \times 3$ images and $P = 16$: 196 patch tokens of dimension 768; projecting to $d_{\text{model}} = 768$ costs 590,592 parameters (checked against `nn.Conv2d(3, 768, 16, stride=16)`). Halving the patch size to 8 gives 784 tokens and, attention being quadratic, roughly sixteen times the attention cost. ViTs have weaker built-in priors than CNNs (no locality or translation equivariance by construction), so they need more data or stronger augmentation, and scale very well when they get it.

VAE: maximise reconstruction minus KL, backpropagate through $\mu + \sigma \odot \epsilon$.
GAN: a two-player game whose optimum matches the data distribution, hard to balance. **Diffusion:** learn to predict the noise. **Contrastive learning:** classify which pair belongs together.

§10.6 A few problems to think about

Problem 10A. Compute $\text{KL}(\mathcal{N}(0, 4) \| \mathcal{N}(0, 1))$ and $\text{KL}(\mathcal{N}(2, 1) \| \mathcal{N}(0, 1))$.

hint · solution



Problem 10B. A discriminator outputs 0.5 everywhere. What does the proposition say about the generator?

[hint](#) · [solution](#)

Problem 10C. Why can't we backpropagate through $\mathbf{z} \sim \mathcal{N}(\mu, \sigma^2)$ directly, and how does the reparameterisation fix it? Name a distribution for which the trick is not available in this simple form.

[hint](#) · [solution](#)

Problem 10D. How many tokens does a ViT with 14×14 patches see for a 224×224 image, including [CLS]? What is the shape of its attention matrix per head?

[hint](#) · [solution](#)

Problem 10E. In DDPM with $\bar{\alpha}_t = 0.25$, write $\mathbf{x}_t$ in terms of $\mathbf{x}_0$ and $\boldsymbol{\varepsilon}$, and give the signal-to-noise ratio $\bar{\alpha}_t/(1 - \bar{\alpha}_t)$.

[hint](#) · [solution](#)

IV

Interview Mode

11 Practice and PyTorch

Knowing the maths is half of an ML-engineer interview; the other half is whether you can make a model train and know why it does not. This chapter is the working knowledge: tensors and broadcasting, the canonical training loop, the bugs that do not crash but quietly ruin results (each demonstrated with numbers), a debugging routine, distributed and mixed-precision training in concept, and one page each on graphs, reinforcement learning and time series.

§11.1 Tensors, modules and the training loop

Prototypical example for this section: $(B, 1) - (B,)$ broadcasts to a 4×4 matrix, and an MSE that should be 0.0 comes out as 2.500.

§11.1.i Tensors and broadcasting

A tensor has a shape, a dtype and a device; operations between tensors of different shapes **broadcast**: align shapes from the right; each pair of dimensions must be equal or one of them 1 (or missing), and size-1 dimensions are stretched. $(3, 1) + (1, 4)$ gives 3×4 ; $(2, 3) + (3)$ gives 2×3 ; $(2, 3) + (2)$ raises a `RuntimeError`. Broadcasting is what makes $X @ W.T + b$ work for a batch; it is also the source of the nastiest shape bug: predictions of shape $(B, 1)$ minus targets of shape (B) silently become (B, B) , and the loss is the mean over all pairs. In the prototype, perfect predictions get an MSE of 2.500. (`F.mse_loss` warns about this; a hand-written loss does not.)

§11.1.ii Modules, autograd, the loop

An `nn.Module` holds parameters (registered automatically when assigned as attributes) and defines `forward`; autograd records operations on tensors with `requires_grad` and `loss.backward()` fills `.grad`; a `Dataset` returns single examples, a `DataLoader` batches, shuffles and loads them in worker processes. The loop every interviewer expects you to write from memory:

```
code/dl/c11_pytorch.py

1 def train(model, loader, epochs, lr=1e-2, loss_fn=nn.CrossEntropyLoss()):
2     opt = torch.optim.AdamW(model.parameters(), lr=lr)
3     for epoch in range(epochs):
4         model.train()                                # dropout on, BN uses batch
5         ↪ statistics
6         for xb, yb in loader:
7             opt.zero_grad()                          # gradients accumulate otherwise
8             loss = loss_fn(model(xb), yb)            # raw logits in, class indices
9             ↪ as targets
10            loss.backward()
11            opt.step()
12        return model
13
14 @torch.no_grad()                                    # no graph, less memory
15 def evaluate(model, X, Y):
```

```

14     model.eval()                                     # dropout off, BN uses running
        ↪ stats
15     return (model(X).argmax(1) == Y).float().mean().item()

```

Five lines carry most of the bugs: `model.train()/model.eval()`, `zero_grad()`, what goes into the loss, `torch.no_grad()` for evaluation, and `shuffle=True`.

§11.2 The silent-bug catalogue

Prototypical example for this section: Training on label-sorted data without shuffling: validation accuracy 49.5% after one epoch, against 92.0% with shuffling.

Every row below was run by `code/dl/c11_pytorch.py` on a 20-feature binary classification task with an MLP (BatchNorm and dropout included).

bug	what the script measured
forgetting <code>model.eval()</code>	validation accuracy 95.0% instead of 96.0%; two forward passes on the same input differ by up to 4.511 in a logit (dropout is random, BN uses batch statistics); a single example raises a <code>ValueError</code>
softmax before <code>CrossEntropyLoss</code>	same accuracy on this easy task (95.0% vs 95.0%) but the loss cannot go below 0.313 (it ends at 0.344 vs 0.091): probabilities in $[0, 1]$ used as logits, gradients squashed, confidence meaningless
softmax over the wrong dim ($B, 1$) vs (B) targets	“probabilities” whose rows sum to 0.43, 1.87, 0.36, 0.34 (normalised over the batch instead of the classes)
forgetting <code>zero_grad()</code>	loss of a perfect model 2.500 instead of 0
no shuffling (sorted labels)	the second backward pass exactly doubles the gradient (asserted): updates use the sum of all past gradients
label leakage	49.5% vs 92.0% after one epoch: every batch has one class, the model chases the last one
resuming without optimiser state	a feature that is the label plus noise gives 100.0% validation accuracy (honest features: 90.0%), and collapses in deployment where the label is unknown
	weights after 20 more steps differ by up to 0.128 from uninterrupted training (Adam’s moments were reset)

Others worth naming: evaluating without `torch.no_grad()` (memory grows), `.item()` or `.numpy()` inside the forward pass (cuts the graph), in-place operations on tensors needed for backward, data augmentation applied to the validation set, normalising with statistics computed on the full dataset including the test split, mismatched tokeniser or image preprocessing between training and serving, and learning-rate schedulers stepped per batch instead of per epoch (chapter 4).

§11.3 Debugging a model that does not learn

Prototypical example for this section: Thirty-two examples, 200 Adam steps: the training loss must reach almost zero (1.3×10^{-4} here); if it does not, the bug is in the code, not in the data.

Algorithm: Debugging routine

1. Check the initial loss: about $\ln K$ for K balanced classes ([chapter 2](#)).
2. **Overfit one batch**: turn off regularisation and fit a single small batch to near-zero loss. Failure means a bug in the model, loss, labels or optimiser wiring.
3. Check shapes and dtypes at every stage; print one batch and its labels; visualise inputs after augmentation.
4. Check gradients: non-zero and finite for every parameter after `backward()`; their norms per layer (vanishing or exploding, [chapter 3](#)).
5. Tune the learning rate first (a sweep over powers of ten), then the schedule, then regularisation.
6. Compare with a trivial baseline (majority class, linear model) and grow the model only after the small version works.

§11.3.i Reproducibility

Seed Python, NumPy and PyTorch (and every `DataLoader` generator and worker); the same seed gives identical initial weights (asserted; a different seed differs by up to 0.652). For bitwise reproducibility also request deterministic algorithms (some GPU kernels are non-deterministic by default) and fix library versions; even then, results differ across hardware. Report the spread over several seeds, not one lucky run.

§11.3.ii Checkpointing

Save everything needed to continue exactly: model and optimiser `state_dicts`, the scheduler, the epoch or step, and the random-generator states.

code/dl/c11_pytorch.py

```

1 buf = io.BytesIO()
2 torch.save({"model": mA.state_dict(), "opt": oA.state_dict(), "rng": gA.get_state()
   ↪ }, buf)
3 buf.seek(0); ck = torch.load(buf)
4 mB = nn.Sequential(nn.Linear(20, 16), nn.ReLU(), nn.Linear(16, 2)); mB.
   ↪ load_state_dict(ck["model"])
5 oB = torch.optim.Adam(mB.parameters(), 1e-2); oB.load_state_dict(ck["opt"])
6 gB = torch.Generator(); gB.set_state(ck["rng"])

```

Resuming from this checkpoint reproduces uninterrupted training bit for bit (asserted); dropping the optimiser state does not (previous table).

§11.4 Scaling up: precision, memory, many GPUs

Prototypical example for this section: Mixed-precision training with Adam keeps about 16 bytes per parameter, so a 7B-parameter model needs about 112 GB before counting activations.

§11.4.i Mixed precision

format	largest value	machine epsilon	smallest normal
fp32	3.40×10^{38}	1.19×10^{-7}	1.18×10^{-38}
fp16	6.55×10^4	9.77×10^{-4}	6.10×10^{-5}
bf16	3.39×10^{38}	7.81×10^{-3}	1.18×10^{-38}

(from `torch.info`). Half precision halves memory and runs much faster on tensor cores. fp16 has a narrow range: a gradient of 10^{-8} becomes 0 in fp16 (it is below the smallest subnormal); multiplying the loss by 65536 before `backward` and dividing afterwards recovers it (1.00×10^{-8}), which is what **loss scaling** (`GradScaler`) automates. bf16 keeps fp32's range with fewer mantissa bits, so it needs no loss scaling but is coarser: a 256×256 matrix product in bf16 has a worst error of 0.22 against 4.3×10^{-5} in fp32. Mixed precision (`torch.autocast`) runs matrix products in half precision while keeping fp32 master weights, reductions and the optimiser in fp32.

§11.4.ii Where training memory goes

Per parameter, mixed-precision Adam stores fp16 weights (2 bytes) and gradients (2), plus fp32 master weights (4) and two moments ($4 + 4$): about 16 bytes. Activations come on top and scale with batch size, sequence length and depth; gradient checkpointing trades recomputation for them. This arithmetic is why LoRA and quantised bases ([chapter 9](#)) make fine-tuning feasible on one GPU.

§11.4.iii Data parallelism and gradient accumulation

Distributed data parallel (DDP): every GPU holds a full replica, processes a different slice of the batch, and the gradients are averaged with an all-reduce before the identical optimiser step on every replica; the effective batch is per-GPU batch times number of GPUs. When the model does not fit on one GPU, **sharded** data parallelism (FSDP/ZERO) splits parameters, gradients and optimiser state across GPUs, and tensor or pipeline parallelism split the layers themselves. **Gradient accumulation** emulates a large batch on one device: run k micro-batches, add their gradients (each loss divided by k), then step. The script asserts that four micro-batches of 8 give exactly the gradient of one batch of 32 (true for a mean loss without BatchNorm, whose statistics do depend on the micro-batch).

§11.5 Three more families, one page each

Prototypical example for this section: A two-feature graph of four nodes, one GCN layer: every node's new features mix its neighbours' features, weighted by $1/\sqrt{d_i d_j}$.

§11.5.i Graph neural networks

Data with relations (molecules, social and road networks, recommendation) are graphs. A **message-passing** layer updates each node from its neighbours,

$$\mathbf{h}'_i = \phi\left(\mathbf{h}_i, \text{AGG}_{j \in \mathcal{N}(i)} \psi(\mathbf{h}_i, \mathbf{h}_j)\right),$$

with a permutation-invariant aggregation (sum, mean, max). The graph convolutional network (GCN) layer is the simplest:

code/dl/c11_pytorch.py

```

1 def gcn_layer(A, H, W):
2     A_hat = A + np.eye(len(A))                # add self-loops
3     d = A_hat.sum(1)
4     A_norm = A_hat / np.sqrt(d[:, None] * d[None, :]) # D^-1/2 (A+I) D^-1/2
5     return np.maximum(0, A_norm @ H @ W)        # aggregate neighbours,
    ↪ transform, ReLU

```

On a 4-node graph the output features are $\begin{pmatrix} 0.311 & 0.933 \\ 0.311 & 0.933 \\ 0.269 & 0.808 \\ 0.177 & 0.530 \end{pmatrix}$; the script checks the matrix form against an explicit per-node message sum and that relabelling the nodes relabels the output (permutation equivariance, as for self-attention, which is message passing on a complete graph). k layers see k -hop neighbourhoods; too many layers **over-smooth** all nodes to similar vectors.

§11.5.ii Reinforcement learning

An agent in state s takes action a , receives reward r , and seeks to maximise expected discounted return $\sum_t \gamma^t r_t$. **Policy gradient** (REINFORCE) increases the log-probability of actions in proportion to the return they led to: $\nabla J = \mathbb{E}[\nabla \log \pi_\theta(a | s) G]$, usually with a baseline subtracted to reduce variance. On a three-armed bandit with success probabilities 0.2, 0.5, 0.8, 2000 REINFORCE updates from a uniform policy end at $\pi = (0.006, 0.014, 0.980)$. **DQN** learns action values with a network and regresses $Q(s, a)$ toward the Bellman target $y = r + \gamma \max_{a'} Q_{\text{target}}(s', a')$, e.g. $y = 1 + 0.9 \cdot 5 = 5.5$ when the next state's values are (2, 5, 3); experience replay and a slowly updated target network stabilise it. RLHF (chapter 9) is policy optimisation with a learned reward.

§11.5.iii Deep models for time series

Frame forecasting as supervised learning with sliding windows: from a series of length 100, windows of $W = 24$ inputs and $H = 6$ targets give $100 - W - H + 1 = 71$ examples. Models: 1-D CNNs (causal, dilated convolutions for long receptive fields), LSTMs (chapter 7), and Transformers over patches of the series. The traps are not architectural: split by time (never shuffle across the split), compute normalisation statistics on the training period only, and beat seasonal-naive and gradient-boosting baselines before claiming anything.

Shuffle, zero_grad, logits into the loss, eval() and no_grad() for evaluation, checkpoint the optimiser too; when stuck, overfit one batch first.

§11.6 A few problems to think about

Problem 11A. preds has shape (64, 1) and target has shape (64). What is the shape of (preds - target) ** 2, and how do you fix it?

hint · solution

Problem 11B. Your validation accuracy changes every time you evaluate the same checkpoint on the same data. Name two likely causes.

hint · solution

Problem 11C. You can fit a batch of 8 on your GPU but want the gradient of a batch of 64. Write the accumulation loop, and say what breaks if the model has BatchNorm.

hint · solution

Problem 11D. Estimate the memory for weights, gradients and Adam state when training a 1.3B-parameter model in mixed precision.

[hint](#) · [solution](#)



12 Implement it

“Implement it in NumPy” is the interview question that separates people who have used deep learning from people who understand it. This chapter assembles a tiny framework, every layer with an explicit forward and backward pass, and proves each piece correct the strict way: the NumPy model and a PyTorch model with the same initial weights are trained side by side on the same batches, and their loss trajectories are asserted equal. Several pieces already appeared where they were derived; the table points to them.

component	where	checked against
MLP backprop, gradient checking	chapter 3; here	autograd; finite differences
reverse-mode autograd	chapter 3	autograd
softmax regression	here	finite differences
SGD, momentum, RMSProp, Adam, AdamW	chapter 4; Adam again here	torch.optim
BatchNorm forward/backward, running statistics	chapter 5; layer here	nn.BatchNorm1d
convolution forward/backward (im2col)	here	F.conv2d + autograd
LSTM cell; LSTM layer over a sequence	chapter 7; here	nn.LSTMCell, nn.LSTM
scaled dot-product and multi-head attention	chapter 8	nn.MultiheadAttention
full encoder-decoder Transformer	chapter 8	PyTorch twin, same logits
BPE tokeniser	chapter 9	round trip
character-level language model	here	PyTorch twin, same trajectory

§12.1 A framework in sixty lines

Prototypical example for this section: `Linear.backward` receives $\partial\mathcal{L}/\partial\text{output}$, stores $\partial\mathcal{L}/\partial\mathbf{W} = \mathbf{g}^\top \mathbf{x}$ and returns $\mathbf{g}\mathbf{W}$; everything else is bookkeeping.

Each layer caches what its backward pass needs, exposes `params` and, after `backward`, `grads` with the same keys; the optimiser walks over both.

code/dl/c12_implement.py

```
1 class Linear:
2     def __init__(self, W, b):
3         self.params = {"W": W.copy(), "b": b.copy()}
4     def forward(self, x):
5         self.x = x
6         return x @ self.params["W"].T + self.params["b"]
7     def backward(self, g):
8         # g = dL/d(output), shape (B,
9         #   ↪ out)
10         self.grads = {"W": g.T @ self.x, "b": g.sum(0)}
11         return g @ self.params["W"]
12
13 class ReLU:
14     params = {}
15     def forward(self, x):
16         self.mask = x > 0
17         return x * self.mask
18     def backward(self, g):
19         return g * self.mask
20
21 class Sequential:
```

```

20     def __init__(self, *layers):
21         self.layers = layers
22     def forward(self, x):
23         for l in self.layers:
24             x = l.forward(x)
25         return x
26     def backward(self, g):
27         for l in reversed(self.layers):
28             g = l.backward(g)
29         return g
30
31     def softmax_ce(logits, y)::                                     # mean cross-entropy and its
    ↪ gradient
32         z = logits - logits.max(1, keepdims=True)
33         p = np.exp(z) / np.exp(z).sum(1, keepdims=True)
34         B = len(y)
35         loss = -np.log(p[np.arange(B), y]).mean()
36         g = p.copy(); g[np.arange(B), y] -= 1
37         return loss, g / B
38
39     class Adam:
40     def __init__(self, layers, lr=1e-3, b1=0.9, b2=0.999, eps=1e-8):
41         self.layers, self.lr, self.b1, self.b2, self.eps, self.t = layers, lr, b1,
    ↪ b2, eps, 0
42         self.m = [{k: np.zeros_like(v) for k, v in l.params.items()} for l in
    ↪ layers]
43         self.v = [{k: np.zeros_like(v) for k, v in l.params.items()} for l in
    ↪ layers]
44     def step(self):
45         self.t += 1
46         for l, m, v in zip(self.layers, self.m, self.v):
47             for k in l.params:
48                 g = l.grads[k]
49                 m[k] = self.b1 * m[k] + (1 - self.b1) * g
50                 v[k] = self.b2 * v[k] + (1 - self.b2) * g * g
51                 mh, vh = m[k] / (1 - self.b1 ** self.t), v[k] / (1 - self.b2 **
    ↪ self.t)
52                 l.params[k] -= self.lr * mh / (np.sqrt(vh) + self.eps)

```

Example 12.1.1 (Softmax regression, gradient-checked)

Softmax regression is Linear followed by softmax_ce. On a 3-class, 10-feature dataset the analytic gradient of all 30 weights agrees with centred finite differences (step 10^{-6}) to 10^{-7} .

§12.2 BatchNorm as a layer

Prototypical example for this section: The same equations as chapter 5, plus the two running averages and a training flag.

code/dl/c12_implement.py

```

1 class BatchNorm1d:
2     def __init__(self, C, momentum=0.1, eps=1e-5):
3         self.params = {"gamma": np.ones(C), "beta": np.zeros(C)}
4         self.rm, self.rv, self.mom, self.eps, self.training = np.zeros(C), np.ones(
    ↪ C), momentum, eps, True

```

```

5     def forward(self, x):
6         if self.training:
7             mu, var = x.mean(0), x.var(0)
8             B = len(x)
9             self.rm = (1 - self.mom) * self.rm + self.mom * mu
10            self.rv = (1 - self.mom) * self.rv + self.mom * var * B / (B - 1)    #
        ↪ unbiased
11        else:
12            mu, var = self.rm, self.rv
13            self.std = np.sqrt(var + self.eps)
14            self.xhat = (x - mu) / self.std
15            return self.params["gamma"] * self.xhat + self.params["beta"]
16    def backward(self, g):
17        B = len(g)
18        self.grads = {"gamma": (g * self.xhat).sum(0), "beta": g.sum(0)}
19        dxh = g * self.params["gamma"]
20        return (B * dxh - dxh.sum(0) - self.xhat * (dxh * self.xhat).sum(0)) / (B *
        ↪ self.std)

```

Example 12.2.1 (A full training run, NumPy against PyTorch)

A $10 \rightarrow 32 \rightarrow 3$ MLP with BatchNorm and ReLU, PyTorch's initial weights copied into the NumPy layers, both trained with Adam ($\eta = 0.01$) on the same 50 random minibatches of 32. The two loss trajectories agree to 10^{-9} at every step (from 1.1128 to 0.5127); so do the final weights, BatchNorm's running variance, and the evaluation-mode outputs. Training accuracy ends at 81.6%. An identical trajectory over 50 Adam steps is a much stronger test than one gradient check: any error in a backward pass, in BatchNorm's unbiased running variance or in Adam's bias correction would make the curves drift apart.

§12.3 Convolution via im2col

Prototypical example for this section: Unfold every $k \times k \times C$ patch into a column; the convolution becomes one matrix product, and its backward pass is two more.

im2col turns a convolution into a matrix multiplication: the input (B, C, H, W) becomes columns (B, Ck^2, H_oW_o) , the weights a matrix (C_{out}, Ck^2) , and the output their product. Backward: $\partial\mathcal{L}/\partial\mathbf{W}$ is the product of the output gradient with the columns, and $\partial\mathcal{L}/\partial\mathbf{x}$ multiplies by $\mathbf{W}^\top$ and then scatter-adds the columns back to the image (**col2im**, the adjoint of **im2col**; overlapping patches accumulate, exactly as in the transposed convolution of [chapter 6](#)).

code/dl/c12_implement.py

```

1  def im2col(x, k, s=1, p=0):
2      """x: (B, C, H, W) -> columns (B, C*k*k, Ho*Wo), one column per output position
        ↪ ."""
3      B, C, H, W = x.shape
4      x = np.pad(x, ((0, 0), (0, 0), (p, p), (p, p)))
5      Ho, Wo = (H + 2 * p - k) // s + 1, (W + 2 * p - k) // s + 1
6      cols = np.empty((B, C, k, k, Ho, Wo))
7      for u in range(k):
8          for v in range(k):
9              cols[:, :, u, v] = x[:, :, u:u + s * Ho:s, v:v + s * Wo:s]
10     return cols.reshape(B, C * k * k, Ho * Wo), (Ho, Wo)

```

```

11
12 def col2im(cols, shape, k, s=1, p=0):
13     """Adjoint of im2col: scatter-add columns back to an image (used in backward).
14     ↪ """
15     B, C, H, W = shape
16     Ho, Wo = (H + 2 * p - k) // s + 1, (W + 2 * p - k) // s + 1
17     cols = cols.reshape(B, C, k, k, Ho, Wo)
18     x = np.zeros((B, C, H + 2 * p, W + 2 * p))
19     for u in range(k):
20         for v in range(k):
21             x[:, :, u:u + s * Ho:s, v:v + s * Wo:s] += cols[:, :, u, v]
22     return x[:, :, p:p + H, p:p + W]
23
24 class Conv2d:
25     def __init__(self, W, b, s=1, p=0):          # W: (Cout, Cin, k, k)
26         self.params, self.s, self.p = {"W": W.copy(), "b": b.copy()}, s, p
27     def forward(self, x):
28         Cout, Cin, k, _ = self.params["W"].shape
29         self.shape, self.k = x.shape, k
30         self.cols, (Ho, Wo) = im2col(x, k, self.s, self.p)
31         y = self.params["W"].reshape(Cout, -1) @ self.cols + self.params["b"][:,
32         ↪ None] # (B, Cout, Ho*Wo)
33         return y.reshape(x.shape[0], Cout, Ho, Wo)
34     def backward(self, g):                      # g: (B, Cout, Ho, Wo)
35         B, Cout = g.shape[:2]
36         g2 = g.reshape(B, Cout, -1)
37         self.grads = {"W": np.einsum("bop,bcp->oc", g2, self.cols).reshape(self.
38         ↪ params["W"].shape),
39         "b": g2.sum((0, 2))}
40         dcols = np.einsum("oc,bop->bcp", self.params["W"].reshape(Cout, -1), g2)
41         return col2im(dcols, self.shape, self.k, self.s, self.p)

```

For a batch of two $3 \times 7 \times 7$ inputs, four 3×3 filters, stride 2 and padding 1, the output has shape $2 \times 4 \times 4 \times 4$, and the output, $\partial \mathcal{L} / \partial \mathbf{x}$, $\partial \mathcal{L} / \partial \mathbf{W}$ and $\partial \mathcal{L} / \partial \mathbf{b}$ all equal PyTorch's. Production libraries use im2col-like lowering, FFT or Winograd transforms, or direct kernels; the memory cost of im2col (each input pixel copied up to k^2 times) is the price of reusing a fast matrix multiply.

§12.4 An LSTM layer

Prototypical example for this section: Loop the cell of chapter 7 over time, carrying $(\mathbf{h}, \mathbf{c})$.

code/dl/c12_implement.py

```

1 def lstm_forward(xs, Wih, Whh, bih, bhh):
2     """xs: (T, B, D); weights in nn.LSTM layout (gates i, f, g, o). Returns all h_t
3     ↪ """
4     T, B, _ = xs.shape
5     H = Whh.shape[1]
6     h, c = np.zeros((B, H)), np.zeros((B, H))
7     hs = []
8     for t in range(T):
9         z = xs[t] @ Wih.T + h @ Whh.T + bih + bhh
10        i, f, g, o = sig(z[:, :H]), sig(z[:, H:2 * H]), np.tanh(z[:, 2 * H:3 * H]),
11        ↪ sig(z[:, 3 * H:])
12        c = f * c + i * g
13        h = o * np.tanh(c)

```

```

12         hs.append(h)
13     return np.stack(hs), (h, c)

```

With weights taken from an `nn.LSTM(4, 6)`, a 9-step sequence with batch 3 gives the same outputs and final cell state as PyTorch. The backward pass is BPTT: run the cell's backward from $t = T$ down to 1, adding the incoming $\partial\mathcal{L}/\partial\mathbf{h}_t$ from the output at each step to the gradient arriving from step $t + 1$, and accumulating the weight gradients over time (gradient-check it as in [chapter 3](#)).

§12.5 A character-level language model

Prototypical example for this section: Three characters in, a distribution over the next character out: embedding, one tanh hidden layer, softmax.

Two more layers complete the framework: an `Embedding` whose backward pass scatter-adds gradients into the rows that were used (`np.add.at`, because a character can appear twice in one batch), and `Tanh`.

code/dl/c12_implement.py

```

1 class Embedding:
2     def __init__(self, E):
3         self.params = {"E": E.copy()}
4     def forward(self, idx):                                # idx: (B, ctx) -> (B, ctx*dim)
5         self.idx = idx
6         return self.params["E"][idx].reshape(len(idx), -1)
7     def backward(self, g):
8         gE = np.zeros_like(self.params["E"])
9         np.add.at(gE, self.idx, g.reshape(*self.idx.shape, -1)) # rows used twice
10        ↪ get both gradients
11        self.grads = {"E": gE}
12
13 class Tanh:
14     params = {}
15     def forward(self, x):
16         self.y = np.tanh(x); return self.y
17     def backward(self, g):
18         return g * (1 - self.y ** 2)

```

Example 12.5.1 (Training it, twice)

A corpus of a few repeated sentences with a vocabulary of 27 characters; context of 3, embeddings of size 8, 64 hidden units: 3571 parameters. Trained with the NumPy Adam above and, from the same initial weights and batches, with `torch.optim.Adam`: the two 300-step loss trajectories agree to 10^{-8} . The loss falls from 3.284 (close to $\ln V = 3.296$, as it should at initialisation) to 0.238, a per-character perplexity of 1.27 on this tiny, repetitive corpus. Sampling from the NumPy model at temperature 0.5, seeded with “the”:

```

output (generated by build.sh)
the weights fall step. repeat the loss. it adjusts weights weights weights

```

Locally fluent, globally aimless, and clearly memorised: a three-character window

cannot track meaning. Longer context needs recurrence or attention ([chapters 7 and 8](#)).

Every layer is a forward function, a cache and a backward function that maps the output gradient to the input gradient and the parameter gradients. Get the shapes right and check against a reference, step by step.

§12.6 A few problems to think about

Problem 12A. Write the forward and backward pass of a `Sigmoid` layer in the style of this chapter.

[hint · solution](#)

Problem 12B. Why does `Embedding.backward` use `np.add.at` instead of `gE[idx] += g`?

[hint · solution](#)

Problem 12C. For a convolution with $C = 64$ input channels, $k = 3$, on a 56×56 map with stride 1 and padding 1, how many numbers does the `im2col` matrix hold per image, compared with the input?

[hint · solution](#)



13 Rapid-fire

§13.1 Rapid-fire

[STUB: Rapid-fire]

14 Problems

§14.1 Problems

[STUB: Problems]



Backmatter

A Hints to problems

- 1A. Each layer contributes $(\text{inputs} + 1) \times \text{outputs}$.
- 1B. NAND is 0 only at $(1, 1)$.
- 1C. Both R and γ are lengths in input space.
- 1D. Where can the slope of f change?
- 1E. Two are true.
- 1F. The other half of the proof bounds $\|\mathbf{w}\|$.
- 2A. Shift invariance.
- 2B. Write $\tanh z = (1 - e^{-2z})/(1 + e^{-2z})$.
- 2C. $\log \sigma(z) = -\log(1 + e^{-z})$ and $\log(1 - \sigma(z)) = -z - \log(1 + e^{-z})$.
- 2D. Four of them.
- 2E. Let S be the sum of the other exponentials; $p_y = e^{z_y}/(e^{z_y} + S)$.
- 2F. What does a softmax force the 20 probabilities to do?
- 3A. Max routes, multiply swaps.
- 3B. Every gradient has the shape of the thing it differentiates.
- 3C. $\delta^{(2)}$ gains a factor $\hat{y}(1 - \hat{y}) \leq \frac{1}{4}$.
- 3D. Look at $\delta^{(l)}$ and $\partial \mathcal{L} / \partial \mathbf{W}^{(l)}$ row by row.
- 3E. Differentiate: $\partial \mathbf{h}^{(l+1)} / \partial \mathbf{h}^{(l)} = \mathbf{I} + \partial F / \partial \mathbf{h}^{(l)}$.
- 3F. Where can the gradient be identically zero?
- 4A. $\lceil N/B \rceil$ per epoch.
- 4B. After bias correction $\hat{m}_1 = g_1$ and $\hat{v}_1 = g_1^2$.
- 4C. Geometric series.
- 4D. $x \leftarrow (1 - \eta\lambda)x$.
- 4E. The norm is 10.
- 4F. $\mathbf{v}_t = \mathbf{g}(1 - \beta^t)/(1 - \beta)$.
- 5A. $\sqrt{6/(n_{\text{in}} + n_{\text{out}})}$ and $\sqrt{2/n_{\text{in}}}$.
- 5B. The variance is zero.
- 5C. Inverted dropout already rescaled during training.
- 5D. Three are true.
- 5E. BN subtracts the batch mean.

- 5F.** Look at the gap and at when validation was best.
- 6A.** Apply the formula layer by layer; pooling has no parameters.
- 6B.** $\lfloor (7 - 3)/3 \rfloor + 1$.
- 6C.** Track r and j .
- 6D.** Depthwise: $9 \cdot 128 + 128$. Pointwise: $128 \cdot 256 + 256$.
- 6E.** Is this overfitting?
- 6F.** Intersection is a 5×10 strip.
- 7A.** $4[H(D + H) + 2H]$ and $3[\cdot]$.
- 7B.** The two directions are concatenated.
- 7C.** $|\tanh'| \leq 1$.
- 7D.** What does the backward direction see?
- 7E.** $\mathbf{c}_t = \mathbf{f} \odot \mathbf{c}_{t-1} + \mathbf{i} \odot \mathbf{g}$.
- 7F.** Each beam is extended by every token.
- 8A.** Scores $\mathbf{q}^\top \mathbf{k} / \sqrt{2}$, softmax per query.
- 8B.** $4d^2 + 4d$ and $2dd_{\text{ff}} + d_{\text{ff}} + d$.
- 8C.** $\sqrt{64} = 8$.
- 8D.** Queries come from the decoder.
- 8E.** What is each trained to predict?
- 8F.** $2 \cdot L \cdot n_{\text{kv}} \cdot d_{\text{head}} \cdot T \cdot B \cdot 2$ bytes.
- 9A.** exp of the mean negative log-probability.
- 9B.** $r(d_{\text{in}} + d_{\text{out}})$.
- 9C.** Top- p keeps the smallest prefix with mass $\geq p$.
- 9D.** Where does training memory go?
- 9E.** What can each position see during pretraining?
- 10A.** $\frac{1}{2}(\mu^2 + \sigma^2 - 1 - \log \sigma^2)$.
- 10B.** $D^\star = p_{\text{data}} / (p_{\text{data}} + p_g)$.
- 10C.** Where is the randomness?
- 10D.** $224/14$.
- 10E.** Closed form of $q(\mathbf{x}_t | \mathbf{x}_0)$.
- 11A.** Broadcast from the right.
- 11B.** Which layers are random or batch-dependent?
- 11C.** Divide the loss.
- 11D.** About 16 bytes per parameter.

12A. Cache the output.

12B. What if an index repeats?

12C. $Ck^2 \times H_o W_o$.

B Solutions to problems

- 1A.** $101 \cdot 50 = 5050$, $51 \cdot 50 = 2550$, $51 \cdot 3 = 153$; total 7753.
- 1B.** $\mathbf{w} = (-2, -2)$, $b = 3$: the sums are 3, 1, 1, -1 on $(0, 0)$, $(0, 1)$, $(1, 0)$, $(1, 1)$, so the output is 1, 1, 1, 0. NAND is functionally complete (every Boolean circuit can be rewritten with NAND gates only), and each gate is one neuron, so any Boolean function is a feed-forward network of step neurons; the depth is the circuit's depth.
- 1C.** R and γ both scale by c , so the bound is unchanged. The run itself is identical up to scale: by induction every weight vector is c times the original one, so $\text{sign}(\mathbf{w}^\top c\mathbf{x})$ equals the original prediction and the same points are misclassified in the same order. (With a bias the constant coordinate 1 is not scaled and exact invariance is lost.)
- 1D.** Each term is linear except at $t = -b_k/w_k$ (if $w_k \neq 0$), where its slope jumps. So f is linear between consecutive elements of the set of at most m such breakpoints, giving at most $m + 1$ intervals. Contrast with the 2^L pieces of the depth- L sawtooth built from $2L$ units.
- 1E.** (c) and (e). (a) fails because the boundary $\sigma(z) = \frac{1}{2}$ is a line; (b) fails because affine maps compose to an affine map; (d) is backwards.
- 1F.** Each update adds $y_i \mathbf{x}_i^\top \mathbf{w}^* \geq \gamma > 0$. But $\mathbf{w}^\top \mathbf{w}^*$ could grow simply because $\|\mathbf{w}\|$ grows without $\mathbf{w}$ turning toward $\mathbf{w}^*$; the bound $\|\mathbf{w}_k\| \leq \sqrt{k}R$ shows the length grows only like $\sqrt{k}$ while the projection grows like k , so the cosine $\mathbf{w}_k^\top \mathbf{w}^* / \|\mathbf{w}_k\| \geq \sqrt{k}\gamma/R$, which cannot exceed 1, forces $k \leq (R/\gamma)^2$.
- 2A.** Both give the uniform distribution, so the loss is $\ln 4 \approx 1.386$ (row 2 of the table).
- 2B.** $2\sigma(2z) - 1 = \frac{2}{1+e^{-2z}} - 1 = \frac{1-e^{-2z}}{1+e^{-2z}} = \tanh z$. Differentiating, $\tanh'(z) = 4\sigma'(2z)$, so $\tanh'(0) = 4 \cdot \frac{1}{4} = 1$.
- 2C.** Using the hint, $\text{BCE} = \log(1 + e^{-z}) + (1 - y)z$. Its derivative is $-\frac{e^{-z}}{1+e^{-z}} + 1 - y = -(1 - \sigma(z)) + 1 - y = \sigma(z) - y$. For $z \geq 0$ the stable form is $z - zy + \log(1 + e^{-z})$, the same expression. For $z < 0$ write $\log(1 + e^{-z}) = -z + \log(1 + e^z)$; then $\text{BCE} = -zy + \log(1 + e^z) = 0 - zy + \log(1 + e^{-|z|})$.
- 2D.** (b), (d), (e), (g). ReLU and GELU are activations; softmax is an output map.
- 2E.** Halving $-\ln p_y$ means the new probability is $\sqrt{p_y}$. With $p = e^z/(e^z + S)$, the odds $p/(1 - p) = e^z/S$, so increasing z by Δ multiplies the odds by e^Δ : $\Delta = \ln \frac{\sqrt{p}/(1-\sqrt{p})}{p/(1-p)}$. For $p = 0.2$: $\sqrt{0.2} = 0.447$, odds 0.809 vs 0.25, so $\Delta = \ln 3.24 \approx 1.17$ (the script checks that the loss indeed halves).
- 2F.** The softmax makes the tags compete: probabilities sum to 1, so an image with four true tags can give each at most about 0.25, and thresholds become meaningless. Use 20 sigmoid outputs with a binary cross-entropy per tag (`BCEWithLogitsLoss` on 20 logits).
- 3A.** $m = \max(1, 4) = 4$, $f = -8$. $\bar{m} = z = -2$; the max routes it to the winner, so $\partial f / \partial y = -2$, $\partial f / \partial x = 0$; $\partial f / \partial z = m = 4$.

- 3B.** With $\Delta = \partial\mathcal{L}/\partial\mathbf{Z} \in \mathbb{R}^{32 \times 50}$: $\partial\mathcal{L}/\partial\mathbf{W} = \Delta^\top \mathbf{X}$, shape (50, 100); $\partial\mathcal{L}/\partial\mathbf{b} = \Delta^\top \mathbf{1}$, shape (50) (sum over the batch); $\partial\mathcal{L}/\partial\mathbf{X} = \Delta\mathbf{W}$, shape (32, 100).
- 3C.** $\delta^{(2)} = (\hat{y} - y)\hat{y}(1 - \hat{y})$. With $\hat{y} = 0.4917$ the factor is 0.2499, so $\delta^{(2)} = -0.1270$ and every gradient in the network is about four times smaller than with BCE; the updates shrink by the same factor (for a fixed η). Far from the target the factor is much smaller still.
- 3D.** By induction: if all rows of $\mathbf{W}^{(l)}$ are equal and all $b_j^{(l)}$ are equal, all units of layer l have the same z , the same h , and (because the outgoing weights from each are also equal) the same δ . The gradient row j is $\delta_j \mathbf{h}^{(l-1)\top}$, the same for every j , so the rows stay equal after the update. This is **symmetry**; random initialisation breaks it. The output units are distinguished by their targets, so their δ s differ and the *rows* of the output matrix become different. But within each row the entries for different hidden units stay equal (they are $\delta_j h_i$ with equal h_i), so every hidden unit keeps seeing the same combination of output errors and the hidden-layer symmetry persists.
- 3E.** The backward product becomes $\prod_k (\mathbf{I} + J_k)$. Expanding, it contains the identity term, so $\partial\mathcal{L}/\partial\mathbf{h}^{(l)}$ includes $\partial\mathcal{L}/\partial\mathbf{h}^{(L)}$ unchanged plus corrections: even if every J_k is small, the gradient does not shrink exponentially. Without the skip the product is $\prod_k J_k$, which does.
- 3F.** (i) All units dead (huge negative biases, or a huge first learning-rate step): check the fraction of zero activations per layer. (ii) The graph is cut: a `.detach()`, `.numpy()` round trip or `torch.no_grad()` in the forward pass, or the optimiser was given the wrong parameters: check that `p.grad` is non-zero after `loss.backward()`. (A learning rate of zero is the third, non-backprop cause.)
- 4A.** $\lceil 60000/256 \rceil = 235$ steps per epoch (234 full batches and one of 96), so 2350 steps.
- 4B.** $\Delta\theta = -\eta \hat{m}_1 / \sqrt{\hat{v}_1} = -0.01 \cdot (-4)/4 = +0.01$. For $g_1 = -0.004$ the step is the same $+0.01$: Adam's first step is $\eta \text{sign}(g)$ whatever the scale (until $|g|$ approaches ε).
- 4C.** $\mathbf{v}_t = (1 - \beta_2) \sum_{s=1}^t \beta_2^{t-s} \mathbf{g}_s^2$, so $\mathbb{E}\mathbf{v}_t = (1 - \beta_2) \frac{1 - \beta_2^t}{1 - \beta_2} \mathbb{E}\mathbf{g}^2 = (1 - \beta_2^t) \mathbb{E}\mathbf{g}^2$. After 10 steps, $1 - 0.999^{10} \approx 0.00996$: without correction $\mathbf{v}$ would underestimate the second moment about 100-fold and the steps would be about $10\times$ too large.
- 4D.** Convergence iff $|1 - \eta\lambda| < 1$, i.e. $0 < \eta < 2/\lambda$; $\eta = 1/\lambda$ gives $x_1 = 0$. At $\eta = 2/\lambda$, $x \leftarrow -x$: the iterate oscillates forever between $\pm x_0$. Between $1/\lambda$ and $2/\lambda$ it converges while flipping sign every step (the zig-zag).
- 4E.** (i) (3, -4, 0); (ii) (5, -5, 0), which points in a different direction (both checked against PyTorch).
- 4F.** The step tends to $\eta\mathbf{g}/(1 - \beta)$: $10\eta\mathbf{g}$ or $100\eta\mathbf{g}$. It reaches 90% when $\beta^t \leq 0.1$: $t \geq \ln 0.1 / \ln 0.9 \approx 21.9$, so 22 steps for $\beta = 0.9$, and $t \geq 229.1$, so 230 steps for $\beta = 0.99$. Large β means a long memory and a slow start.
- 5A.** Both equal $\sqrt{6/1024} = \sqrt{2/512} = 0.0765$.
- 5B.** $\mu = 4$, $\sigma^2 = 0$, so $\hat{x} = 0/\sqrt{\varepsilon} = 0$ and the output is β for every example; the ε prevents division by zero. All information in that feature is lost for this batch, and the gradient with respect to the inputs is zero. Constant features (or batch size 1) make BN degenerate.

- 5C.** Inverted dropout trained the weights to expect unscaled activations at test time; multiplying by 0.8 shrinks every dropout layer's output by 0.8, compounding over layers (0.8^k after k such layers, before any normalisation). The model's logits shrink and its calibration and accuracy change.
- 5D.** (b), (d), (e). (a) It uses running averages; (c) dropout lowers effective capacity.
- 5E.** A bias adds the same constant b_j to every example's feature j ; the batch mean then increases by b_j and the subtraction removes it exactly, so the bias has no effect (its gradient is zero); β plays its role. Parameters: $256 \cdot 128 = 32,768$ weights plus $2 \cdot 128 = 256$ for γ, β , total 33,024 (plus 256 buffers), as PyTorch counts.
- 5F.** Overfitting: tiny training loss, large and growing gap, validation best early. (1) Early stopping / restore the epoch-8 checkpoint; (2) stronger regularisation: augmentation, weight decay, dropout; (3) more data or a smaller model. Also check for leakage-free splits before trusting either number.
- 6A.** $64 \times 64 \times 32$ with $(27+1) \cdot 32 = 896$ parameters; $32 \times 32 \times 32$ with 0; $\lfloor (32+4-5)/2 \rfloor + 1 = 16$, so $16 \times 16 \times 64$ with $(25 \cdot 32 + 1) \cdot 64 = 51,264$ (all checked in PyTorch).
- 6B.** $\lfloor 4/3 \rfloor + 1 = 2$, so 2×2 . The windows cover rows/columns 0–2 and 3–5; row and column 6 are never used: $7 + 7 - 1 = 13$ pixels are ignored.
- 6C.** (r, j) : start $(1, 1)$; $(3, 1)$; $(5, 2)$; $(9, 2)$; pool: $r = 9 + 1 \cdot 2 = 11$, $j = 4$; last conv: $r = 11 + 2 \cdot 4 = 19$. Receptive field 19×19 (confirmed by the gradient probe).
- 6D.** Depthwise 1,280, pointwise 33,024, total 34,304. Standard: $(9 \cdot 128 + 1) \cdot 256 = 295,168$, about $8.6\times$ more.
- 6E.** Higher training error is not overfitting, so a regulariser such as dropout will make it worse. This is the degradation problem: the deeper plain net is harder to optimise. Use residual connections (and BatchNorm, proper initialisation); the deeper residual net should then match or beat the shallower one.
- 6F.** Intersection 50, union $100 + 100 - 50 = 150$, $\text{IoU} = 1/3 \approx 0.3333$. Below 0.5, so both boxes survive.
- 7A.** $H(D + H) + 2H = 64 \cdot 114 + 128 = 7,424$. LSTM: 29,696; GRU: 22,272 (both checked in PyTorch).
- 7B.** $2 \cdot 64 = 128$ inputs; $128 \cdot 5 + 5 = 645$ parameters.
- 7C.** $\partial h_t / \partial h_{t-1} = w(1 - h_t^2)$ with $0 < 1 - h_t^2 \leq 1$, so the product of $T - k$ factors has absolute value at most $|w|^{T-k}$, which vanishes geometrically for $|w| < 1$. For $|w| > 1$ the factors $1 - h_t^2$ can be small (saturation), so the product may still shrink: large weights push the tanh into its flat region.
- 7D.** The backward RNN at position t has read the tokens after t ; for next-word prediction that leaks the answer (and is unavailable at generation time). In tagging the whole sentence is given, and the right context legitimately helps.
- 7E.** Copy: $\mathbf{f} = \mathbf{1}$, $\mathbf{i} = \mathbf{0}$. Overwrite: $\mathbf{f} = \mathbf{0}$, $\mathbf{i} = \mathbf{1}$ (then $\mathbf{c}_t = \mathbf{g}_t$). Initialise the forget-gate bias positive so that $\mathbf{f}$ starts near 1: the cell remembers by default and gradients flow through many steps early in training.
- 7F.** $V^T = 10^{80}$ sequences. Beam search scores $kV = 50,000$ extensions per step and keeps 5.

- 8A.** Query 1: scores $(1, 0)/\sqrt{2} = (0.707, 0)$, weights $(0.670, 0.330)$, output 13.30. Query 2: scores $(2, 2)/\sqrt{2}$, weights $(0.500, 0.500)$, output 15.00 (checked against PyTorch).
- 8B.** Attention $4 \cdot 65,536 + 1,024 = 263,168$; FFN $524,288 + 1,024 + 256 = 525,568$.
- 8C.** Unscaled: $\text{softmax}(8, 4, 0, -4) = (0.9817, 0.0180, 0.0003, 0.0000)$, essentially one-hot, so almost no gradient reaches keys 2–4. Scaled: $\text{softmax}(1, 0.5, 0, -0.5) = (0.455, 0.276, 0.167, 0.102)$, a soft distribution that still prefers key 1.
- 8D.** After splitting heads: **Q** $(2, 4, 5, 8)$, **K** and **V** $(2, 4, 9, 8)$, scores $(2, 4, 5, 9)$, per-head output $(2, 4, 5, 8)$, merged output $(2, 5, 32)$.
- 8E.** The decoder predicts token $t + 1$ from tokens $\leq t$ at every position simultaneously; without the mask position t could read token $t + 1$ and the loss would be trivial. BERT predicts masked-out tokens from both sides; the information to hide is removed from the input (the mask token), so all positions may attend to all others.
- 8F.** $2 \cdot 24 \cdot 16 \cdot 64 \cdot 2048 \cdot 8 \cdot 2$ bytes = 1.50 GiB. With 4 K/V heads it is a quarter, 0.375 GiB.
- 9A.** NLLs $(1.609, 0.693, 2.303)$, mean 1.535, so $\text{PPL} = e^{1.535} = (0.2 \cdot 0.5 \cdot 0.1)^{-1/3} = 4.64$.
- 9B.** $16 \cdot (1024 + 4096) = 81,920$, i.e. 1.95% of the 4,194,304 weights.
- 9C.** With $T = 0.5$ the logits become $(4, 2, 0)$ and the distribution is $(0.867, 0.117, 0.016)$. With $k = 1$: $(1, 0, 0)$. Top- p : $\text{softmax}(2, 1, 0) = (0.665, 0.245, 0.090)$; the first token alone has mass ≥ 0.5 , so $(1.000, 0.000, 0.000)$. Top- k with $k = 1$ and this top- p are deterministic (greedy).
- 9D.** LoRA: gradients and Adam moments only for the low-rank matrices (well under 1% of parameters), instead of for all 3B. Quantise the frozen base (e.g. 4-bit), cutting weight memory about four-fold from fp16. Gradient checkpointing ([chapter 3](#)) additionally cuts activation memory.
- 9E.** BERT was trained to fill masked tokens from both sides and never to model $p(x_t \mid x_{<t})$; it has no causal structure for sampling one token at a time. GPT's representation of a token sees only the left context, so early tokens of a sentence carry little information about the rest; a bidirectional encoder's representations use the whole sentence.
- 10A.** First: $\frac{1}{2}(0 + 4 - 1 - \ln 4) = 0.8069$. Second: $\frac{1}{2}(4 + 1 - 1 - 0) = 2.0$.
- 10B.** If this D is optimal, $p_{\text{data}} = p_g$ everywhere: the generator has matched the data (and $V = -\log 4$). In practice a constant D may just be a weak discriminator, so it is evidence, not proof.
- 10C.** Sampling is not a differentiable function of (μ, σ) . Writing $z = \mu + \sigma\varepsilon$ with $\varepsilon \sim \mathcal{N}(0, 1)$ independent of the parameters makes z differentiable in them: $\partial f(z)/\partial \mu = f'(z)$ and $\partial f(z)/\partial \sigma = f'(z)\varepsilon$. Discrete distributions (categorical, Bernoulli) have no such continuous reparameterisation; one uses the score-function (REINFORCE) estimator or continuous relaxations such as Gumbel-softmax.
- 10D.** $(224/14)^2 = 256$ patches plus one: 257 tokens; attention matrix 257×257 per head.
- 10E.** $\mathbf{x}_t = 0.5 \mathbf{x}_0 + \sqrt{0.75} \varepsilon \approx 0.5 \mathbf{x}_0 + 0.866 \varepsilon$; $\text{SNR} = 0.25/0.75 = 1/3$.
- 11A.** $(64, 64)$. Use `preds.squeeze(1)` (or `target.unsqueeze(1)`) so both are (64) or both $(64, 1)$.

- 11B.** The model is in training mode (dropout active, BatchNorm using batch statistics), or the evaluation data pipeline applies random augmentation. Call `model.eval()` and use deterministic preprocessing.

11C.

```
1 opt.zero_grad()
2 for x, y in micro_batches:           # 8 micro-batches of 8
3     (loss_fn(model(x), y) / 8).backward()
4 opt.step()
```

The gradients add up to the mean-loss gradient of the 64 examples. BatchNorm still normalises with statistics of 8 examples, so the result is not identical to a true batch of 64 (use GroupNorm/LayerNorm or accept the difference).

- 11D.** $1.3 \times 10^9 \cdot 16 \approx 20.8$ GB, plus activations.

12A.

```
1 class Sigmoid:
2     params = {}
3     def forward(self, x):
4         self.y = 1 / (1 + np.exp(-x))
5         return self.y
6     def backward(self, g):
7         return g * self.y * (1 - self.y)
```

Caching y rather than x saves recomputing the exponential.

- 12B.** With fancy indexing, `gE[idx] += g` is a buffered assignment: when the same row appears several times in `idx`, only one contribution survives. `np.add.at` performs an unbuffered accumulation, so every occurrence adds its gradient.
- 12C.** $64 \cdot 9 \cdot 56 \cdot 56 = 1,806,336$ versus $64 \cdot 56 \cdot 56 = 200,704$: nine times larger, the cost of turning convolution into a matrix product.

Bibliography

- [ArChBo17] MARTIN ARJOVSKY, SOUMITH CHINTALA, and LÉON BOTTOU. “Wasserstein Generative Adversarial Networks”. In: *International Conference on Machine Learning (ICML)*. 2017 (cited p. 103)
- [BaChBe15] DZMITRY BAHDANAU, KYUNGHYUN CHO, and YOSHUA BENGIO. “Neural Machine Translation by Jointly Learning to Align and Translate”. In: *International Conference on Learning Representations (ICLR)*. 2015 (cited p. 77)
- [BaKiHi16] JIMMY LEI BA, JAMIE RYAN KIROS, and GEOFFREY E. HINTON. Layer Normalization. In: *arXiv preprint arXiv:1607.06450*, (2016) (cited p. 51)
- [Ch+14] KYUNGHYUN CHO et al. “Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation”. In: *Proceedings of EMNLP*. 2014 (cited p. 72)
- [Ch+20] TING CHEN, SIMON KORNBLITH, MOHAMMAD NOROUZI, and GEOFFREY HINTON. “A Simple Framework for Contrastive Learning of Visual Representations”. In: *International Conference on Machine Learning (ICML)*. 2020 (cited p. 104)
- [Cy89] GEORGE CYBENKO. Approximation by Superpositions of a Sigmoidal Function. In: *Mathematics of Control, Signals and Systems*, **2**: (1989) (cited p. 17)
- [Da+22] TRI DAO, DANIEL Y. FU, STEFANO ERMON, ATRI RUDRA, and CHRISTOPHER RÉ. “FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness”. In: *Advances in Neural Information Processing Systems*. 2022 (cited p. 85)
- [De+19] JACOB DEVLIN, MING-WEI CHANG, KENTON LEE, and KRISTINA TOUTANOVA. “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding”. In: *Proceedings of NAACL-HLT*. 2019 (cited p. 94)
- [DoBeKo+21] ALEXEY DOSOVITSKIY, LUCAS BEYER, ALEXANDER KOLESNIKOV, et al. “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale”. In: *International Conference on Learning Representations (ICLR)*. 2021 (cited p. 105)
- [GIBe10] XAVIER GLOROT and YOSHUA BENGIO. “Understanding the Difficulty of Training Deep Feedforward Neural Networks”. In: *Proceedings of AISTATS*. 2010 (cited p. 47)
- [Go+14] IAN GOODFELLOW et al. “Generative Adversarial Nets”. In: *Advances in Neural Information Processing Systems*. 2014 (cited p. 102)
- [Gr+06] ALEX GRAVES, SANTIAGO FERNÁNDEZ, FAUSTINO GOMEZ, and JÜRGEN SCHMIDHUBER. “Connectionist Temporal Classification: Labelling Unsegmented Sequence Data with Recurrent Neural Networks”. In: *International Conference on Machine Learning (ICML)*. 2006 (cited p. 75)

- [He+15] KAIMING HE, XIANGYU ZHANG, SHAOQING REN, and JIAN SUN. “Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification”. In: *IEEE International Conference on Computer Vision (ICCV)*. 2015 (cited p. 47)
- [He+16] KAIMING HE, XIANGYU ZHANG, SHAOQING REN, and JIAN SUN. “Deep Residual Learning for Image Recognition”. In: *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2016 (cited p. 65)
- [HeGi16] DAN HENDRYCKS and KEVIN GIMPEL. Gaussian Error Linear Units (GELUs). In: *arXiv preprint arXiv:1606.08415*, (2016) (cited p. 23)
- [HoBoMe+22] JORDAN HOFFMANN, SEBASTIAN BORGEAUD, ARTHUR MENSCH, et al. “Training Compute-Optimal Large Language Models”. In: *Advances in Neural Information Processing Systems*. 2022 (cited p. 95)
- [HoJaAb20] JONATHAN HO, AJAY JAIN, and PIETER ABBEEL. “Denoising Diffusion Probabilistic Models”. In: *Advances in Neural Information Processing Systems*. 2020 (cited p. 103)
- [HoSc97] SEPP HOCHREITER and JÜRGEN SCHMIDHUBER. Long Short-Term Memory. In: *Neural Computation*, 9:8 (1997) (cited p. 71)
- [Hu+22] EDWARD J. HU et al. “LoRA: Low-Rank Adaptation of Large Language Models”. In: *International Conference on Learning Representations (ICLR)*. 2022 (cited p. 95)
- [IoSz15] SERGEY IOFFE and CHRISTIAN SZEGEDY. “Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift”. In: *International Conference on Machine Learning (ICML)*. 2015 (cited p. 49)
- [Ka+20] JARED KAPLAN et al. Scaling Laws for Neural Language Models. In: *arXiv preprint arXiv:2001.08361*, (2020) (cited p. 95)
- [KiBa15] DIEDERIK P. KINGMA and JIMMY BA. “Adam: A Method for Stochastic Optimization”. In: *International Conference on Learning Representations (ICLR)*. 2015 (cited p. 41)
- [KiWe14] DIEDERIK P. KINGMA and MAX WELLING. “Auto-Encoding Variational Bayes”. In: *International Conference on Learning Representations (ICLR)*. 2014 (cited p. 101)
- [Li+17] TSUNG-YI LIN, PRIYA GOYAL, ROSS GIRSHICK, KAIMING HE, and PIOTR DOLLÁR. “Focal Loss for Dense Object Detection”. In: *IEEE International Conference on Computer Vision (ICCV)*. 2017 (cited p. 27)
- [LoHu19] ILYA LOSHCHILOV and FRANK HUTTER. “Decoupled Weight Decay Regularization”. In: *International Conference on Learning Representations (ICLR)*. 2019 (cited p. 43)
- [Mi+13] TOMAS MIKOLOV, ILYA SUTSKEVER, KAI CHEN, GREG CORRADO, and JEFFREY DEAN. “Distributed Representations of Words and Phrases and their Compositionality”. In: *Advances in Neural Information Processing Systems*. 2013 (cited p. 105)
- [MiPa69] MARVIN MINSKY and SEYMOUR PAPERT. *Perceptrons*. MIT Press, 1969 (cited p. 15)

- [OuWuJi+22] LONG OUYANG, JEFF WU, XU JIANG, et al. “Training Language Models to Follow Instructions with Human Feedback”. In: *Advances in Neural Information Processing Systems*. 2022 (cited p. 95)
- [PrSmLe22] OFIR PRESS, NOAH A. SMITH, and MIKE LEWIS. “Train Short, Test Long: Attention with Linear Biases Enables Input Length Extrapolation”. In: *International Conference on Learning Representations (ICLR)*. 2022 (cited p. 81)
- [Ra+23] RAFAEL RAFAILOV, ARCHIT SHARMA, ERIC MITCHELL, STEFANO ERMON, CHRISTOPHER D. MANNING, and CHELSEA FINN. “Direct Preference Optimization: Your Language Model is Secretly a Reward Model”. In: *Advances in Neural Information Processing Systems*. 2023 (cited p. 95)
- [RaKiHa+21] ALEC RADFORD, JONG WOOK KIM, CHRIS HALLACY, et al. “Learning Transferable Visual Models From Natural Language Supervision”. In: *International Conference on Machine Learning (ICML)*. 2021 (cited p. 104)
- [Ro58] FRANK ROSENBLATT. The Perceptron: A Probabilistic Model for Information Storage and Organization in the Brain. In: *Psychological Review*, **65**:6 (1958) (cited p. 12)
- [RoFiBr15] OLAF RONNEBERGER, PHILIPP FISCHER, and THOMAS BROX. “U-Net: Convolutional Networks for Biomedical Image Segmentation”. In: *Medical Image Computing and Computer-Assisted Intervention (MICCAI)*. 2015 (cited p. 68)
- [RuHiWi86] DAVID E. RUMELHART, GEOFFREY E. HINTON, and RONALD J. WILLIAMS. Learning Representations by Back-Propagating Errors. In: *Nature*, **323**: (1986) (cited p. 29)
- [SeHaBi16] RICO SENNRICH, BARRY HADDOW, and ALEXANDRA BIRCH. “Neural Machine Translation of Rare Words with Subword Units”. In: *Proceedings of ACL*. 2016 (cited p. 93)
- [Sr+14] NITISH SRIVASTAVA, GEOFFREY HINTON, ALEX KRIZHEVSKY, ILYA SUTSKEVER, and RUSLAN SALAKHUTDINOV. Dropout: A Simple Way to Prevent Neural Networks from Overfitting. In: *Journal of Machine Learning Research*, **15**: (2014) (cited p. 52)
- [Va+17] ASHISH VASWANI et al. “Attention Is All You Need”. In: *Advances in Neural Information Processing Systems*. 2017 (cited p. 77)